



TM471

ACOPOS function blocks

Requirements

Training modules	TM416 - Motion control: Basic functions TM417 - Motion control: Axis coupling TM470 - Motion control: Axis coupling - Cam Automat
Software	Automation Studio 4.6 or higher mapp Motion Technology Package 5.15 or higher
Hardware	X20 controller ACOPOS servo family / ARsim

Table of contents

1 Introduction.....	4
1.1 Learning objectives.....	5
1.2 Document structure and handling.....	6
2 Road to success.....	7
2.1 Architecture and design.....	7
2.2 Engineering.....	12
2.3 Diagnostics.....	14
2.4 Information for participants.....	15
3 Function block collection 1.....	16
3.1 VAR - Parameter variables.....	16
3.2 ARITH - Arithmetic operation.....	16
3.3 MUX - Multiplexer.....	18
3.4 Group exercise: Virtual encoder emulation - Phase1.....	19
4 Function block collection 2.....	21
4.1 CMP - Comparator.....	21
4.2 EVWR - Event controlled parameter writing.....	23
4.3 LOGIC - Logical operation.....	25
4.4 DELAY - Delay time.....	26
4.5 Group exercise: Virtual encoder emulation - Phase 2.....	27
5 Function block collection 3.....	28
5.1 MPGEN - Motion profile generator.....	28
5.2 LATCH - Event-triggered position capture.....	31
5.3 Group exercise: Registration mark correction.....	36
6 Function block collection 4.....	37
6.1 CAMCON - Cam control.....	37
6.2 Task/Exercise.....	39
6.3 Group exercise: Flying saw.....	40
6.4 Group exercise: Torque limiter.....	41
7 Additional collections.....	42
7.1 Further function blocks.....	42
7.2 Not frequently used function blocks.....	46
7.3 Hardware-dependent.....	55
8 Solutions of group exercise.....	57
8.1 Exercise: Encoder emulation.....	57
8.2 Exercise: Registration mark correction.....	60
8.3 Exercise: Flying saw.....	62
8.4 Exercise: Torque limiter.....	66
8.5 Solution - Task/Exercise.....	75
9 Summary.....	84

1 Introduction

ACOPOS function blocks are freely configurable technology function blocks (PID control, arithmetic operations, set value generation, handling of digital and analog IOs on the drive, etc.) those are executed in real-time directly on the drive.

Using ACOPOS function blocks, the application engineer can influence, modify and extend the control chain in the drive. ACOPOS function blocks should be used for special purpose motion applications that require ultra-fast response times. Compared to doing the calculations on the PLC, the response times can be cut down significantly by avoiding the network time to and from the PLC, instead doing all calculations directly on the drive completely in sync with the ACOPOS firmware and operating system.

ACOPOS function blocks has produced amazing results in numerous series production machines with ACOPOS servo drives by tremendously cutting down production times through its options and reaction times in the sub-millisecond range, for example:

- Injection molding: Switch over from speed to torque control based on an analog pressure signal wired directly to the drive
- Packaging: Detecting registration marks on paper or other material for highly precise cut length correction (see application example below)

Application example: Registration mark evaluation/control

A sensor on the "trigger1" input of the ACOPOS servo drive detects registration mark signals. The position of the carrier tape is evaluated using an encoder position. When a positive edge of the registration mark signal occurs, the encoder position should be latched (saved) and an offset should be added. The result can then be used for further processing in the drive, e.g. cam automat parametrization or subsequent ACOPOS function blocks.



Figure 1: Detecting registration marks for precise cut length correction

Solution sketch using LATCH, ARITH and VAR, which are included with the ACOPOS function blocks. LATCH saves a snapshot of the external encoder position when a positive edge of the "trigger1" input occurs; ARITH adds an offset to it, that is saved in a VAR ("variable").

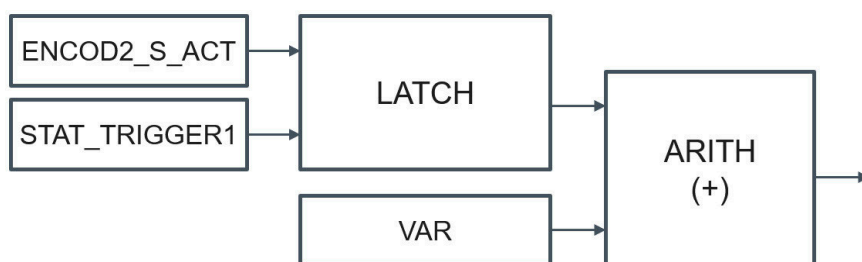


Figure 2: Registration mark solution sketch

The purpose of this training module is to bring the engineers closer to ACOPOS function blocks with the help of application examples so they get a "feeling" for the types of applications for which ACOPOS function blocks can be beneficial and boost application performance.

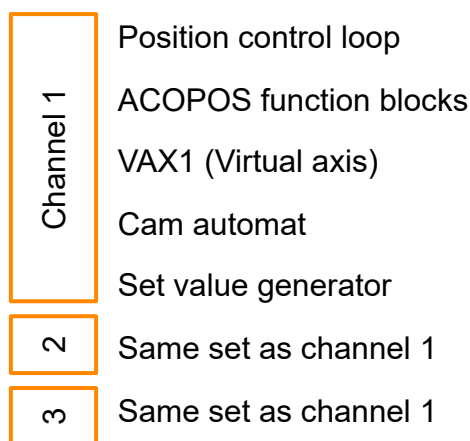
Additionally, the full range of function blocks is explained so engineers will be able to design smart and efficient ACOPOS function block solutions on B&R ACOPOS servo drives.

Computing time and calculation sequence

Cyclic processing of the function block is inserted when the "Create" call is made. This allows the calculation sequence to be determined. This should correspond to the direction of the data flow (from the "creator" to the "user") to prevent unnecessary delays. The function block processing cycle corresponds to a position controller cycle / set value generator cycle of 400 µs. Some operating system functions are executed before the function blocks (data acquisition, network communication) and some after the function blocks (cam automat, set value generator, VAX1). Total computing time overflow is monitored and, if an error occurs, an error message is displayed. For better understanding, an image is shown below illustrating the execution order inside a drive with multiple channels.

Network communication

Data acquisition



Resources available on the drive

ACOPOS function blocks are a limited resource on the ACOPOS drive. Hence, the user should utilize them wisely. The user can implement up to 8 instances (0..7) of each function block. These function blocks are shared resources between real and virtual axes but those are not shared between channels. Let's look at an example: The user has an ACOPOS P3 with 3 channels, which means that the drive has total 24 instances of each function block making 8 instances for each channel. So, each channel has separate set of function blocks.

1.1 Learning objectives

This training module explains ACOPOS function blocks. Numerous exercises help solidify participant's understanding. In addition, this training module will frequently refer to the extensive help documentation "Automation Help", an invaluable reference for completing the exercises here.

- Participants will become familiar with the available resources for ACOPOS function blocks.
- Participants will find out the response/reaction times of ACOPOS function blocks.
- Participants will be able to implement and use different ACOPOS function blocks.
- Participants will learn about sequencing of ACOPOS function blocks in order to specify execution priority.
- Participants will learn that there are many applications that require extensive usage of ACOPOS function blocks and the immediate action that needs to be performed.
- Participants will become familiar with high-speed operation of digital inputs/outputs on the drive.
- Participants will find out about use-cases and gain deep insight into ParIDs and use them with ACOPOS function blocks.

1.2 Document structure and handling

The training manual is split into several parts including so called "function block collections".

With the help of this structure, it is possible to have condensed content for participants. At the end of the sections, exercises also combine the described ACOPOS function blocks to represent customer demands.

2 Road to success

As with all successful applications, the first step is collecting requirements and specifying the functionality that is expected. This is indispensable for creating a properly working architecture and design that will facilitate implementation afterwards.

2.1 Architecture and design

Since the configuration of ACOPOS function blocks refers to text-based table representation, we highly recommend also drawing and maintaining a graphical representation. These diagrams will provide you with a global overview and also speed up implementation at a later stage.

The most useful representation is derived from function blocks, which are well-known from using Automation Studio and Automation Help, including input and output interfaces connected with variables and parameters.

Tooling

Many different tools are available for creating these diagrams. For this training course, a tool that is free of charge is going to be used.

2.1.1 Configuration methods

For "ACOPOS function blocks" training course, function blocks are used in planning phase for illustration purpose. For efficient and error-free implementation, it is necessary to start with a proper design and get informed for the different **configuration methods** including their pros and cons.

To illustrate the different configuration methods, the following two samples are used to compare and describe the various aspects.

Sample - Finding the minimum and maximum lag error

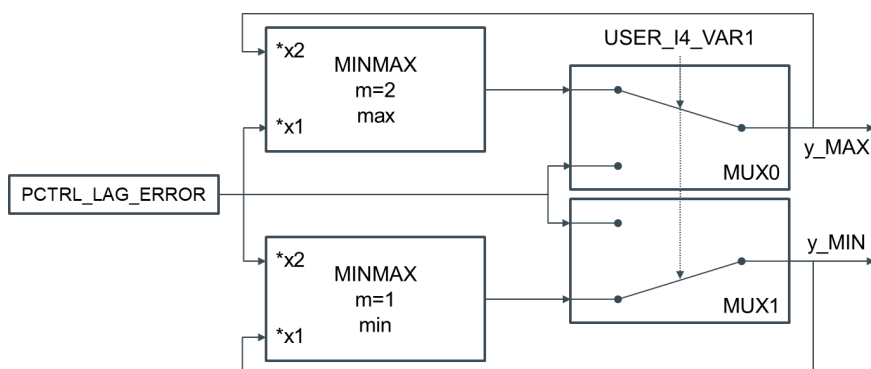


Figure 3: Sample: Finding the minimum and maximum lag error

Sample - Limiting the value range

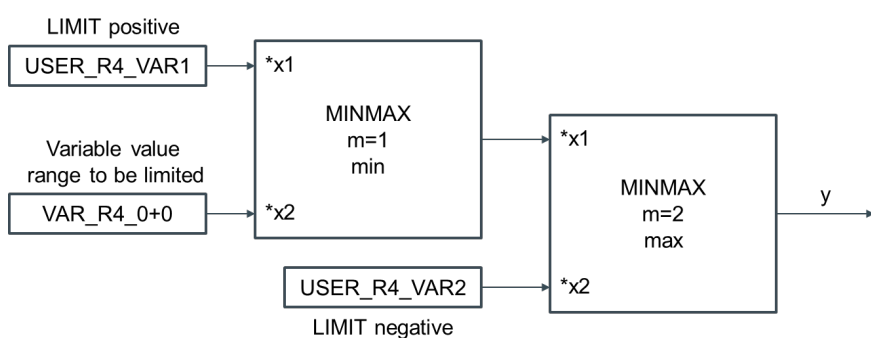


Figure 4: Sample: Limiting the value range

The next section provides a detailed description of possible designs and approaches for the configuration sequence. It is important to mention the direct impact on the ACOPOS parameter table. Depending on the chosen method, one or more parameter tables can be configured and assigned to the drive.

Generally, it is possible to freely place line after line in the parameter table and for some lines have a special effect, e.g. execution order is determined by **FUNCTION_BLOCK_CREATE**; all other general constraints, e.g. **FUNCTION_BLOCK_CREATE**, must be called before configuring the functions blocks so the ACOPOS firmware can allocate memory and initiate the procedure required to perform function block execution.

2.1.1.1 Method: All at once

With this configuration method, all function blocks are created at once. It is quite difficult to diagnose when the user is not getting the desired results and the user is performing an execution order check. Execution order is based on the sequence of the function blocks created. So, the user must consider the way the function block sequence is organized and not the sequence order in which function block parameter passing is done. The user should avoid this method when the execution order is most important to the application.

With this method, the user may find it easy to see all the function blocks created at once and possibly the execution order. The user must consider execution order when reorganizing the sequence of function block creation.

Creating all function blocks and determining the computing sequence

FUNCTION_BLOCK_CREATE	← MINMAX_MODE+0	Creating MINMAX FB0
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+1	Creating MINMAX FB1
FUNCTION_BLOCK_CREATE	← MUX_MODE+0	Creating MUX FB0
FUNCTION_BLOCK_CREATE	← MUX_MODE+1	Creating MUX FB1
FUNCTION_BLOCK_CREATE	← VAR_I4_0+0	Creating VAR FB0
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+2	Creating MINMAX FB2
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+3	Creating MINMAX FB3

Configuring and defining the connection structure of the MINMAX FB0

MINMAX_IN1_PARID+0	← PCTRL_LAG_ERROR	Link input1 with lag error
MINMAX_IN2_PARID+0	← MUX_VALUE_R4+0	Link input2 to MUX output
MINMAX_MODE+0	← 2	Enable FB in mode MAXIMUM

Configuring and defining the connection structure of the MINMAX FB1

MINMAX_IN1_PARID+1	← PCTRL_LAG_ERROR	Link input1 with lag error
MINMAX_IN2_PARID+1	← MUX_VALUE_R4+1	Link input2 to MUX output
MINMAX_MODE+1	← 1	Enable FB in mode MINIMUM

Configuring and defining the connection structure of the MUX FB0

MUX_SELECTOR_PARID+0	← USER_I4_VAR1	Link selector input with control variable
MUX_SELECTOR_MAX+0	← 1	Selector values 0,1
MUX_IN0_PARID+0	← PCTRL_LAG_ERROR	Link input0 with lag error
MUX_IN1_PARID+0	← MINMAX_VALUE_R4+0	Link input1 to MAX output
MUX_MODE+0	← 1	Enable FB in active as SWITCH

Configuring and defining the connection structure of the MUX FB1

MUX_SELECTOR_PARID+1	← USER_I4_VAR1	Link selector input with control variable
MUX_SELECTOR_MAX+1	← 1	Selector values 0,1
MUX_IN0_PARID+1	← PCTRL_LAG_ERROR	Link input0 with lag error

MUX_IN1_PARID+1	← MINMAX_VALUE_R4+1	Link input1 to MIN output
MUX_MODE+1	← 1	Enable FB in active as SWITCH

Configuring and defining VAR FB0

VAR_R4_0+0	← 10000.0	Maximum limit range
VAR_R4_1+0	← -10000.0	Minimum limit range

Configuring and defining the connection structure of the MINMAX FB2

MINMAX_IN1_PARID+2	← VAR_R4_0+0	Link input1 with maximum limit range
MINMAX_IN2_PARID+2	← USER_R4_VAR2	Link input2 with control variable
MINMAX_MODE+2	← 1	Enable FB in mode MINIMUM

Configuring and defining the connection structure of the MINMAX FB3

MINMAX_IN1_PARID+3	← VAR_R4_1+0	Link input1 with minimum limit range
MINMAX_IN2_PARID+3	← MINMAX_VALUE_R4+2	Link input2 with MIN output
MINMAX_MODE+3	← 2	Enable FB in mode MAXIMUM

2.1.1.2 Method: Per Function Unit

With this configuration method, the user can create a function block and pass the parameter based on the functional unit. This method is useful when the user is planning a modular approach for reusability. However, this method still does not provide full flexibility in term of reorganizing execution priority.

With this method, all function blocks are created at once according to the functional unit and the user must focus on this while reorganizing execution order.

Finding minimum and maximum of lag error

Creating all function blocks and determining the computing sequence		
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+0	Creating MINMAX FB0
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+1	Creating MINMAX FB1
FUNCTION_BLOCK_CREATE	← MUX_MODE+0	Creating MUX FB0
FUNCTION_BLOCK_CREATE	← MUX_MODE+1	Creating MUX FB1
Configuring and defining the connection structure of the MINMAX FB0		
MINMAX_IN1_PARID+0	← PCTRL_LAG_ERROR	Link input1 with lag error
MINMAX_IN2_PARID+0	← MUX_VALUE_R4+0	Link input2 to MUX output
MINMAX_MODE+0	← 2	Enable FB in mode MAXIMUM
Configuring and defining the connection structure of the MINMAX FB1		
MINMAX_IN1_PARID+1	← PCTRL_LAG_ERROR	Link input1 with lag error
MINMAX_IN2_PARID+1	← MUX_VALUE_R4+1	Link input2 to MUX output
MINMAX_MODE+1	← 1	Enable FB in mode MINIMUM
Configuring and defining the connection structure of the MUX FB0		
MUX_SELECTOR_PARID+0	← USER_I4_VAR1	Link selector input with control variable
MUX_SELECTOR_MAX+0	← 1	Selector values 0,1
MUX_IN0_PARID+0	← PCTRL_LAG_ERROR	Link input0 with lag error
MUX_IN1_PARID+0	← MINMAX_VALUE_R4+0	Link input1 to MAX output

MUX_MODE+0	← 1	Enable FB in active as SWITCH
Configuring and defining the connection structure of the MUX FB1		
MUX_SELECTOR_PARID+1	← USER_I4_VAR1	Link selector input with control variable
MUX_SELECTOR_MAX+1	← 1	Selector values 0,1
MUX_IN0_PARID+1	← PCTRL_LAG_ERROR	Link input0 with lag error
MUX_IN1_PARID+1	← MINMAX_VALUE_R4+1	Link input1 to MIN output
MUX_MODE+1	← 1	Enable FB in active as SWITCH
Limiting value range		
Creating all function blocks and determining the computing sequence		
FUNCTION_BLOCK_CREATE	← VAR_I4_0+0	Creating VAR FB0
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+2	Creating MINMAX FB2
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+3	Creating MINMAX FB3
Configuring and defining VAR FB0		
VAR_R4_0+0	← 10000.0	Maximum limit range
VAR_R4_1+0	← -10000.0	Minimum limit range
Configuring and defining the connection structure of the MINMAX FB2		
MINMAX_IN1_PARID+2	← VAR_R4_0+0	Link input1 with maximum limit range
MINMAX_IN2_PARID+2	← USER_R4_VAR2	Link input2 with control variable
MINMAX_MODE+2	← 1	Enable FB in mode MINIMUM
Configuring and defining the connection structure of the MINMAX FB3		
MINMAX_IN1_PARID+3	← VAR_R4_1+0	Link input1 with minimum limit range
MINMAX_IN2_PARID+3	← MINMAX_VALUE_R4+2	Link input2 with MIN output
MINMAX_MODE+3	← 2	Enable FB in mode MAXIMUM

2.1.1.3 Method: Discrete

This method is flexible and easy to adapt when the user wants to reorganize execution priority. It is easy to follow and the user can see that function block has been created just above parameter passed for function block configuration. It is also helpful in terms of modularity, flexibility and observing execution order. The user can reorganize execution order by shifting an entire package consisting of function block creation and parameter passing. The only drawback of this method is that the user cannot use a function block output before the function block is created.

Limitation to a value range

Configuring and defining VAR FB0		
FUNCTION_BLOCK_CREATE	← VAR_I4_0+0	Creating VAR FB0
VAR_R4_0+0	← 10000.0	Maximum limit range
VAR_R4_1+0	← -10000.0	Minimum limit range
Configuring and defining the connection structure of the MINMAX FB2		
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+2	Creating MINMAX FB2
MINMAX_IN1_PARID+2	← VAR_R4_0+0	Link input1 with maximum limit range
MINMAX_IN2_PARID+2	← USER_R4_VAR2	Link input2 with control variable

MINMAX_MODE+2	← 1	Enable FB in mode MINIMUM
Configuring and defining the connection structure of the MINMAX FB3		
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+3	Creating MINMAX FB3
MINMAX_IN1_PARID+3	← VAR_R4_1+0	Link input1 with minimum limit range
MINMAX_IN2_PARID+3	← MINMAX_VALUE_R4+2	Link input2 with MIN output
MINMAX_MODE+3	← 2	Enable FB in mode MAXIMUM

2.1.1.4 Method: Hybrid

This is a mixture of the function block creation per functional unit and discrete methods. This method breaks down each functional package into the micro level where it provides the possibility for a discrete level of function block creation just above function block parameter passing and per functional unit where the function block output is used before the parameter is passed for function block configuration.

This method can provide extensive flexibility, reusability, readability and modularity. The user can reorganize execution order easily since function block creation and parameter passing are done as far in the micro level as possible.

Minimum & Maximum of Lag error

Maximum of past the time		
Creating all function blocks and determining the computing sequence		
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+0	Creating MINMAX FB0
FUNCTION_BLOCK_CREATE	← MUX_MODE+0	Creating MUX FB0
Configuring and defining the connection structure of the MINMAX FB0		
MINMAX_IN1_PARID+0	← PCTRL_LAG_ERROR	Link input1 with lag error
MINMAX_IN2_PARID+0	← MUX_VALUE_R4+0	Link input2 to MUX output
MINMAX_MODE+0	← 2	Enable FB in MAXIMUM mode
Configuring and defining the connection structure of the MUX FB0		
MUX_SELECTOR_PARID+0	← USER_I4_VAR1	Link selector input with control variable
MUX_SELECTOR_MAX+0	← 1	Selector values 0,1
MUX_IN0_PARID+0	← PCTRL_LAG_ERROR	Link input0 with lag error
MUX_IN1_PARID+0	← MINMAX_VALUE_R4+0	Link input1 to MAX output
MUX_MODE+0	← 1	Enable FB in active as SWITCH
Minimum of past the time		
Creating all function blocks and determining the computing sequence		
FUNCTION_BLOCK_CREATE	← MINMAX_MODE+1	Creating MINMAX FB1
FUNCTION_BLOCK_CREATE	← MUX_MODE+1	Creating MUX FB1
Configuring and defining the connection structure of the MINMAX FB1		
MINMAX_IN1_PARID+1	← PCTRL_LAG_ERROR	Link input1 with lag error
MINMAX_IN2_PARID+1	← MUX_VALUE_R4+1	Link input2 to MUX output
MINMAX_MODE+1	← 1	Enable FB in mode MINIMUM
Configuring and defining the connection structure of the MUX FB1		
MUX_SELECTOR_PARID+1	← USER_I4_VAR1	Link selector input with control variable
MUX_SELECTOR_MAX+1	← 1	Selector values 0,1

	MUX_IN0_PARID+1	← PCTRL_LAG_ERROR	Link input0 with lag error
	MUX_IN1_PARID+1	← MINMAX_VALUE_R4+1	Link input1 to MIN output
	MUX_MODE+1	← 1	Enable FB in active as SWITCH

Limiting value range

Configuring and defining VAR FB0			
	FUNCTION_BLOCK_CREATE	← VAR_I4_0+0	Creating the VAR FB0
	VAR_R4_0+0	← 10000.0	Maximum limit range
	VAR_R4_1+0	← -10000.0	Minimum limit range
Configuring and defining the connection structure of the MINMAX FB2			
	FUNCTION_BLOCK_CREATE	← MINMAX_MODE+2	Creating the MINMAX FB2
	MINMAX_IN1_PARID+2	← VAR_R4_0+0	Link input1 with maximum limit rang
	MINMAX_IN2_PARID+2	← USER_R4_VAR2	Link input2 with control variable
	MINMAX_MODE+2	← 1	Enable FB in mode MINIMUM
Configuring and defining the connection structure of the MINMAX FB3			
	FUNCTION_BLOCK_CREATE	← MINMAX_MODE+3	Creating the MINMAX FB3
	MINMAX_IN1_PARID+3	← VAR_R4_1+0	Link input1 with minimum limit range
	MINMAX_IN2_PARID+3	← MINMAX_VALUE_R4+2	Link input2 with MIN output
	MINMAX_MODE+3	← 2	Enable FB in mode MAXIMUM

Recommendation



Users are free to choose the method that they would like to use and that is convenient for them. However, we recommend users using the hybrid method based on the opinion of application engineers and use cases. This is because it has more advantages than other methods.

Advantages of the **Method - Hybrid** are

- Easy reordering and reorganizing
- Easy modularity
- Easy navigation through execution order

2.2 Engineering

Engineering for the ACOPOS function blocks is done in Automation Studio and has some prerequisites.

In addition to the Automation Studio project and the added drives and motors, some settings need to be made so that ACOPOS function blocks can be put to use for ACOPOS drives.

2.2.1 Drive features and setup

Cam profile

Add **Cam** from "Toolbox - Object Catalog" with description "Cam for mapp Motion" if mapp Motion is being used and the user wants to use it in ACOPOS function blocks.



Cam for mapp Motion

Parameter table

Add **ACOPOS parameter table** from "Toolbox - Object Catalog" with description "ACOPOS parameter table for mapp Motion" if mapp Motion is being used and the user wants to configure ACOPOS function blocks.

 ACOPOS Parameter Table ACOPOS Parameter Table for mapp Motion

Axis feature

Add **Axis feature** in the mapp Motion package in the Automation Studio Configuration View to download the cam profile and ACOPOS parameter table.

 Axis Feature Axis feature configuration file

After adding **Axis feature**, the user should find the options for the axis feature below and then select related feature as shown below.

Cam automat	Name	Value	Unit	Description
Cam list	FeatureCamList			
Profile generator	Type	Cam list		Type of the axis feature
Analog inputs	Cam list reference	AcpCamList		Name of the cam list
Digital cam switch				
ACOPOS parameter table				
Mechanical deviation compensation				
Alternative value source				
Brake				
Digital output				

	Name	Value	Unit	Description
	FeatureAcpParTable			
	Type	ACOPOS parameter table		Type of the axis feature
	ACOPOS parameter table reference	AcpParTab		Name of the ACOPOS parameter table

Channel feature

Configure the desired **Axis feature** in the drive configuration as a channel configuration, e.g. Cam list feature and ACOPOS parameter table for ACOPOS function blocks. It is important to choose correct order of ACOPOS feature to download to the ACOPOS. Using the incorrect order of the ACOPOS feature to download to the ACOPOS, it could lead to an error when the PLC transfers respective ACOPOS feature to the ACOPOS drive.

Name	Value	Unit	Description
80VD100PD.C000-01			
POWERLINK configuration			
Drive configuration			
Module			Contains parameters which influence
Encoder			
Channel 1			
Channel 2			
Real axis			
Virtual axis	Not used		
Channel features			Features for the channel of a module
Feature reference 1	FeatureCamList		Name of the axis feature reference
Feature reference 2	FeatureAcpParTable		Name of the axis feature reference
Feature reference 3			Name of the axis feature reference

2.2.2 Mechanical to electrical configuration

There is an essential point that must be considered when it comes to axis configuration.

The example shown below illustrates the impact of measurement units and resolution internally on the drive system.

Axis configuration - Configuration View in Automation Studio

Measurement unit = Degrees (generally 360°)

Measurement resolution = 0.01 measurement units

Drive configuration - Physical View in Automation Studio

Gearbox:

Input = 1 revolutions

Output = 1 revolutions

Rotary to linear transformation:

Reference distance = 1 Measurement units/gearbox output revolution

Actual rotary to linear transformation internally to axis:

= Measurement unit * 1 / Measurement resolution

$$= 360^\circ \cdot 1 / 0.01$$

$$= 36000 \text{ counts}$$

Observation to participants

Participants will find 1 revolution of motor as 360.00 (with measurement resolution in two digits after decimal) on the PLC. However, inside to drive system, it is 36000 for 1 revolution of the motor according to the calculation shown above.

2.2.3 Working with mapp Cockpit

Automation Help



Diagnostics and service \ mapp Cockpit \ Getting started with mapp Cockpit \

- Interacting with mapp components in the web-based mapp Cockpit HMI application
- Tracing an ACOPOS data point in the web-based HMI application
- Configuring a trace of a process variable in the web-based HMI application

One time reading or writing value of desired ParID

- Change the value of ParID using **Write ParID** command from **common** for the desired axis, which is available in **Component Overview** in mapp Cockpit.
- Read the value of ParID using **Read ParID** command from **common** of desired axis, which is available in **Component Overview** in mapp Cockpit.

Graphical representation of value of desired ParID

- Configure desired ParIDs as **Data Point** in the **trace configuration** available in the **Trace Overview** in mapp Cockpit.
- Focus on the ACOPOS sampling time, especially, while working with ACOPOS function block. Recommended ACOPOS sampling time is 0.0004 seconds (400 μ s) when observing value changes in each ACOPOS scan cycle.
- Configure a trigger point if required, especially, when the user has a limited time frame for tracing ACOPOS data points due to limited buffer memory. This would avoid randomness and result in the user getting consistent tracing of the ParID value from the desired starting point.
- If the user wants to have a trigger data point as an ACOPOS ParID, use the same data point format as with tracing the ACOPOS data point. For ease of understanding, an example is shown below of the ACOPOS ParIDs accepted in mapp Cockpit as an ACOPOS data point.



2.3 Diagnostics

Diagnostics is a very crucial element when it comes to fault finding. For diagnostics, the following different possibilities are available to debug problems.

Automation Studio

Network command trace can be used to get information about exchanging a command and status between the PLC and the ACOPOS drive system. Commands are generally for movement, ParID reading and writing, etc. Status values are generally for updates related to movement completion, ParID values if reading is done, error information, etc. Network command trace is useful while debugging the creation order of ACOPOS function blocks and parameter configurations.

Motion logger is used to get information about events that occurred during exchange between the PLC task and drive. Events are categorized in different severity levels as shown below.

- Success
- Information
- Warning
- Error

mapp Cockpit

The **mapp Motion drive log** is same as network command trace in mapp Cockpit to provide tool-free support in a web browser with an added filtering mechanism to drill down to an individual axis, drive node number and much more.

Parameter trace can be used for getting information about the behavior of ACOPOS function blocks at runtime and other axis parameters such as actual and set positions, actual and set velocity, actual quadrature and direct current, actual torque with a very high sampling rate down to 400 μ s or even less than 400 μ s.

2.4 Information for participants

Abbreviations

Short text	Full text
FB	Function block
Acc	Acceleration
Dec	Deceleration
Trq	Torque
ParID	Parameter ID
Min	Minimum
Max	Maximum

Automation Help

Participant must read Automation Help to get information about ACOPOS function blocks for the reasons shown below.

- Input and output ParIDs
- ParID data types and units
- Access level (read and write options)
- Online parameter changes
- Acceptable value range for inputs
- Possible value range for output
- Supported data types
- Behavior of outputs based on selected mode and function block configuration

Group exercise

Participants should first do group brainstorming for connecting function block inputs and outputs either offline or on paper so you get the desired results. The rest will consist of implementation in the ACOPOS parameter table in Automation Studio and tracing the desired ParIDs.

3 Function block collection 1

3.1 VAR - Parameter variables

Automation Help: Examples



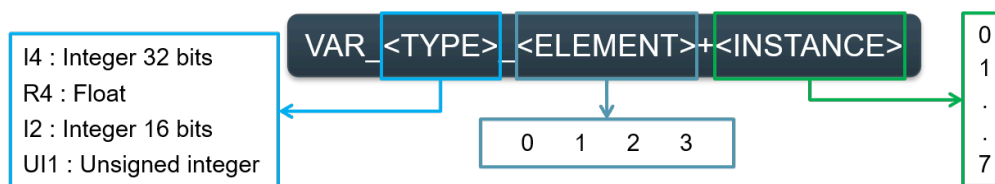
Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \
ACOPOS Function Blocks \ VAR - Parameter Variables \ Example

About the function block

Free parameter variables are needed to exchange data with a PLC task and to make connections using function blocks. These user-defined parameters are used to set function block inputs to any values. If the predefined parameter IDs (USER_I4_VAR1, USER_I4_VAR2, USER_R4_VAR1, USER_R4_VAR2) are not sufficient, another 16 variables can be generated for every VAR function block.

After creating the function block, four variables are provided from each of these data types: Integer32, Float, Integer16 and unsigned Integer8. These variables can be set to any value. The VAR function block does not have any cyclic functions.

The following is to be noted when defining the connection structure on position inputs: Only Integer32 variables have the proper overflow behavior and can therefore be used for **cyclic axes** and **endless positioning**. Integer16 and Float variables can only be used for a limited **movement range**. Integer8 variables are useful to save space when transferring data on the **cyclic user channel** (data to and from the drive).



The ParIDs CONST_I4_ONE (for logical 1) and CONST_I4_ZERO (for logical 0) are also available as predefined constants if required to use in the ACOPOS parameter table for logical operation or inputs to the function block.

3.2 ARITH - Arithmetic operation

Automation Help: Examples

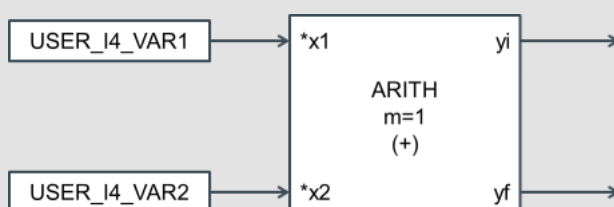


Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \
ACOPOS Function Blocks \ ARITH - Arithmetic Operation \ Example

About the function block

This function block uses an arithmetic function to link two input values. This function block provides different modes such as addition, subtraction, multiplication, addition with weighing factors and float division. This function block can also be used for adjusting INT32 overflow.

Exercise: Simple addition, subtraction, multiplication and division (live)



Parameter passing

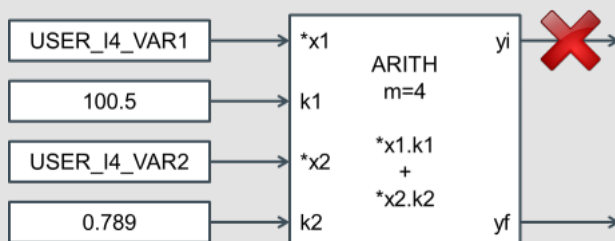
Creating, configuring and defining the connection structure of ARITH FBO

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Creating ARITH FBO
ARITH_IN1_PARID+0	3592	USER_I4_VAR1	Link input1 with User I4 variable 1
ARITH_IN2_PARID+0	3600	USER_I4_VAR2	Link input1 with User I4 variable 2
ARITH_MODE+0	3584	1	Enable FB in mode ADDITION

Steps to follow during the exercise

- Change value of USER_I4_VAR1 (584) and USER_I4_VAR2 (585) in mapp Cockpit and observe ARITH_VALUE_I4+0 (3608) and ARITH_VALUE_R4+0 (3624) either in trace or with Read ParID.
- Change value of ARITH_MODE+0 (3584) from 1(addition) to 2 (subtraction), 3 (multiplication) and 5 (floating division).
- During the course of changing mode, repeat the first step.

Exercise: Addition with weighting factors (self-paced)



Parameter passing

Creating, configuring and defining the connection structure of ARITH FBO

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Creating ARITH FBO
ARITH_IN1_PARID+0	3592	USER_I4_VAR1	Link input1 with User I4 variable 1
ARITH_IN2_PARID+0	3600	USER_I4_VAR2	Link input1 with User I4 variable 2
ARITH_K1+0	3632	100.5	Multiplication factor 1
ARITH_K2+0	3640	0.789	Multiplication factor 2
ARITH_MODE+0	3584	4	Enable FB in addition with weighting factors

Steps to follow during the exercise

- Change value of USER_I4_VAR1 (584), USER_I4_VAR2 (585), ARITH_K1+0 (3632) and ARITH_K2+0 (3640) in mapp Cockpit and observe ARITH_VALUE_I4+0 (3608) and ARITH_VALUE_R4+0 (3640) either in trace or with Read ParID.
- Observe value of ARITH_VALUE_I4+0 (3608), even though it is not supported in mode 4 (Addition with weighting factors)

3.3 MUX - Multiplexer

Automation Help: Examples



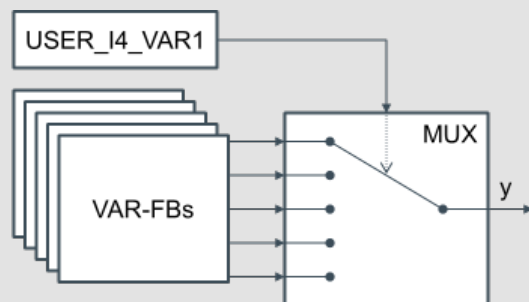
Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ MUX - Multiplexer \ Example

About the function block

The MUX function block selects one of given inputs and connects it through to the output. The selection is controlled via a **selector input**. The method of switching can be set using mode m. This function block provides different methods of switching configuration modes as shown below. Here, we are mainly focusing on mode **active as switch**.

- Active as switch
- With offset compensation
- With OFFSET_K ramp
- With OFFSET_T ramp

Exercise: Parameter switching based on user variable value (live)



Parameters passing

Creating and configuring VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Creating VAR FB0
VAR_R4_0+0	4128	12345.123	Passing value R4_0
VAR_R4_1+0	4136	54321.321	Passing value R4_1
VAR_R4_2+0	4144	98765.987	Passing value R4_2
VAR_R4_3+0	4152	56789.567	Passing value R4_3

Creating and configuring VAR FB1

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+1	Creating VAR FB1
VAR_R4_0+1	4129	11111.111	Passing value R4_0
VAR_R4_1+1	4137	22222.222	Passing value R4_1
VAR_R4_2+1	4145	33333.333	Passing value R4_2
VAR_R4_3+1	4153	44444.444	Passing value R4_3

Creating, configuring and defining the connection structure of MUX FB0

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Creating MUX FB0
MUX_SELECTOR_PARID+0	11272	USER_I4_VAR1	Link selector input with User I4 Variable 1
MUX_SELECTOR_MAX+0	11280	7	Selector values 0..7
MUX_IN0_PARID+0	11320	VAR_R4_0+0	Link input 0 with Float parameters 0 VAR-FB0

MUX_IN1_PARID+0	11328	VAR_R4_1+0	Link input 1 with Float parameters 1 VAR-FB0
MUX_IN2_PARID+0	11336	VAR_R4_2+0	Link input 2 with Float parameters 2 VAR-FB0
MUX_IN3_PARID+0	11344	VAR_R4_3+0	Link input 3 with Float parameters 3 VAR-FB0
MUX_IN4_PARID+0	11352	VAR_R4_0+1	Link input 4 with Float parameters 0 VAR-FB1
MUX_IN5_PARID+0	11360	VAR_R4_1+1	Link input 5 with Float parameters 1 VAR-FB1
MUX_IN6_PARID+0	11368	VAR_R4_2+1	Link input 6 with Float parameters 2 VAR-FB1
MUX_IN7_PARID+0	11376	VAR_R4_3+1	Link input 7 with Float parameters 3 VAR-FB1
MUX_MODE+0	11264	1	Enable FB in mode active as switch

Steps to follow during the exercise

- Change value of USER_I4_VAR1 (584) in mapp Cockpit in range from 0 to 7 and observe MUX_VALUE_I4+0 (11288) and MUX_VALUE_R4+0 (11296).
- Trace below ParIDs in mapp Cockpit.
 - USER_I4_VAR1 (584)
 - MUX_VALUE_I4+0 (11288)
 - MUX_VALUE_R4+0 (11296)
- The user should also try VAR_I4_x+0 and VAR_I4_x+1 instead of VAR_R4_x+0 and VAR_R4_x+1 simply for trial purposes and to observe both outputs of the MUX function block.

3.4 Group exercise: Virtual encoder emulation - Phase1

An axis should be coupled to a master that does not deliver an analog encoder signal, but just provides a digital pulse every shaft rotation. The master velocity should therefore be emulated.

Machine example: Gang saw coupled to wood infeed



Figure 5: Group exercise: Gang saw coupled to wood infeed

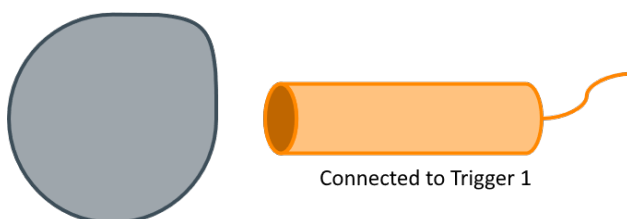


Figure 6: Sensor connection to trigger1



Figure 7: Tperiod for virtual encoder speed calculation

Digital sensor is connected to trigger 1 on the drive. To simulate the condition of the digital sensor, the homing sensor is connected to trigger 1 on the ACOPOSmicro drive. Participants should consider homing as a digital sensor and calculate the speed of rotation in units/second. To calculate the speed of rotation, consider 1 rotation as 36000 units for the virtual encoder.

Participants do not need to create a virtual axis and/or encoder for this exercise. They should calculate the speed for the virtual encoder and participants can use the function blocks below to do this. Consider real axis feedback as a virtual encoder to relate the speed calculation. Enter a velocity movement for the real axis to simulate this example.

- VAR
- ARITH
- MUX

Here, the result of the speed calculation for the virtual encoder would not be 100% correct.

Hint



Find the rising edge to rising edge time and store/hold the time in a variable/function block output.
 $\text{Speed} = \text{Counts of one rotation} / T_{\text{period per second}}$

4 Function block collection 2

4.1 CMP - Comparator

Automation Help: Examples

?

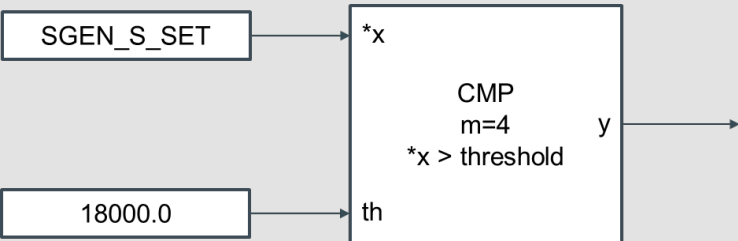
Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ CMP - Comparator \ Example

About the function block

The CMP function block creates a comparator which can be configured. Different comparison operation is set as a mode as shown below with output getting high on level or edge sensitive base comparison of input(*x) to threshold(th).

- Less than (*x < th)
- Equal to (*x == th)
- Less than or equal to (*x <= th)
- Greater than (*x > th)
- Not equal to (*x != th)
- Greater than or equal to (*x >= th)

Exercise: Compare position controller input set position (live)



Parameter passing

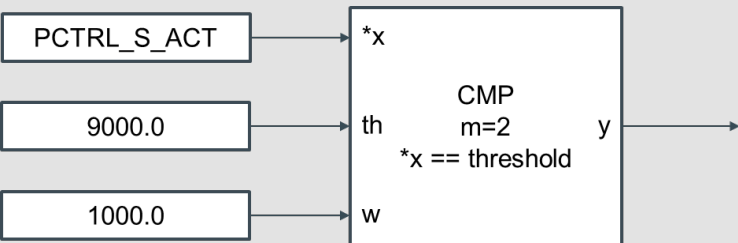
Creating, configuring and defining the connection structure of CMP FB0

FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Creating CMP FB0
CMP_IN_PARID+0	6656	SGEN_S_SET	Link input to position controller input set position
CMP_THRESHOLD+0	6664	18000.0	Define level
CMP_MODE+0	6688	4	Set comparison operator to larger & level sensitive

Steps to follow during the exercise

- Enter a discrete, jog or continuous movement for the axis to have position greater than the threshold value.
- During time of movement, observe output CMP_VALUE+0(6696) in mapp Cockpit trace.

Exercise: Compare position controller actual position (live)



Parameter passing

Creating, configuring and defining the connection structure of CMP FB0

FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Creating CMP FB0
CMP_IN_PARID+0	6656	PCTRL_S_ACT	Link input to position controller actual position
CMP_THRESHOLD+0	6664	9000.0	Define level
CMP_WINDOW+0	6672	1000.0	Define window
CMP_MODE+0	6688	2	Set comparison operator to equal & level sensitive

Steps to follow during the exercise

- Enter a discrete, jog or continuous movement for the axis to have a position around threshold value in $\pm$ window range.
- During time of movement, observe output CMP_VALUE+0(6696) in mapp Cockpit trace.

4.2 EVWR - Event controlled parameter writing

Automation Help: Examples

?

Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ EVWR - Event Controlled Parameter Writing \ Example

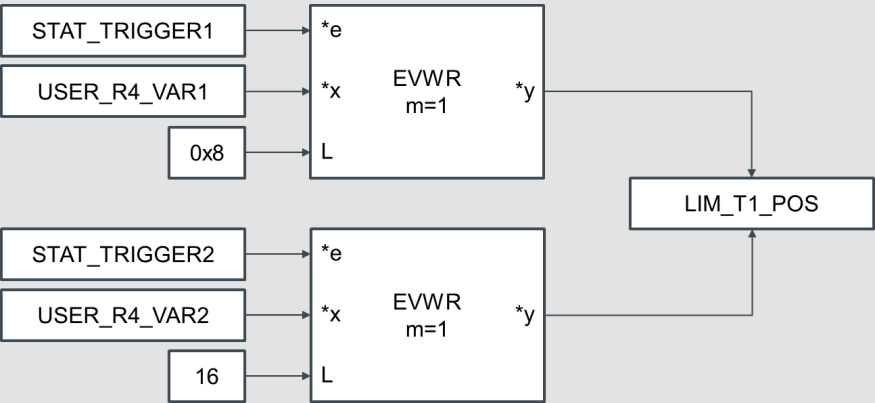
About the function block

An ACOPOS event should cause a parameter change. This could basically be done as a PLC application via the network by cyclically reading Parameter IDs, checking the event and writing the respective parameter. For fast reaction times, this function can only be implemented directly on the ACOPOS drive.

This function block provides different modes of operation as shown below for writing inputs to outputs based on an event.

- Edge sensitive writing (edge at the time of the event, input matched to the event level)
- Level sensitive writing (until the time event, input is matched to the event level)
- At every change of the event, input-based writing

Exercise: Setting torque limit based on event (live)



Parameter passing

Torque limiter functionality enabling			
TLIM_MODE	1480	1	Static mode
Setting value of user I4 variables			
USER_R4_VAR1	586	0.2	Setting value in Nm
USER_R4_VAR2	587	1.0	Setting value in Nm
Creating, configuring and defining the connection structure of EVWR FB0			
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Creating EVWR FB0
EVWR_EVENT_PARID+0	4608	STAT_TRIGGER1	Link input to status trigger1
EVWR_EVENT_LEVEL+0	4624	8	Level at trigger1 closed
EVWR_IN_PARID+0	4616	USER_R4_VAR1	Link Input with User R4 variable 1
EVWR_WR_PARID+0	4632	LIM_T1_POS	Link with write parameter "max. acceleration torque in positive direction"
EVWR_MODE+0	4640	1	Enable FB with mode "edge sensitive"
Creating, configuring and defining the connection structure of EVWR FB1			
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+1	Creating EVWR FB1

EVWR_EVENT_PARID+1	4609	STAT_TRIGGER2	Link input to status trigger2
EVWR_EVENT_LEVEL+1	4625	16	Level at trigger2 closed
EVWR_IN_PARID+1	4617	USER_R4_VAR2	Link Input with User R4 variable 2
EVWR_WR_PARID+1	4633	LIM_T1_POS	Link with write parameter “max. acceleration torque in positive direction”
EVWR_MODE+1	4641	1	Enable FB with mode “edge sensitive”

Steps to follow during the exercise

- Force or provide input to trigger 1 and 2 to set or change output variable value according to the configured events. Use **MC_BR_ForceHardwareInputs** to force trigger2 if participant would choose software approach with condition that configured source is set to **Force by function block** in configuration for desired drive channel in the Physical View.
- During the time when the status of trigger 1 (STAT_TRIGGER1 (463)) and trigger 2 (STAT_TRIGGER2 (464)) is changing, observe output LIM_T1_POS (248) in mapp Cockpit trace.

4.3 LOGIC - Logical operation

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ LOGIC - Logical Operation \ Example



Implement all LOGIC examples given in Automation Help. However, start implementation with a simple exercise at first and then lastly **RS-Flip-Flop Composed of NAND Gates**.

About the function block

This function block links a maximum of four input values. Logical operation is set as the mode. This function block provides different modes based on two and four input values as shown below. However, user can generate a mode based on truth table that is explained in help document.

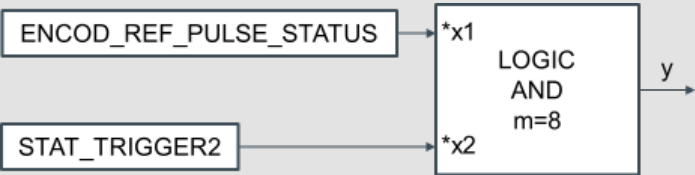
Two input values

NOR	NOT x1	NAND	EXNOR
OR	NOT x2	AND	EXOR

Four input values

NOR	NAND	(x1 AND x2) OR (x3 AND x4)
OR	AND	(x1 OR x2) AND (x3 OR x4)

Exercise: Combinational logic 1 (self-paced)



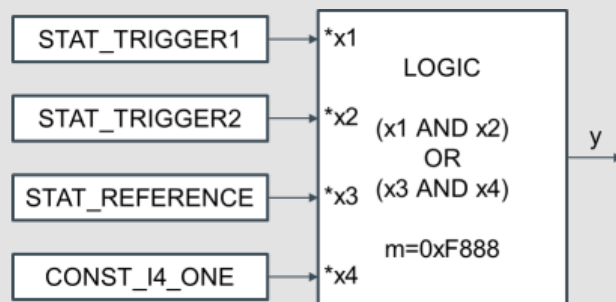
Parameter passing

Creating, configuring and defining the connection structure of LOGIC FB0

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Creating LOGIC FB0
LOGIC_IN1_PARID+0	3080	ENCOD_REF_PULSE_STATUS	Motor encoder reference pulse
LOGIC_IN2_PARID+0	3088	STAT_TRIGGER2	Trigger2 of ACOPOS input
LOGIC_MODE+0	3072	8	Enable FB in mode AND

Steps to follow during the exercise

- Rotate the motor using logic or manually by hand and also mean time, force or provide input to trigger2 of ACOPOS. Use **MC_BR_ForceHardwareInputs** to force trigger2 if participant would choose software approach with condition that configured source as **Force by function block** in configuration of desired channel of drive in Physical View.
- Trace below ParIDs in mapp Cockpit trace.
 - ENCOD_REF_PULSE_STATUS (369)
 - STAT_TRIGGER2 (464)
 - LOGIC_VALUE+0 (3096)

Exercise: Combinational logic 2 (live)**Parameter passing**

Creating, configuring and defining the connection structure of LOGIC FB0

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Creating LOGIC FB0
LOGIC_IN1_PARID+0	3080	STAT_TRIGGER1	Link input1 to trigger1 input
LOGIC_IN2_PARID+0	3088	STAT_TRIGGER2	Link input2 to trigger2 input
LOGIC_IN3_PARID+0	3104	STAT_REFERENCE_SWITCH	Link input3 to Reference input
LOGIC_IN4_PARID+0	3112	CONST_I4_ONE	Link input4 to Logical Constant 1-one
LOGIC_MODE+0	3072	0xF888	Enable FB in mode (x1 AND x2) OR (x3 AND x4)

Steps to follow during the exercise

- Force or provide input to trigger1, trigger2 and reference of ACOPOS in various combinations and user should get output according to the configured mode.
- Trace below ParIDs in mapp Cockpit trace.
 - STAT_TRIGGER1 (463)
 - STAT_TRIGGER2 (464)
 - STAT_REFERENCE_SWITCH (460)
 - LOGIC_VALUE+0 (3096)

Exercise: RS-Flip-Flop composed of NAND gates (self-paced)

- Change value of USER_I4_VAR1 (584) (reset) and USER_I4_VAR2 (585) (set) from mapp Cockpit and observe LOGIC_VALUE+0 (3096) (!Q) and LOGIC_VALUE+1 (3097) (Q) in trace of mapp Cockpit.

4.4 DELAY - Delay time

Automation Help: Examples

Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ DELAY - Delay Time \ Example



Implement DELAY example given in Automation Help.

About the function block

The DELAY function block can be used to implement time delays. In simple terms, the input signal is delayed by the defined time t (rounded to multiple to $400 \mu s$) and an output is provided. The initial value is passed as an output until the dead time is completed.

This function block can be used to compensate delay time that has occurred due to network latency.

Exercise: Delay current motor encoder position by 3.2ms (live)

- Enter a positive and negative movement from the Watch window for the axis as desired by the participant and observe outputs DELAY_VALUE_I4+0 (7184) and ENCOD1_S_ACT (91) in mapp Cockpit trace.

4.5 Group exercise: Virtual encoder emulation - Phase 2

Refer to "Group exercise - Virtual encoder - Phase 1" for basic introduction to the machine and a functionality overview. In this group exercise, participants should come up with correct solution because all required function blocks would have been explained.

Participants may use function blocks from following list

- VAR
- ARITH
- LOGIC
- DELAY
- MUX
- EVWR

5 Function block collection 3

5.1 MPGEN - Motion profile generator

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ MPGEN - Motion Profile Generator \ Example

About the function block

The MPGEN function block creates a motion profile that can be configured. This profile corresponds to a compensation function that can work as a position coupling or a time-controlled movement.

This function block has different modes of operation depending on the configuration made for a mode. This function block generates a motion profile from the old set value to the new set value depending on the configuration made for position-based mode. The function block provides the set position or speed depending on the mode as a setpoint-based value (direct input) or ParID-based value (pointer input). The user can use the ParID-based set position or speed if the user has already done some calculations and link that variable or another function block output.

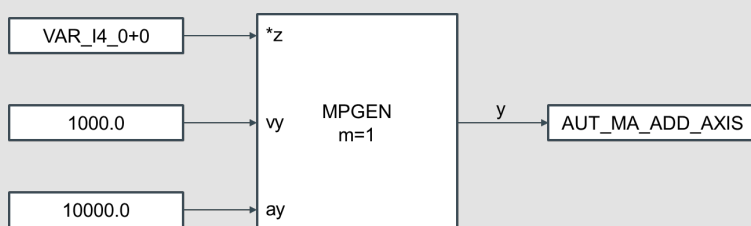
- Time controlled
- Position coupling, time optimized, setting movement parameters
- Position coupling, jolt minimized, setting master compensation distance
- Speed controlled
- Position coupling time optimized, setting movement parameters, compensation only within one zone

Additive axis



Use MPGEN_VALUE_I4 as master or slave additive axis.

Exercise: Time controlled motion profile generation (self-paced)



Parameter passing

Creating and defining VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creating VAR FB0
VAR_I4_0+0	4096	0	Defining value for I4 Variable 0 of FB0

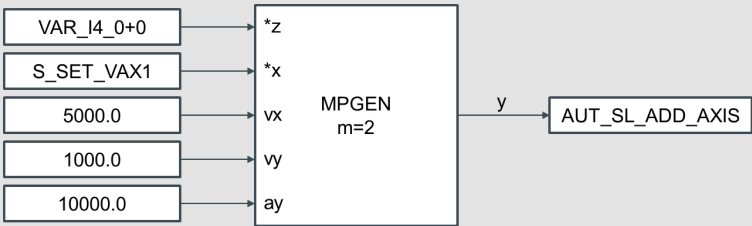
Creating, configuring and defining the connection structure of MPGEN FB0

FUNCTION_BLOCK_CREATE	777	MPGEN_MODE+0	Creating MPGEN FB0
MPGEN_SET_VALUE_PARID+0	5208	VAR_I4_0+0	Link set value to I4 Variable 0 of FB0
MPGEN_V_MAX+0	5184	1000.0	Setting max velocity for profile generation
MPGEN_A_MAX+0	5192	10000.0	Setting max acceleration for profile generation
MPGEN_MODE+0	5120	1	Enables FB with mode time controlled

Steps to follow during the exercise

- Change value of VAR_I4_0+0 (4096) via mapp Cockpit.
- Trace below ParIDs in mapp Cockpit trace if the value of VAR_I4_0+0 (4096) is changed.
 - VAR_I4_0+0 (4096)
 - MPGEN_VALUE_I4+0 (5136)
 - MPGEN_VALUE_R4+0 (5152)
 - MPGEN_STATUS+0 (5160)

Exercise: Position coupling, time optimized motion profile generation (live)



Parameter passing

Creating and defining VAR FB0

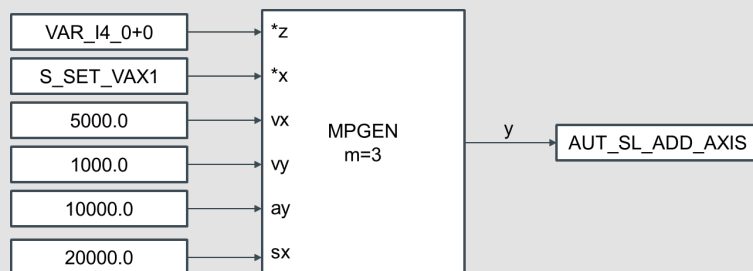
FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creating VAR FB0
VAR_I4_0+0	4096	0	Defining value for I4 Variable 0 of FB0

Creating, configuring and defining the connection structure of MPGEN FB0

FUNCTION_BLOCK_CREATE	777	MPGEN_MODE+0	Creating MPGEN FB0
MPGEN_SET_VALUE_PARID+0	5208	VAR_I4_0+0	Link set value to I4 Variable 0 of FB0
MPGEN_MA_PARID+0	5168	S_SET_VAX1	Link master position to virtual axis position
MPGEN_MA_V_MAX+0	5176	5000.0	Setting max master velocity
MPGEN_V_MAX+0	5184	1000.0	Setting max velocity for profile generation
MPGEN_A_MAX+0	5192	10000.0	Setting max acceleration for profile generation
MPGEN_MODE+0	5120	2	Enables FB with mode position coupling, time optimized

Steps to follow during the exercise

- Change value of VAR_I4_0+0 (4096) and speed of virtual axis via mapp Cockpit.
- Trace below ParIDs in mapp Cockpit trace if values of VAR_I4_0+0 (4096) and/or virtual axis speed are changed .
 - S_SET_VAX1 (412)
 - V_SET_VAX1 (413)
 - VAR_I4_0+0 (4096)
 - MPGEN_VALUE_I4+0 (5136)
 - MPGEN_VALUE_R4+0 (5152)
 - MPGEN_STATUS+0 (5160)

Exercise: Position coupling, jolt optimized motion profile generation (self-paced)

Parameter passing

Creating and defining VAR FB0

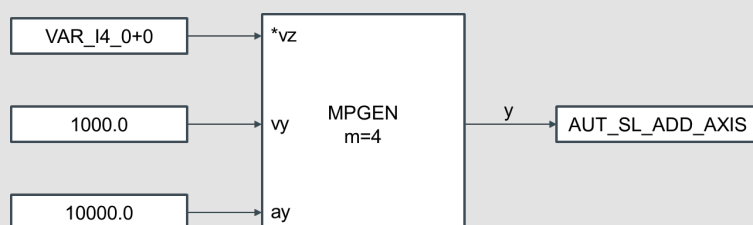
FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creating VAR FB0
VAR_I4_0+0	4096	0	Defining value for I4 Variable 0 of FB0

Creating, configuring and defining the connection structure of MPGEN FB0

FUNCTION_BLOCK_CREATE	777	MPGEN_MODE+0	Creating MPGEN FB0
MPGEN_SET_VALUE_PARID+0	5208	VAR_I4_0+0	Link set value to I4 Variable 0 of FB0
MPGEN_MA_PARID+0	5168	S_SET_VAX1	Link master position to virtual axis position
MPGEN_MA_S_COMP+0	5200	20000.0	Setting master compensation distance
MPGEN_MA_V_MAX+0	5176	5000.0	Setting max master velocity
MPGEN_V_MAX+0	5184	1000.0	Setting max velocity for profile generation
MPGEN_A_MAX+0	5192	10000.0	Setting max acceleration for profile generation
MPGEN_MODE+0	5120	3	Enables FB with mode position coupling, jolt optimized

Steps to follow during the exercise

- Change value of VAR_I4_0+0 (4096) and speed of virtual axis via mapp Cockpit.
- Trace below ParIDs in mapp Cockpit trace if values of VAR_I4_0+0 (4096) or virtual axis speed are changed.
 - S_SET_VAX1 (412)
 - V_SET_VAX1 (413)
 - VAR_I4_0+0 (4096)
 - MPGEN_VALUE_I4+0 (5136)
 - MPGEN_VALUE_R4+0 (5152)
 - MPGEN_STATUS+0 (5160)
 - MPGEN_ERROR_COUNT+0 (5232)
- As further trial, change value of MPGEN_MA_S_COMP+0 (5200) via mapp Cockpit and observe values as given above to see effect.

Exercise: Speed controlled motion profile generation (self-paced)


Parameter passing

Creating and defining VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creating VAR FB0
VAR_I4_0+0	4096	0	Defining value for I4 Variable 0 of FB0

Creating, configuring and defining the connection structure of MPGEN FB0

FUNCTION_BLOCK_CREATE	777	MPGEN_MODE+0	Creating MPGEN FB0
MPGEN_V_SET_VALUE_PARID+0	5224	VAR_I4_0+0	Link target speed to I4 Variable 0 of FB0
MPGEN_V_MAX+0	5184	1000.0	Setting max velocity for profile generation
MPGEN_A_MAX+0	5192	10000.0	Setting max acceleration for profile generation
MPGEN_MODE+0	5120	4	Enables FB with mode speed controlled

Steps to follow during the exercise

- Change value of VAR_I4_0+0 (4096) via mapp Cockpit.
- Trace below ParIDs in mapp Cockpit trace if value of VAR_I4_0+0 (4096) is changed.
 - MPGEN_VALUE_I4+0 (5136)
 - MPGEN_VALUE_R4+0 (5152)
 - MPGEN_STATUS+0 (5160)

5.2 LATCH - Event-triggered position capture

Automation Help: Examples

Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ LATCH - Event Triggered Storage \ Example



Implement LATCH examples given in Automation Help. Also, please, pay attention to changes in ParIDs and function block mode as shown in exercises.

Touch Probe \ Registration mark detection - PLCopen function blocks for use in PLC

Participants can also use the function blocks shown below in PLC code if a delay in response is acceptable.
mapp Motion

Motion control \ mapp Motion \ Libraries \ Core \ McAxis \ Function blocks \

MC_TouchProbe

MC_BR_TouchProbe

Mechatronics \ mapp Control \ mapp Web Handling \ Libraries \ MpRegMark \

Function blocks \ MpRegMarkDetection

About the function block

The value of a changing input is to be saved (i.e. **latched**) when certain trigger events occur. The trigger signal must fulfill certain conditions, such as the trigger edge occurring within a certain window and with a certain signal width. In most cases, the input value is a position value and has a corresponding position sensor as trigger event. Therefore, the latch function block can be used for trigger positioning, length measurement and registration mark evaluation/control.

This function block has different modes based on shifting of the expected trigger position mainly as shown below

- Shifting of window starting from latch position
- Shifting of window starting from expected trigger position
- No shifting of window (All triggers are valid)

The mode has additives such as:

- Without trigger, replace latch value by window position or old latch value remains
- Without trigger at expected trigger position, latch status/count (yes or no)
- Initialization of the first window position

Since the trigger for capturing a position is based on an event, there are different possibilities for event evaluation as shown below

- Positive edge
- Negative edge
- Positive edge and evaluation of signal width, with and without signal high level while mode activated
- Negative edge and evaluation of signal width, with and without signal high level while mode activated
- Positive edge and evaluation of signal minimum width only, with and without signal high level while mode activated
- Time average value (latch value respectively) of the positive and negative edge and evaluation of signal width. Processing starts at the negative edge, with and without signal high level while mode activated

Compensation of delays

Compensation for the delay of a sensor would avoid a speed-dependent error. How user can observe a speed-dependent error. The user can run the machine at a low speed and observe that the registration mark position when cutting is at a constant position. The user can increase speed from low to moderate to high and observe that the registration mark position when cutting is still constant. However, the registration mark position is shifted little back. If this phenomena happens, it is considered that there is a requirement to compensate for sensor delay.

Sensor delay is generally provided by the sensor manufacturer in the technical data sheet for the sensor.

Delay due to

- Sensor delay is the delay time between the registration mark sensed and when the sensor gives a trigger to the ACOPOS input. This could include the internal sensor circuitry, the internal sensor detection threshold.
- It could be possible that the input would also have some delay recognizing +24 VDC. However, this delay is always constant and would be specified in the technical data sheet.
- Sensor delay could happen due to a long connection cable.

Compensation

- A negative value (interpolation) means detection values are from the past (history).
- A positive value (extrapolation) means detection values are for the future (forecast).

Configuring the delay compensation value

- **Sensor delay between 0 to 400 μ s**
 - The user should use only LATCH_T_DELAY if the value is negative e.g LATCH_T_DELAY+0 $\leftarrow$ -200 (μ s)
- **Sensor delay higher than 400 μ s**
 - The user should use combination of DELAY + LATCH_T_DELAY for best result
 - Let us consider that the sensor delay is 600 μ s in that case DELAY_TIME $\leftarrow$ 0.0004 s (400 μ s) and LATCH_T_DELAY $\leftarrow$ -200 (μ s)
 - ParID attached to LATCH_IN_PARID will become input to DELAY_IN_PARID and DELAY_VALUE_I4 output of DELAY function block will become input to LATCH_IN_PARID.
- **DELAY_TIME $\leftarrow$ 0.0 (s) does not add any delay**

Scenario to be handled



This scenario does not apply to mode 4 and mode +64 if the mode is changed from 0 to the mode that is actually required after homing.

If the LATCH function block is used in the machine logic, the user should follow the procedure shown below to avoid LATCH not functioning properly.

Procedure to follow after homing if LATCH_MODE is not 4

- $LATCH_MODE \leftarrow 0$
- Calculation of $LATCH_WINDOW_POS$ only applicable if +64 (shifting of window from first valid registration mark) not added to mode.
 - Calculate $LATCH_WINDOS_POS$ (as shown below)
 - $LATCH_WINDOS_POS \leftarrow$ Calculated value (not for mode +64)
- $LATCH_MODE \leftarrow$ original

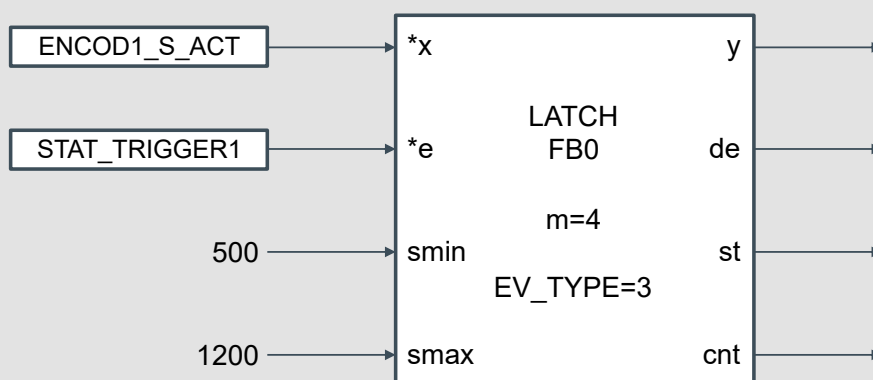
Procedure for calculating $LATCH_WINDOW_POS$

- Read: x – Position ParID value whose position/value will be captured and $LATCH_POS_IV$. After the calculation, $LATCH_WINDOW_POS$ is the remainder value.
- $Remainder = ParIDValue \bmod LATCH_POS_IV$
- Let's consider an example.
- $ParIDValue = -178593452$ and $LATCH_POS_IV = 36000$
 $Remainder = -178593452 \bmod 36000$
 $Remainder = 2548$ (should be between 0 to modulo-1. If the value is in negative, add $LATCH_POS_IV$ to bring it in- to the required range)
- Check if the remainder is greater than expected window position, add $LATCH_POS_IV$ to remainder to make the final result for the window position for the next cycle's registration mark.
 $Old\ modulo\ start\ position = ParIDValue - Remainder$
 $Old\ modulo\ start\ position = -178593452 - 2548$
 $Old\ modulo\ start\ position = -178596000$
- $LATCH_WINDOW_POSITION = Old\ modulo\ start\ position + Expected\ position$
 $LATCH_WINDOW_POSITION = -178596000 + 5000$
 $LATCH_WINDOW_POSITION = -178591000$

Exercise: Capture registration mark position with trigger 1 - All triggers are valid (self-paced)



The goal of this exercise to understand and experience event configuration, sensor delay, compensation, and how mode 4 (all triggers accepted) works.



Creating, configuring and defining the connection structure of LATCH FB0

FUNCTION_BLOCK_CREATE	777	LATCH_MODE+0	Creates LATCH FB0
LATCH_IN_PARID+0	9736	ENCOD1_S_ACT	Link input with encoder1 position
LATCH_EV_PARID+0	9744	STAT_TRIGGER1	Link trigger event with digital input
LATCH_EV_TYPE+0	9752	3	Negative edge and evaluation of the signal width
LATCH_EV_WIDTH_MIN+0	9760	500	Minimum trigger signal width 500 units
LATCH_EV_WIDTH_MAX+0	9768	1200	Maximum trigger signal width 1200 units
LATCH_T_DELAY+0	9808	0	No delay compensation
LATCH_MODE+0	9728	4	Enable FB - all triggers are valid

Steps to follow during the exercise

- Implement with different trigger event to understand behavior of events, latch value and latch status. Implement LATCH_EV_TYPE of 0,1,2,3,4 and +16 would be an added advantage to understand mode activated on high level of event.



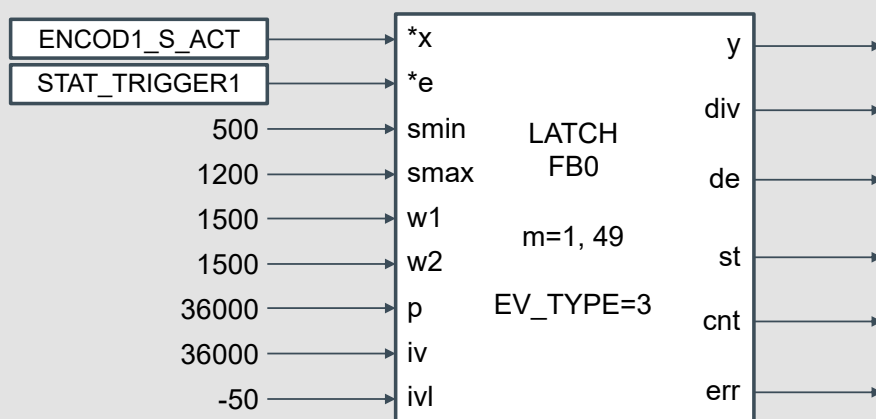
Motion control \ ACP10/ARNC0 \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ LATCH - Event Triggered Storage \ Function \ Trigger Event

- Enter a velocity movement for the axis and force or provide input to trigger 1 so that function block receives an event.
- Also, implement changing delay time compensation value so that sensor delay can be adjusted.
- During the time of movement, observe below connected ParIDs in mapp Cockpit trace at a 400 μ s (0.0004 s) sampling rate.
 - ENCOD1_S_ACT (91)
 - STAT_TRIGGER1 (463)
 - LATCH_VALUE+0 (9824)
 - LATCH_STATUS+0 (9840)

Exercise: Capture registration mark position with trigger 1 - Shifting window position (live)



The goal of this exercise to understand and experience shifting of window position, window range and mode configuration.



Creating, configuring and defining the connection structure of LATCH FB0

FUNCTION_BLOCK_CREATE	777	LATCH_MODE+0	Creates LATCH FB0
LATCH_IN_PARID+0	9736	ENCOD1_S_ACT	Link input with encoder1 position
LATCH_EV_PARID+0	9744	STAT_TRIGGER1	Link trigger event with digital input
LATCH_EV_TYPE +0	9752	3	Negative edge and evaluation of the signal width
LATCH_EV_WIDTH_MIN+0	9760	500	Minimum trigger signal width 500 units
LATCH_EV_WIDTH_MAX+0	9768	1200	Maximum trigger signal width 1200 units
LATCH_WINDOW1+0	9856	1500	Latch window in negative direction 200 units
LATCH_WINDOW2+0	9864	1500	Latch window in positive direction 200 units
LATCH_WINDOW_POS+0	9784	36000	First window position = first expected value
LATCH_POS_IV+0	9792	36000	Interval of the signal period
LATCH_POS_IV_ELONG+0	9800	-50	Interval reduction when a trigger occurs
LATCH_T_DELAY+0	9808	0	No delay compensation

LATCH_MODE+0	9728	1	Enable FB
--------------	------	---	-----------

Steps to follow during the exercise

- Enter a velocity movement for the axis and force or provide input to trigger 1 so that function block receives an event.
- During the time of movement, the user can observe below connected ParIDs in mapp Cockpit trace at a 400 μ s (0.0004 s) sampling rate.
 - ENCOD1_S_ACT (91)
 - STAT_TRIGGER1 (463)
 - LATCH_VALUE+0 (9824)
 - LATCH_STATUS+0 (9840)
- Stop the axis and perform homing of axis.
- Follow step shown in **Scenario to be handled**.
- To set and read ParIDs of LATCH function block, use mapp Cockpit.
- Enter a velocity movement for the axis and force or provide the input to trigger 1 so that function block receives an event.
- During the time of movement, observe below connected ParIDs in mapp Cockpit trace at a 400 μ s (0.0004 s) sampling rate.
 - ENCOD1_S_ACT (91)
 - STAT_TRIGGER1 (463)
 - LATCH_VALUE+0 (9824)
 - LATCH_STATUS+0 (9840)



Participants to try below modes as well (self-paced)

- Implement mode 1 and 49 to understand shifting of window position and the corresponding output relationship.
- Implement mode 2 and 50 to understand shifting of window position and the corresponding output relationship.
- Implement +64 to understand initialization of the first window position.

Exercise: Homing of axis during LATCH FB is enabled (live)



The goal of this exercise to perform homing of an axis after a certain axis movement to observe LATCH value and window position.

Steps to follow during the exercise

- Follow the steps given in "Exercise: Capture registration mark position with trigger 1 - shifting window position".
- Only perform homing of axis if actual position of motor or position ParID is higher than window position.
- Perform homing of the axis and initiate axis movement to observe shown below ParIDs in mapp Cockpit trace at a 400 μ s (0.0004 s) sampling rate while applying the input to trigger 1.
 - ENCOD1_S_ACT (91)
 - STAT_TRIGGER1 (463)
 - LATCH_WINDOW_POS+0 (9784)
 - LATCH_VALUE+0 (9824)
 - LATCH_STATUS+0 (9840)
- Stop the axis and perform homing of axis.
- Follow step given in **Scenario to be handled**.
- To setting and reading of ParIDs of LATCH function block, use mapp Cockpit.
- Enter a velocity movement for the axis and force or provide input to trigger 1 so that function block receives an event.
- During the time of movement, observe shown below connected ParIDs in mapp Cockpit trace at a 400 μ s (0.0004 s) sampling rate.
 - ENCOD1_S_ACT (91)
 - STAT_TRIGGER1 (463)
 - LATCH_VALUE+0 (9824)
 - LATCH_STATUS+0 (9840)

5.3 Group exercise: Registration mark correction

Registration mark correction is a widely used application in packaging machines such as a horizontal flow wrap, a vertical form fill seal or cut to length bagging machines. In such applications, a fast response to registration mark position disturbance is required. Registration mark position disturbance can occur due to various factors such as friction incurred when the film passes through rollers, a tensioner with inconsistencies, incorrect entry of the pulling length, a pressure variation on pulling rollers, etc. In such conditions, the registration mark needs to be corrected. However, sometimes due to wrapper changeover, there could be a potential disturbance in registration mark position. In such cases, registration mark position correction must be limited in order to safeguard material inside the package.



Figure 8: Group exercise: Registration mark correction

Participants may use function blocks from the following list.

- VAR
- LATCH
- MINMAX
- ARITH
- MPGEN

Hint



LATCH provides an output related to the deviation of the registration mark. Consider a check for the limits of correction.

To provide the correction via MPGEN, the axis should be working in slave mode, either in a cam automat, gear or coupling, and the correction output works as an additive element for the slave.

MPGEN provides an output only if the input is changed, and the output works in "relative mode". E.g. If the input is changed from 100 to 200, MPGEN will only generate a profile from 100 to 200 with the specified parameter settings.

6 Function block collection 4

6.1 CAMCON - Cam control

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ CAMCON - Cam Control \ Example

About the function block

The CAMCON function block switches an output depending on the positions of a "master axis". The on and off positions can be set for each cam. This refers to the entire cam switching sequence for an output, which is identified as a track. The start procedure and the method of operation for cam control is set up differently using a mode.

Homing of axis while CAMCON is active

The user must avoid homing an axis while CAMCON is active. This could lead to high frequency output generation for a certain amount of time. The image shown below is an actual trace of the CAMCON output.



Follow the steps shown below to avoid high frequency output generation

- CAMCON mode (CAMCON_MODE+0 (10752)) set to ZERO.
- CAMCON value (CAMCON_VALUE+0 (10848)) set to ZERO.
- Complete homing of axis.
- CAMCON mode (CAMCON_MODE+0 (10752)) back to original / as per requirement.

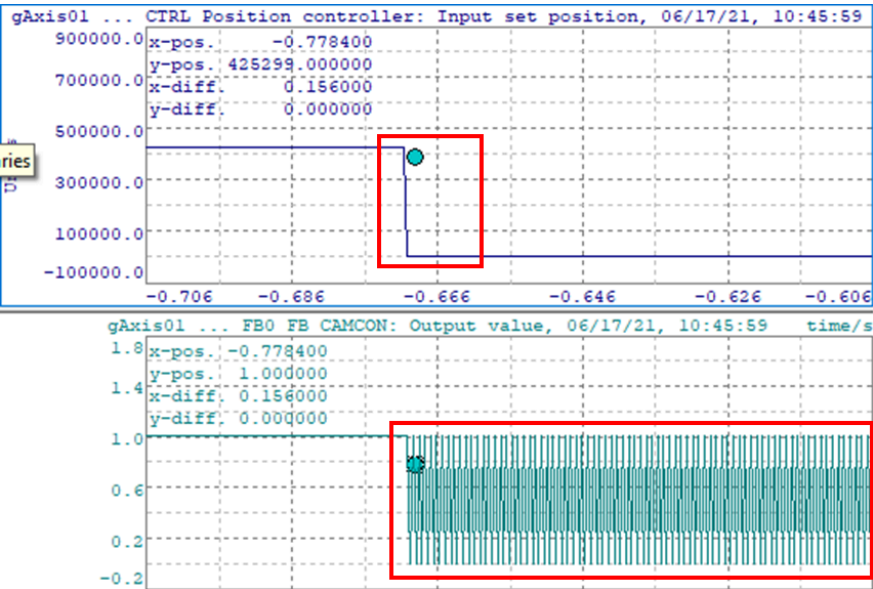


Figure 9: Homing while CAMCON is active, high frequency CAMCON output generation

Exercise: Logical/Digital output coupled with position controller: Set position (live)

Here, an example is taken from Automation Help. However, some ParIDs have been changed for training purposes, such as: LOGIC input ParID is changed to VAR_I2_0+0 and output ParID is not connected to DIO_OUT5_PARID+4. Also, some ParIDs are removed and not configured, such as T_DELAY_ON, T_DELAY_OFF, T1_DELAY and HYSTERESIS.

Creating and configuring VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I2_0+0	Creates VAR FB0
VAR_I2_0+0	4160	0	Enable bit

Creating, configuring and defining the connection structure of CAMCON FBO

FUNCTION_BLOCK_CREATE	777	CAMCON_MODE+0	Creates CAMCON FBO
CAMCON_IN_PARID+0	10760	PCTRL_S_SET	Link input with set position
CAMCON_S_START+0	10768	0	Shift start interval
CAMCON_S_IV+0	10776	36000	Units per encoder revolution
CAMCON_MAX_CAM+0	10816	2	2 Cams
CAMCON_CAM_INDEX+0	10824	1	Index of first cam
CAMCON_CAM_S1+0	10832	4500.0	Start position of first cam
CAMCON_CAM_S2+0	10840	9000.0	End position of first cam
CAMCON_CAM_INDEX+0	10824	2	Index of second cam
CAMCON_CAM_S1+0	10832	18000.0	Start position of second cam
CAMCON_CAM_S2+0	10840	27000.0	End position of second cam
CAMCON_VALUE+0	10848	0	Start level at low
CAMCON_MODE+0	10752	1	Enable FB directly in the interval

Creating, configuring and defining the connection structure of LOGIC FBO

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Creates LOGIC FBO
LOGIC_IN1_PARID+0	3080	CAMCON_VALUE+0	Link input 1 with CAMCON result value
LOGIC_IN2_PARID+0	3088	VAR_I2_0+0	Link input 2 with I2 variable of FBO
LOGIC_MODE+0	3072	8	Enable FB with (x1 AND x2)

Steps to follow during the exercise

- Enter a velocity movement for the axis and provide value 1 to VAR_I2_0+0 (4160) via mapp Cockpit so that CAMCON generates pulses based on provided cam start and end positions, and CAMCON output is interlinked with logical AND operation so participant can observe the logical output operation.
- Record trace in mapp Cockpit for ParIDs PCTRL_S_SET (113), CAMCON_VALUE+0 (10848) and LOGIC_VALUE+0 (3096).

Exercise: Mode change: 1 → 2 (self-paced)

- Change mode (CAMCON_MODE+0 (10752)) from 1 to 2 and perform homing of axis at random position where CAMCON output is set to high.
- Repeat steps listed in first exercise to observe CAMCON output.

Exercise: Mode change: 2 → 4 (self-paced)

- Change mode (CAMCON_MODE+0 (10752)) from 2 to 4 and configure CAMCON start position (CAMCON_CAM_S1+0 (10832)) (5000) and perform homing of axis at random position where user can initiate both conditions as below.
 - 1) Axis Position (8000) is higher than CAMCON start position
 - 2) Axis Position (3000) is lower than CAMCON start position
- Repeat steps listed in first exercise to observe CAMCON output.

Exercise: Mode change: 4 → 5 (self-paced)

- Change mode (CAMCON_MODE+0 (10752)) from 4 to 5 and CAMCON start position (5000) is not valid now, so this could be deleted or disabled; Configure CAMCON_EV_PARID (10864) ← VAR_I2_1+0 (4168).
- Set value of VAR_I2_1+0 (4168) to 1 using mapp Cockpit.
- Repeat steps listed in first exercise to observe CAMCON output (CAMCON_VALUE +0 (10848)).
- Enter a positive and negative movement for the axis to observe output.

Exercise: Mode change to 17 (+16: only with positive direction of movement) (self-paced)

- Enter a positive and negative movement for the axis so the CAMCON output (CAMCON_VALUE +0 (10848)) can be observed in both directions in mapp Cockpit trace.
- Change mode to 17 and repeat the steps listed above. Now, observe that the CAMCON output (CAMCON_VALUE +0 (10848)) is only available in positive direction and no output generated during negative movement.
- Enter a small distance positive and negative movement between CAMCON start and the stop zone and observe that the output is still high in mapp Cockpit.

Exercise: Mode change to 1 and Overlapping of cams (self-paced)

- Change mode (CAMCON_MODE+0 (10752)) to 1 and the user can configure CAMCON_CAM_S2+0 (10840) for the 2nd cam to 11000.0 and observe that the CAMCON output (CAMCON_VALUE+0 (10848)) is now extending towards next cycle and overlapping cycles.
- Set value of VAR_I2_1+0 (4168) to 1 with help of mapp Cockpit.
- Repeat steps listed in first exercise 1 to observe CAMCON output (CAMCON_VALUE +0 (10848)).
- Enter a positive and negative movement for the axis to observe the output.

6.2 Task/Exercise



Use ACOPOS function blocks for the exercise shown below. However, the participant is permitted to use PLC logic wherever functionality is not possible with ACOPOS function blocks.

Task 1: OldValue <> NewValue, generate a pulse for one cycle only

For this exercise, consider that variable USER_I4_VAR1 is being changed/updated from the PLC/mapp Cockpit and a pulse must be generated for one scan cycle only. It could be possible that the output is going to be used in a further logic process made in the ACOPOS parameter table using ACOPOS function blocks.

Task 2: Find minimum, maximum and RMS values for monitoring parameters in each last cycle

Here, find the RMS, minimum and maximum current or torque in each last cycle. For the movement cycle of the axis, consider the operational cycle shown below.

- One full rotation of axis in positive direction completes in 500 ms with 33% acceleration + 34% constant velocity + 33% deceleration time
- Dwell time of 500 ms
- The axis comes back to start position in 500 ms with 33% acceleration + 34% constant velocity + 33% deceleration time.
- Dwell time of 500 ms

Cycle should continue to move until the "stop" is pressed.



Participants can choose the option for axis movement as shown below

- Absolute movement
- Relative movement
- Cam automat

6.3 Group exercise: Flying saw

A flying saw is an application where material needs to be cut based on a length measurement, a counting event for holes or both. Here, in this group exercise, it is expected that participants generate a trigger to move the flying saw based on counting a specified number of holes. A sensor to sense/count holes is connected to trigger 1. There should be the possibility to add a cutting offset, so cutting can start after a specified distance from a hole. There could be different methods to achieve the required functionality, and participants are free to choose any.

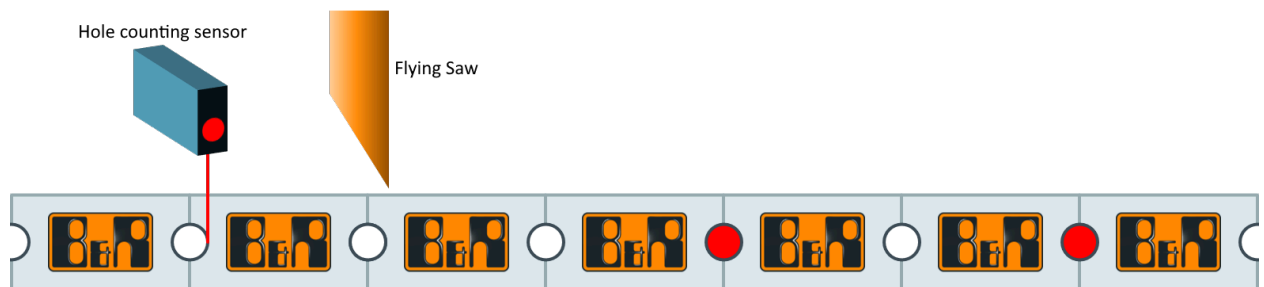


Figure 10: Group exercise: Flying saw

For this exercise, participants may use function blocks from the following list.

- VAR
- EVWR
- COUNT
- ARITH
- CMP
- LOGIC
- CAMCON

Hint



Participants need not trigger a movement for the flying saw; they need to trace trigger and show this to the trainer.

Participant can provide fixed offset from a hole for cutting in the ACOPOS parameter table, and participant should be able demonstrate difference in position trace for hole and trigger position.

Participants can use "COUNT" output as event to store the axis position to generate the cutting offset.

6.4 Group exercise: Torque limiter

Torque limit should be lowered in certain cam automat states or based on the position of the cylinder.

Machine example: Close a press/mold, prevent damage if press/mold is mechanically blocked (e.g. product not removed)



Figure 11: Group exercise: Torque limiter

There are different possibilities for a torque limit that could be based on percentage, Nm or Nm in ParIDs.

Participants can achieve a solution using various combinations and participants are permitted to use function blocks from following list.

- VAR
- CMP
- EVWR
- MUX

Automation Help

The user can visit the following path in Automation Help to get information about torque limiter functionality.



Motion Control \ Acp10/ARNCO \ Reference manual \ ACOPOS drive functions
Drive control \ Torque limiter

Hint



Torque limit is only applied if TLIM_MODE is changed.
TLIM_MODE = 1 (Static)
TLIM_MODE = 2 (Cyclic) - Based on ParID

7 Additional collections

There are more function blocks than the ones already mentioned.

7.1 Further function blocks

7.1.1 MINMAX - Minimum Maximum

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ MINMAX - Minimum Maximum \ Example



Implement all MINMAX example shown in Automation Help.

- Maximum absolute value past the time
- Maximum past the time
- Limitation to a value range

About the function block

In each cycle, the function block determines the current extreme value of all inputs. The extreme value is set using the mode: "Minimum" for the smallest value or "Maximum" for the largest value.

Exercise: Maximum absolute value past the time (self-paced)

Steps to follow during the exercise

-) Implement example provided in Automation Help.
- a) Enter a positive and negative movement from the Watch window for the axis as desired by the user and observe output MINMAX_VALUE_R4+0 (12304) in trace of mapp Cockpit.
- b) MINMAX_VALUE_I4+0 (12296) would not be supported because data type of PCTRL_V_ACT (92) is float.

Exercise: Maximum past the time (live)

Steps to follow during the exercise

- Implement example provided in Automation Help.
- Follow steps listed in **Exercise: Maximum absolute value past the time** and observe output of MINMAX_VALUE_R4+0 (12304) and MINMAX_VALUE_R4+1 (12305) in mapp Cockpit.
- Reset MINMAX_VALUE_R4+0 (12304) to live/actual PCTRL_LAG_ERROR (112) by passing value 0 to USER_I4_VAR1 (584).
- Change value of USER_I4_VAR1 (584) as shown below and observe the output.
 - 0) MINMAX_VALUE_R4+0 (12304) and MINMAX_VALUE_R4+1 (12305) to live/actual PCTRL_LAG_ERROR (112).
 - 1) Maximum value of past MINMAX_VALUE_R4+0 (12304) and Minimum value of past MINMAX_VALUE_R4+1 (12305).

Exercise: Limitation to a value range (self-paced)

Parameter passing for Limiting variable range



The provided sample in Automation Help is partially done. For successful solution, implement the parameter table as listed.

Creating, configuring and defining VAR FB0

FUNCTION_BLOCK_CREATE

777

VAR_R4_0+0

Creating VAR FB0

VAR_R4_0+0	4128	0.0	Initialization value
Configuring and defining user variable for setting upper and lower limit			
USER_R4_VAR1	586	10000.0	Setting the upper limit value
USER_R4_VAR2	587	-10000.0	Setting the lower limit value
Creating, configuring and defining the connection structure of MINMAX FB0			
FUNCTION_BLOCK_CREATE	777	MINMAX_MODE+0	Creating MINMAX FB0
MINMAX_IN1_PARID+0	12312	VAR_R4_0+0	Link input2 with the calculation result
MINMAX_IN2_PARID+0	12320	USER_R4_VAR1	Link input1 with upper limit
MINMAX_MODE+0	12288	1	Enable function block in mode MINIMUM
Creating, configuring and defining the connection structure of MINMAX FB1			
FUNCTION_BLOCK_CREATE	777	MINMAX_MODE+1	Creating MINMAX FB1
MINMAX_IN1_PARID+1	12313	MINMAX_VALUE_R4+0	Link input1 with output MINMAX FB0
MINMAX_IN2_PARID+1	12321	USER_R4_VAR2	Link input1 with lower limit
MINMAX_MODE+1	12289	2	Enable function block in mode MAXIMUM
Steps to follow during the exercise • Change value of VAR_R4_0+0 (4128) from mapp Cockpit and observe output MINMAX_VALUE_R4+1 (12304), which will be limited in the range from USER_R4_VAR1 (586) to USER_R4_VAR2 (587). • Change value of USER_R4_VAR1 (586) and USER_R4_VAR2 (587) and observe output MINMAX_VALUE_R4+1 (12305).			

7.1.2 PID - Transfer function

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ PID - Transfer Function \ Example

About the function block

The PID function block creates a real PID controller, which can be configured (PID transfer function). This Proportional Integral Differential controller with delay and anti-windup is constructed in additive (parallel) form.

Exercise: Checking PID functionality (live)

Parameter passing

Here, an example is taken from Automation Help. However, some ParIDs are changed for training purposes such as: The PID output is not connected to SCTRL_ADD_SET_PARID (288) and the PID input is changed to USER_R4_VAR1 (586).

Creating, configuring and defining the connection structure of PID FB0

FUNCTION_BLOCK_CREATE	777	PID_IN_PARID+0	Creating PID FB0
PID_IN_PARID+0	7680	USER_R4_VAR1	Link PID input with user variable
PID_KP+0	7688	1.0	Set proportional gain
PID_TI+0	7696	0.1	Set integrator-integral action time in seconds
PID_I_MAX+0	7704	60.0	Maximum integral action
PID_TD+0	7712	0.0	Set differential derivative time in seconds
PID_T1+0	7720	0.0	Set time constant of time delay in seconds
PID_ENABLE_PARID+0	7736	CONST_I4_ONE	Enable the PID controller

Steps to follow during the exercise

- Change value of USER_R4_VAR1 (586) from mapp Cockpit or cyclic writing of value from PLC with **MC_BR_CyclicProcessParID_AcpAx** function block.
- Trace the ParIDs shown below while changing the variable value.
 - USER_R4_VAR1 (586)
 - PID_VALUE+0 (7728)
 - PID_P_VALUE+0 (7744)
 - PID_I_VALUE+0 (7752)
- PID_DT1_VALUE+0 (7760) can also be observed if PID_TD+0 (7712) value is not ZERO.

7.1.3 VARITH - Vector arithmetic

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ VARITH - Vector Arithmetic \ Example

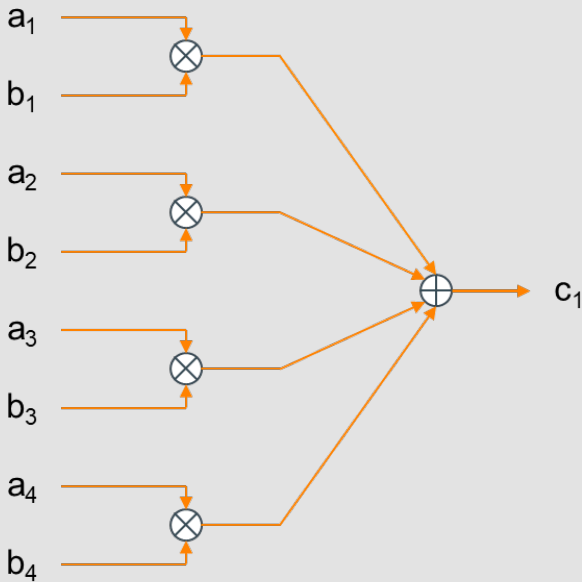
About the function block

This function block uses an arithmetic function to link two input vectors. The input and output values consist of an array of n similar elements. The arithmetic operation of these vectors is set as a mode. Depending on the operation, the result is provided as an output vector or scalar in the first element.

Exercise: Scalar product of two vectors (inner product) (live)



The given sample in Automation Help is partially adapted here. For successful solution, implement the parameter table as listed.



Parameter passing

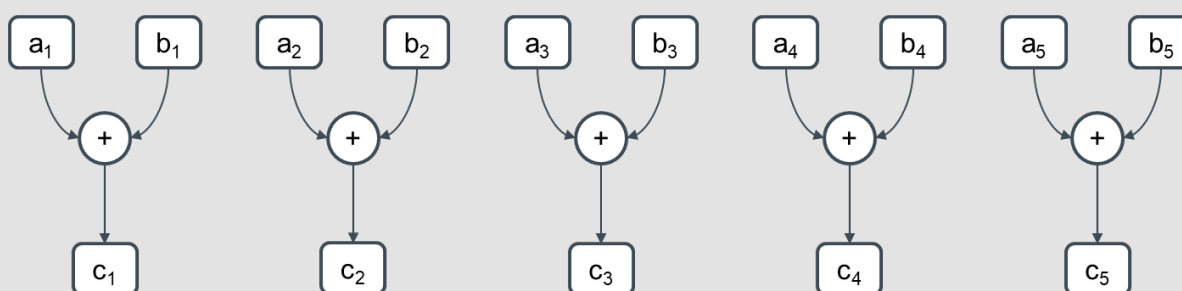
Creating and configuring VAR FB0				
FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Creates VAR FB0	
VAR_R4_0+0	4128	11.11	Setting initialization value	
VAR_R4_1+0	4136	22.22	Setting initialization value	
VAR_R4_2+0	4144	33.33	Setting initialization value	
VAR_R4_3+0	4152	44.44	Setting initialization value	
Creating and configuring VAR FB1				
FUNCTION_BLOCK_CREATE	777	VAR_R4_0+1	Creates VAR FB1	
VAR_R4_0+0	4129	55.55	Setting initialization value	
VAR_R4_1+0	4137	66.66	Setting initialization value	
VAR_R4_2+0	4145	77.77	Setting initialization value	
VAR_R4_3+0	4153	88.88	Setting initialization value	
Creating, configuring and defining the connection structure of VARITH FB0				
FUNCTION_BLOCK_CREATE	777	VARITH_MODE+0	Creates VARITH FB0	
VARITH_DIMENSION+0	9224	4	Define the number of elements	
VARITH_IN_A1_PARID+0	9232	VAR_R4_0+0	Link input A1 with R4 var 0 FB0	
VARITH_IN_A2_PARID+0	9240	VAR_R4_1+0	Link input A2 with R4 var 1 FB0	
VARITH_IN_A3_PARID+0	9248	VAR_R4_2+0	Link input A3 with R4 var 2 FB0	
VARITH_IN_A4_PARID+0	9256	VAR_R4_3+0	Link input A4 with R4 var 3 FB0	

VARITH_IN_B1_PARID+0	9272	VAR_R4_0+1	Link input B1 with R4 var 0 FB1
VARITH_IN_B2_PARID+0	9280	VAR_R4_1+1	Link input B2 with R4 var 1 FB1
VARITH_IN_B3_PARID+0	9288	VAR_R4_2+1	Link input B3 with R4 var 2 FB1
VARITH_IN_B4_PARID+0	9296	VAR_R4_3+1	Link input B4 with R4 var 3 FB1
VARITH_MODE+0	9216	3	Enable FB in mode scalar product

Steps to follow during the exercise

- Change values of R4 variables from mapp Cockpit.
- Trace or read vector arithmetic output VARITH_VALUE1+0 (9312) in mapp Cockpit.

Exercise: Vector addition (self-paced)



Steps to follow during the exercise

- Use the parameters listed in "Exercise: Scalar product of two vectors (inner product)"
- Change mode of function block VARITH_MODE+0 (9216) from 3 to 1 for vector addition.
- Change values of R4 variables from mapp Cockpit.
- Trace the ParIDs shown below in mapp Cockpit.
 - VARITH_VALUE1+0 (9312)
 - VARITH_VALUE2+0 (9320)
 - VARITH_VALUE3+0 (9328)
 - VARITH_VALUE4+0 (9336)

7.2 Not frequently used function blocks

7.2.1 BIT - Bit operation

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \
ACOPOS Function Blocks \ BIT - Bit Operation \ Example

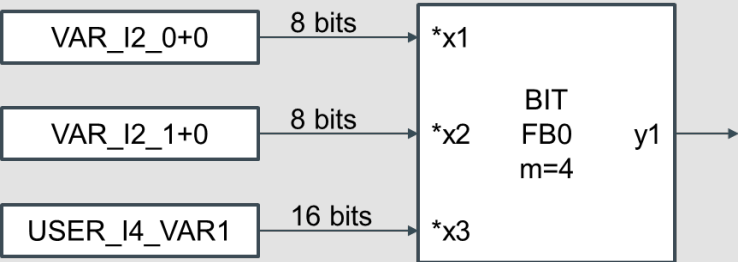
About the function block

This function block carries out various bit manipulations with n input values and n internal operands. The bit operation is specified by the mode.

This function block provides different modes of operation as listed below .

- Logical AND operation by bit
- Logical OR operation by bit
- Logical EXCLUSIVE OR operation by bit
- Assembly (compiling, compressing) - Adjusting more than one variable in I4 data type as per bit configuration
- Inverting bits
- Extracting left
- Extracting right (decompiling, separating) - Segregating bit wise information as per configuration from input

Exercise: Compressing/Assembling data (live)



Parameter passing

Setting value of user I4 variable 1

USER_I4_VAR1	584	0x21ABBA12	Setting value
--------------	-----	------------	---------------

Creating and defining VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I2_0+0	Creating VAR FB0
VAR_I2_0+0	4160	0xABCD	Setting value
VAR_I2_1+0	4168	0xDCBA	Setting value

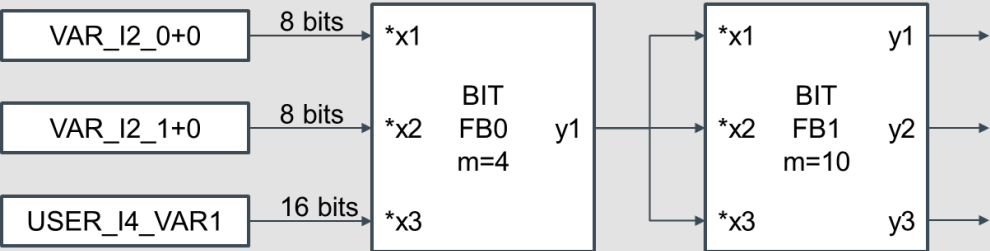
Creating, configuring and defining the connection structure of BIT FB0

FUNCTION_BLOCK_CREATE	777	BIT_MODE+0	Creating BIT FB0
BIT_A1+0	12808	255	AND operation with bit mask=0xFF (255)
BIT_A2+0	12816	255	AND operation with bit mask=0xFF (255)
BIT_A3+0	12824	0xFFFF	AND operation with bit mask=0xFFFF (65535)
BIT_B1+0	12872	0	To shift 0 bit to the left
BIT_B2+0	12880	8	To shift 8 bits to the left
BIT_B3+0	12888	16	To shift 16 bits to the left
BIT_IN1_PARID+0	12936	VAR_I2_0+0	Link input1 with I2 variable 0 of FB0
BIT_IN2_PARID+0	12944	VAR_I2_1+0	Link input2 with I2 variable 1 of FB0
BIT_IN3_PARID+0	12952	USER_I4_VAR1	Link input3 with I4 user variable 1
BIT_MODE+0	12800	4	Enable FB in mode Assemble

Steps to follow during the exercise

- Change value of VAR_I2_0+0 (4160), VAR_I2_1+0 (4168) and USER_I4_VAR1 (584) and observe output BIT_VAL-UE1+0 (13016) in mapp Cockpit trace.
- Record trace in mapp Cockpit to find compression or assembling of data according to the configuration.

Exercise: Decompile/Separating data (live)



Parameter passing

Setting value of user I4 variable 1

USER_I4_VAR1	584	0x21ABBA12	Setting value
--------------	-----	------------	---------------

Creating and defining VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I2_0+0	Creating the VAR FB0
VAR_I2_0+0	4160	0xABCD	Setting value
VAR_I2_1+0	4168	0xDCBA	Setting value

Creating, configuring and defining the connection structure of BIT FB0

FUNCTION_BLOCK_CREATE	777	BIT_MODE+0	Creating BIT FB0
BIT_A1+0	12808	0xFF	AND operation with bit mask=0xFF (255)
BIT_B1+0	12872	0	To shift 0 bit to the left
BIT_IN1_PARID+0	12936	VAR_I2_0+0	Link input1 with I2 variable 0 of FB0
BIT_A2+0	12816	0xFF	AND operation with bit mask=0xFF (255)
BIT_B2+0	12880	8	To shift 8 bits to the left
BIT_IN2_PARID+0	12944	VAR_I2_1+0	Link input2 with I2 variable 1 of FB0
BIT_A3+0	12824	0xFFFF	AND operation with bit mask=0xFFFF (65535)
BIT_B3+0	12888	16	To shift 16 bits to the left
BIT_IN3_PARID+0	12952	USER_I4_VAR1	Link input3 with I4 user variable 1
BIT_MODE+0	12800	4	Enable FB in mode "Assemble"

Creating, configuring and defining the connection structure of BIT FB1

FUNCTION_BLOCK_CREATE	777	BIT_MODE+1	Creating the BIT FB1
BIT_A1+1	12809	255	AND operation with bit mask=0xFF
BIT_B1+1	12873	0	To shift 0 bit to the right
BIT_IN1_PARID+1	12937	BIT_VALUE1+0	Link input1 with output value1 of BIT FB0
BIT_A2+1	12817	255	AND operation with bit mask=0xFF
BIT_B2+1	12881	8	To shift 8 bits to the right
BIT_IN2_PARID+1	12945	BIT_VALUE1+0	Link input1 with output value1 of BIT FB0
BIT_A3+1	12825	65535	AND operation with bit mask=0xFFFF
BIT_B3+1	12889	16	To shift 16 bits to the right
BIT_IN3_PARID+1	12953	BIT_VALUE1+0	Link input1 with output value1 of BIT FB0
BIT_MODE+1	12801	10	Enable FB in mode separating

Steps to follow during the exercise

- Change value of VAR_I2_0+0 (4160), VAR_I2_1+0 (4168) and USER_I4_VAR1 (584) and observe output BIT_VALUE1+0 (13016) in mapp Cockpit trace.
- Record trace in mapp Cockpit to find compression or assembling of data according to the configuration.
- BIT_VALUE1+0 (13016) is used as input for separation purpose. However, BIT FB1 outputs BIT_VALUE1+1 (13017), BIT_VALUE2+1 (13025) and BIT_VALUE3+1 (13033) may not be the same as inputs VAR_I2_0+0 (4160), VAR_I2_1+0 (4168) and USER_I4_VAR1 (584) due to masking and the configured number of bits.

7.2.2 COUNT - Counter

Automation Help: Examples



Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ COUNT - Counter \ Application examples

About function block

The COUNT function block counts events. The type of event evaluation and the counting direction are defined as a mode.

This function block provides a different mode configuration based on counting direction, event comparison and level sensitivity. Details are provided below.

Event comparison condition

- Value1 smaller than Value2 (ev1 < ev2)
- Value1 equal to Value2 (ev1 == ev2)
- Value1 less than or equal to Value2 (ev1 <= ev2)
- Value1 greater than Value2 (ev1 > ev2)
- Value1 not equal to Value2 (ev1 != ev2)
- Value1 greater than or equal to Value2 (ev1 >= ev2)

Counting direction

- Up counting
- Down counting

Counting based on level or edge sensitivity

- Level sensitive
- Positive edge
- Negative edge
- Positive and Negative edge

Exercise: Measurement of the duration of a trigger signal level (self-paced)



The provided sample in Automation Help is partially adapted here. For successful solution, implement the parameter table as listed.

Creating and configuring VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creates VAR FB0
VAR_I4_0+0	4096	400	Ta = 400µs
VAR_I2_0+0	4160	0	Resetting input if set to high

Creating, configuring and defining the connection structure of LOGIC FB0

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Creates LOGIC FB0
LOGIC_IN1_PARID+0	3080	STAT_TRIGGER1	Link input 1 with status trigger1
LOGIC_IN2_PARID+0	3088	VAR_I2_0+0	Link input 2 with I2 variable of FB0
LOGIC_MODE+0	3072	4	Enable FB with (NOT x1 AND x2)

Creating, configuring and defining the connection structure of COUNT FB0

FUNCTION_BLOCK_CREATE	777	COUNT_MODE+0	Creates COUNT FB0
COUNT_EV1_PARID+0	13832	STAT_TRIGGER1	Links event input1 with status trigger1
COUNT_SET_TRIG_PARID+0	13848	LOGIC_VALUE+0	Links trigger input with LOGIC FB0 output for resetting the counter value to 0

COUNT_MODE+0	13824	5	Enable FB with mode ev1 != 0, level-sensitive. I.e. count upward as long as the value on the event input 1 is unequal to 0
--------------	-------	---	---

Creating, configuring and defining the connection structure of ARITH FB0

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Creates ARITH FB0
ARITH_IN1_PARID+0	3592	COUNT_VALUE+0	Link input 1 with counter result value
ARITH_IN2_PARID+0	3600	VAR_I4_0+0	Link input 2 with I4 variable of FB0
ARITH_MODE+0	3584	3	Enable FB in mode multiplication

Steps to follow/observe during the exercise

- Find the time duration of the input indicating how long it was on/high with this exercise.
- Force or provide the input to trigger 1 (STAT_TRIGGER1 (463)) to get a time duration on ARITH_VALUE_I4+0 (3608).
- Reset the counter value by providing a VAR_I2_0+0 (4160) value not equal to 0 (ZERO) and set it back to 0 so the counter can increment again if the trigger 1 level goes high.
- During the time trigger 1 status (STAT_TRIGGER1 (463)) and/or VAR_I2_0+0 (4160) value is changing, observe outputs ARITH_VALUE_I4+0 (3608), COUNT_VALUE+0 (13880), COUNT_EV_STATUS+0 (13896) in mapp Cockpit trace.

Exercise: Count number of movement cycles (live)

Implement/configure COUNT function block according to the example provided in Automation Help.

- Enter a "to and fro" movement little higher than USER_I4_VAR1 (584) so the counter function block can recognize an event.
- The counter comparator status will automatically reset counter value to the defined counter set value.
- During the time of "to and fro movement", the user should observe outputs COUNT_VALUE+0 (13880), COUNT_EV_STATUS+0 (13896) and COUNT_CMP_STATUS+0 (13888) in mapp Cockpit trace.

7.2.3 CURVE - Curve function f(x)

Automation Help: Examples

Motion control \ ACP10/ARNCO \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ CURVE - Curve Function f(x) \ Example

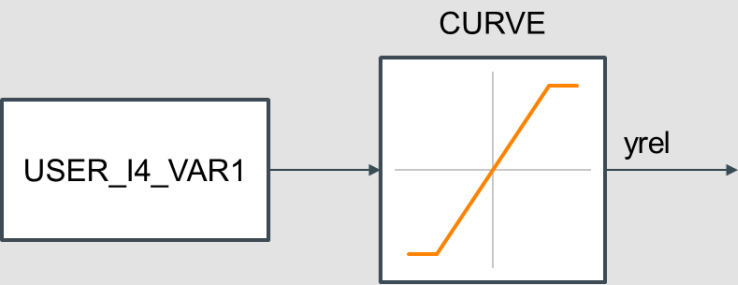
About the function block

The CURVE function block provides a function $f(x)$ between the input and output value. This function is implemented with cam polynomials. Therefore, all existing mechanisms for creating and downloading cams can be used. Some applications of the CURVE function block include: Creating characteristic curves, cyclic overrides and implementation of correction functions.

This function block provides different mode configurations for the input to cam and output relationship as shown below.

- Non-cyclic (limited input range)
- Cyclic (cam is repeated and factors could be adapted while entering into next interval)
- Non-cyclic centrally symmetrical (exactly opposite x & y symmetry across origin)
- Non-cyclic axially symmetrical (only y axis symmetry across origin)
- Cyclic with periodic input signal (conversion of input to I4 by adding interval in progressive directions)

Exercise: Limiter (live)



Parameter passing

Initializing user variable			
USER_I4_VAR1	584	100	Initializing I4 User var1
Creating, configuring and defining the connection structure of CURVE FB0			
FUNCTION_BLOCK_CREATE	777	CURVE_MODE+0	Creates CURVE FB0
CURVE_IN_PARID+0	8200	USER_I4_VAR1	Link input with I4 var 0 FB0
CURVE_AUT_DATA_INDEX+0	8208	0xFFFF	Use pre-compiled 1:1 straight line
CURVE_X_FACTOR+0	8272	10000	Limit value
CURVE_Y_FACTOR+0	8280	10000	Limit value
CURVE_MODE+0	8192	3	Enable FB in mode “centrally symmetrical”

Steps to follow during the exercise

- Change the value of USER_I4_VAR1 (584) in mapp Cockpit and observe that curve function block output CURVE_VALUE_REL_R4+0 (8256) is limited in the positive and negative range because the X & Y factors are same and this defines limiting range.
- Read or trace ParIDs USER_I4_VAR1 (584) and CURVE_VALUE_REL_R4+0 (8256) in mapp Cockpit.

Exercise: Signal conversion - From Automation Help (self-paced)

Steps to follow during the exercise

- Use parameter table for this exercise provided in Automation Help.
- Replace MA1_CYCLIC_POS (593) with USER_I4_VAR1 (584) for this exercise.
- Create a self incrementing variable on the PLC until the specified period (0 → 35999 → 0) and configure function block “MC_BR_CyclicProcessParID_AcpAx” to write a self-incrementing variable to USER_I4_VAR1.
- Observe USER_I4_VAR1(584), CURVE_VALUE_I4+0 (8216) and CAMCON_VALUE+0 (10848) in mapp Cockpit trace.

7.2.4 FIFO - First-In-First-Out Memory

Automation Help: Examples



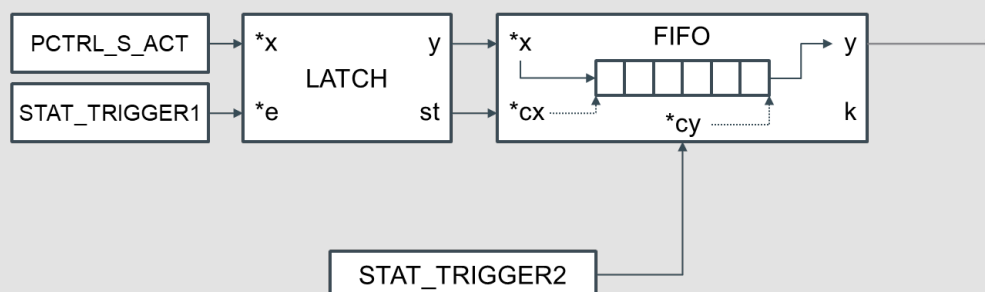
Motion control \ ACP10/ARNC0 \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ FIFO - First-In-First-Out Memory \ Example

About function block

In the basic method of operation, the FIFO function block is similar to a shift register: The data appears in the same sequence as it was entered. The first value entered (First In) is also the first value read (First Out). Unlike a shift register, this procedure can run completely asynchronous in a FIFO (i.e. the output rate is not dependent on the input rate). This function block provides a different mode configuration as shown below .

- Stand-by mode (disabled or frozen)
- No input and shift through if buffer is full
- Shift through even though buffer is full

Exercise: Buffering/storing position samples in FIFO memory (live)



Parameter passing

Creating, configuring and defining the connection structure of LATCH FB0

FUNCTION_BLOCK_CREATE	777	LATCH_MODE+0	Creating LATCH FB0
LATCH_IN_PARID+0	9736	PCTRL_S_ACT	Link input to Position controller actual position
LATCH_EV_PARID+0	9744	STAT_TRIGGER1	Link input1 with User I4 variable 1
LATCH_EV_TYPE+0	9752	4	Positive edge + evaluation of min signal width
LATCH_EV_WIDTH_MIN+0	9760	200	Configure minimum signal width
LATCH_MODE+0	9728	4	Enable FB in all trigger valid

Creating, configuring and defining the connection structure of FIFO FB0

FUNCTION_BLOCK_CREATE	777	FIFO_MODE+0	Creating FIFO FB0
FIFO_MAX_LENGTH+0	11784	5	FIFO length for 5 elements
FIFO_VALUE2_DISTANCE+0	11792	1	Y2 output 1 element before last element

FIFO_IN_PARID+0	11800	LATCH_VALUE+0	Link input with latch value
FIFO_IN_EV_PARID+0	11808	LATCH_STATUS+0	Link input event with latch status
FIFO_OUT_EV_PARID+0	11816	STAT_TRIGGER2	Link output event with status trigger 2
FIFO_MODE+0	11776	2	Enable FB in mode “no input when FIFO is full”

Steps to follow during the exercise

- Enter a velocity movement for the axis in positive direction and meantime force/provide input to trigger 1 of ACOPOS.
- Trace following ParIDs in mapp Cockpit
 - LATCH_VALUE+0 (9824)
 - LATCH_STATUS+0 (9840)
 - STAT_TRIGGER1 (463)
 - PCTRL_S_ACT (111)
 - FIFO_ACT_LENGTH (11824)
 - FIFO_VALUE_I4+0 (11832)
 - FIFO_VALUE2_I4+0 (11848)
 - STAT_TRIGGER2 (464)
- Witness during testing/trial with FIFO that if buffer is full and it does not accept any input.
- Observe that FIFO buffer reduce by one record per forced/provided input to trigger 2 of ACOPOS.

Exercise: Change FIFO mode to 3 (shift through if FIFO buffer is full) (self-paced)

Steps to follow/observe during the exercise

- Change FIFO_MODE+0 (11776) to 3 from 2.
- Repeat steps listed in first exercise and observe that FIFO value shifts through even when FIFO is full.

7.2.5 IPL - Interpolator

Automation Help: Examples



Motion control \ ACP10/ARNC0 \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks \ IPL - Interpolator \ Example

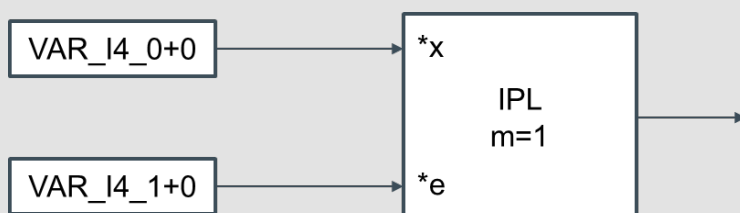
About the function block

The IPL function block allows varying sampling systems to be adjusted. Interpolation uses existing data points $x(t_n)$ to generate additional values $y(t_i)$. This is how, for example, the set value steps from the PLC task class cycle are interpolated and adjusted to the ACOPOS cycle.

This function block is mostly used when entering set values via the POWERLINK network if a master cycle of 400 μ s cannot be set. This function block provides a similar mode of interpolation as provided by master coupling and is shown below .

- Linear interpolation
- Quadratic interpolation
- Quadratic interpolation, without overshoot, increased delay

Exercise: Simple linear interpolation (live)



Creating and configuring VAR FB 0

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creates VAR FB0
VAR_I4_0+0	4096	0	Initializing I4 var 0 FB0
VAR_I4_1+0	4104	0	Initializing I4 var 1 FB0

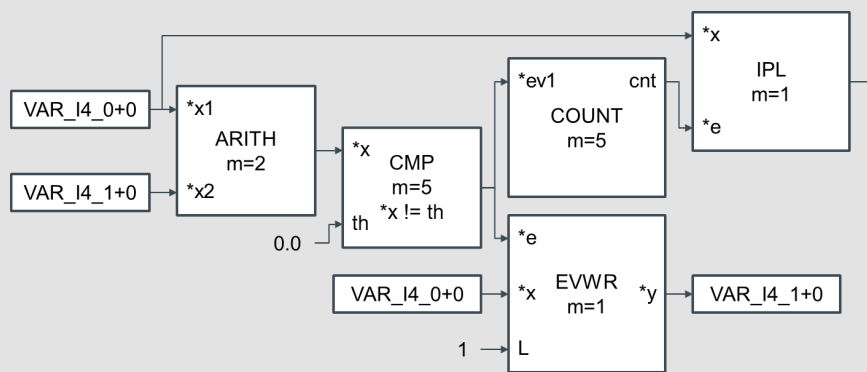
Creating, configuring and defining the connection structure of IPL FB0

FUNCTION_BLOCK_CREATE	777	IPL_MODE+0	Creates IPL FB0
IPL_IN_PARID+0	8712	VAR_I4_0+0	Link input with I4 var 0 FB0
IPL_EV_PARID+0	8760	VAR_I4_1+0	Link input event with I4 var 1 FB0
IPL_CYCLE_TIME+0	8720	10000	Cycle time (in μ s) for the set values
IPL_EXTRAPOLATION_TIME+0	8728	0	Time (in ms) when set values are missing
IPL_MODE+0	8704	1	Enable FB with linear interpolation

Steps to follow during the exercise

- Change value of VAR_I4_0+0 (4096) and VAR_I4_1+0 (4104) via mapp Cockpit so the interpolator can generate a ramp output from the old value to the new value of VAR_I4_0+0 (4096) within the specified interpolation cycle time.
- A condition to be taken care of is ensuring a new value or event is not provided to the interpolator before interpolation is completed.
- Observe value of VAR_I4_0+0 (4096), VAR_I4_1+0 (4104) and IPL_VALUE_I4+0 (8736) in mapp Cockpit trace.

Exercise: Auto trigger to IPL on input value changed (self-paced)



Creating and configuring VAR FB0

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Creates VAR FB0
VAR_I4_0+0	4096	0	Initializing I4 var 0 FB0
VAR_I4_1+0	4104	0	Initializing I4 var 1 FB0

Creating, configuring and defining the connection structure of ARITH FB0

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Creates ARITH FB0
ARITH_IN1_PARID+0	3592	VAR_I4_0+0	Link input 1 with User set value
ARITH_IN2_PARID+0	3600	VAR_I4_1+0	Link input 2 with copy User set value
ARITH_MODE+0	3584	2	Enable FB in mode subtraction

Creating, configuring and defining the connection structure of CMP FB0

FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Creates CMP FB0
CMP_IN_PARID+0	6656	ARITH_VALUE_I4+0	Link input with I4 ARITH output

CMP_THRESHOLD+0	6664	0.0	Define level
CMP_MODE+0	6688	5	Input is not equal to threshold

Creating, configuring and defining the connection structure of COUNT FB0

FUNCTION_BLOCK_CREATE	777	COUNT_MODE+0	Creates COUNT FB0
COUNT_EV1_PARID+0	13832	CMP_VALUE+0	Links event input1 with compare output FB0
COUNT_MODE+0	13824	5	Enable FB with mode ev1 != 0, level-sensitive. I.e. count upward as long as the value on the event input 1 is unequal to 0

Creating, configuring and defining the connection structure of EVWR FB0

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Creates EVWR FB0
EVWR_EVENT_PARID+0	4608	CMP_VALUE+0	Link event with compare output FB0
EVWR_IN_PARID+0	4616	VAR_I4_0+0	Link input 1 with User set value
EVWR_WR_PARID+0	4632	VAR_I4_1+0	Link input 1 with copy User set value
EVWR_EVENT_LEVEL+0	4624	1	Event level high of compare output
EVWR_MODE+0	4640	1	Enables the FB with mode "edge sensitive"

Creating, configuring and defining the connection structure of IPL FB0

FUNCTION_BLOCK_CREATE	777	IPL_MODE+0	Creates IPL FB0
IPL_IN_PARID+0	8712	VAR_I4_0+0	Link input with I4 var 0 FB0
IPL_EV_PARID+0	8760	COUNT_VALUE+0	Link input event with counter value
IPL_CYCLE_TIME+0	8720	10000	Cycle time (in μ s) for the set values
IPL_MODE+0	8704	1	Enable FB with linear interpolation

Steps to follow during the exercise

- Change value of VAR_I4_0+0 (4096) via mapp Cockpit so the interpolator can generate a ramp output from the old value to the new value of VAR_I4_0+0 (4096) within the specified interpolation cycle time.
- A condition to be taken care of is ensuring a new value is not provided to the interpolator before interpolation is completed.
- Observe value of VAR_I4_0+0 (4096), VAR_I4_1+0 (4104) and IPL_VALUE_I4+0 (8736) in mapp Cockpit trace.
- ARITH FB0 subtracts VAR_I4_0+0 (4096) and VAR_I4_1+0 (4104), so a difference will be generated. ARITH_VALUE_I4+0 (3608) is provided as input to CMP FB0 with mode 5 (IN != TH), which compares the input to the threshold and generates output CMP_VALUE+0 (6696) when the input is not equal to the threshold. CMP_VALUE+0 (6696) is used as an event ParID for EVWR FB0 and an event ParID1 for COUNT FB0. COUNT FB0 will increment the counter value by 1 with every scan. However, EVWR FB0 will write value of VAR_I4_0+0 (4096) to VAR_I4_1+0 (4104) and, in next scan cycle, the ARITH FB output will be zero. Eventually, it will turn CMP_VALUE+0 (6696) to zero as well. Updated counter value (COUNT_VALUE+0 (13880)) will trigger interpolation to the new VAR_I4_0+0 (4096) value from the old.
- Trace the ParIDs shown below in mapp Cockpit trace with trace time of 400 μ s (0.0004 s).
 - VAR_I4_0+0 (4096)
 - VAR_I4_1+0 (4104)
 - ARITH_VALUE_I4+0 (3608)
 - CMP_VALUE+0 (6696)
 - COUNT_VALUE+0 (13880)
 - IPL_VALUE_I4+0 (8736)

7.3 Hardware-dependent

These hardware-dependent function blocks, which are only available in the ACOPOS servo drive series, are shown below

- AIO - Analog IO interface
- DIO - Digital IO interface

The function block gets enabled automatically by the ACOPOS drive during booting after the ACOPOS firmware detects a plug-in card. Supported ParIDs change based on different plug-in cards. These two function blocks are dependent on plug-in card selections for ACOPOS, ACOPOSMulti, ACOPOS P3 devices. These function blocks are not supported by ACOPOSmicro, ACOPOSmotor, ACOPOSremote devices since there is no possibility to insert plug-in card in those ACOPOS product groups. Supported modules for AIO and DIO function blocks are shown below.

AIO - Analog IO interface

Group	Plug-in card	Input and outputs
ACOPOS	8AC131.60-1	Analog input 1..2 Digital input/output 1..2
ACOPOS P3	8EAC0134.000-1	Analog input & output 1..3 (Configurable output: $\pm 10V$ / 0..20mA) Digital input/output 1..10
ACOPOSMulti	8BAC0132.000-1	Analog input 1..4

DIO - Digital IO interface

Group	Plug-in card	Input and outputs
ACOPOS	8AC130.60-1	Digital input/output 1..8 Digital output 9..10
	8AC131.60-1	Analog input 1..2 Digital input/output 1..2
ACOPOS P3	8EAC0130.000-1	Digital input/output 1..10
	8EAC0134.000-1	Analog input & output 1..3 Digital input/output 1..10
ACOPOSMulti	8BAC0130.000-1	Digital input 1..2 → DI7..8 (DIO channel) Digital output 1 → DO1 (DIO channel) Digital output 2 → DO3 Digital output 3..4 → DO5..6
	8BAC0130.001-1	Digital output 1 → DO1 (DIO channel) Digital output 2 → DO3 Digital output 3..6 → DO5..8
	8BAC0133.000-1	Only encoder emulation supported

For detailed information and supported ParIDs, functionality and examples, visit Automation Help using the path shown below.



Motion control \ ACP10/ARNC0 \ Reference manual \ ACOPOS Drive Functions \ ACOPOS Function Blocks

- AIO - Analog IO interface
- DIO - Digital IO interface

8 Solutions of group exercise

8.1 Exercise: Encoder emulation

8.1.1 Phase 1

About the solution

ARITH FB0 works as a time counter and adding a scan time of 400 μ s via MUX FB1. MUX FB1 works as a time counter resetter based on a trigger1 event. This solution is not 100% correct. However, this solution is the base for a 100% correct solution that will be shown in the next phase as part of the next group exercise.

MUX FB0 works as a time counter value storage/holding register because it loops the output value MUX_VALUE_R4 to MUX_IN8_PARID. ARITH FB1 is a speed calculator for a virtual encoder based on 36000 units for one revolution and a time counting value of Tperiod for trigger1 edge to edge.

Schematic

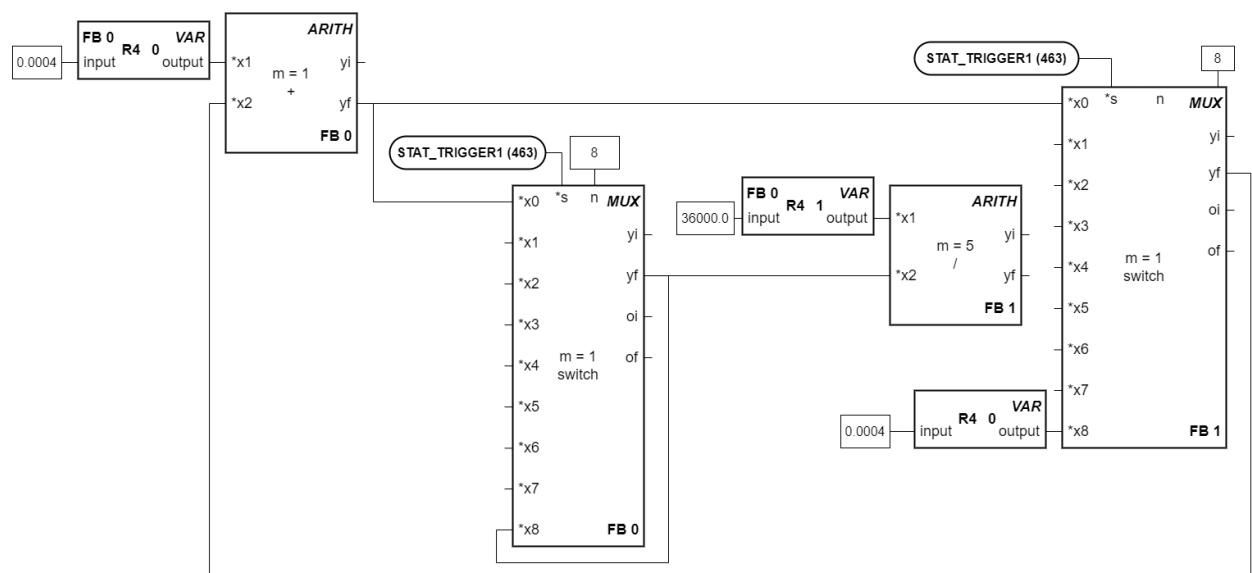


Figure 12: Solution: Group exercise - Virtual encoder - Phase-1

Parameter table

Function block creation			
FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Variable for value passing
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Timer counter
FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Time counter value holding
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+1	Speed calculation
FUNCTION_BLOCK_CREATE	777	MUX_MODE+1	Time counter resetting
VAR - 0 - Variable value passing			
VAR_R4_0+0	4128	0.0004	400 μ s scan time for counting
VAR_R4_1+0	4136	36000.0	1 revolution counter for virtual encoder
ARITH - 0 - Time counter			
ARITH_IN1_PARID+0	3592	VAR_R4_0+0	400 μ s scan time for counting
ARITH_IN2_PARID+0	3600	MUX_VALUE_R4+1	Value from time counter resetter

ARITH_MODE+0	3584	1	Addition mode for timer counting
--------------	------	---	----------------------------------

MUX - 0 - Time counter value holder

MUX_SELECTOR_PARID+0	11272	STAT_TRIGGER1	Sensor is connected to trigger 1
MUX_SELECTOR_MAX+0	11280	8	Status of trigger 1 is 0 and 8
MUX_IN0_PARID+0	11320	ARITH_VALUE_R4+0	Time counter value if trigger1 is off
MUX_IN1_PARID+0	11328	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN2_PARID+0	11336	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN3_PARID+0	11344	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN4_PARID+0	11352	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN5_PARID+0	11360	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN6_PARID+0	11368	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN7_PARID+0	11376	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN8_PARID+0	11384	MUX_VALUE_R4+0	Holding old value of timer counter
MUX_MODE+0	11264	1	Mode - active as Switch

ARITH - 1 - Speed for Virtual Encoder

ARITH_IN1_PARID+1	3593	VAR_R4_1+0	Counts of Virtual encoder
ARITH_IN2_PARID+1	3601	MUX_VALUE_R4+0	Time between trigger to trigger
ARITH_MODE+1	3585	5	Dividing for speed calculation

MUX - 1 - Time counter resetter

MUX_SELECTOR_PARID+1	11273	STAT_TRIGGER1	Sensor is connected on trigger 1
MUX_SELECTOR_MAX+1	11281	8	Status of trigger 1 is 0 and 8
MUX_IN0_PARID+1	11321	ARITH_VALUE_R4+0	Time counter value if trigger1 is off
MUX_IN1_PARID+1	11329	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN2_PARID+1	11337	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN3_PARID+1	11345	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN4_PARID+1	11353	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN5_PARID+1	11361	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN6_PARID+1	11369	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN7_PARID+1	11377	ARITH_VALUE_R4+0	This input has no meaning, just to pass
MUX_IN8_PARID+1	11385	VAR_R4_0+0	Resetting time counter to 400 μ s
MUX_MODE+1	11265	1	Mode - active as Switch

8.1.2 Phase 2

About the solution

Trigger1 is delayed by 400 μ s using DELAY FB0 and the output from DELAY is passed on as an input to LOGIC FB0. The combination of trigger1 and delayed trigger1 generates a rising edge of trigger1 in modes X1 and !X2.

With a marginal change to parameter passing for phase 1, this solution achieves the required behavior and functionality to calculate speed for the virtual axis.

ARITH FB0 works as a time counter and adding scan time of 400 μ s via MUX FB1 and MUX FB1 works as a time counter resetter based on the event from the rising edge of trigger1 via a LOGIC FB0 output. MUX FB0 works as a storage/hold-ing register for time counter values because it loops output value MUX_VALUE_R4 to MUX_IN0_PARID and a new time

counter value is available via LOGIC FB0 on the rising edge of trigger1. ARITH FB1 is a speed calculator for a virtual encoder based on 36000 units for one revolution and a time counting value of Tperiod for trigger1 edge to edge .

Schematic

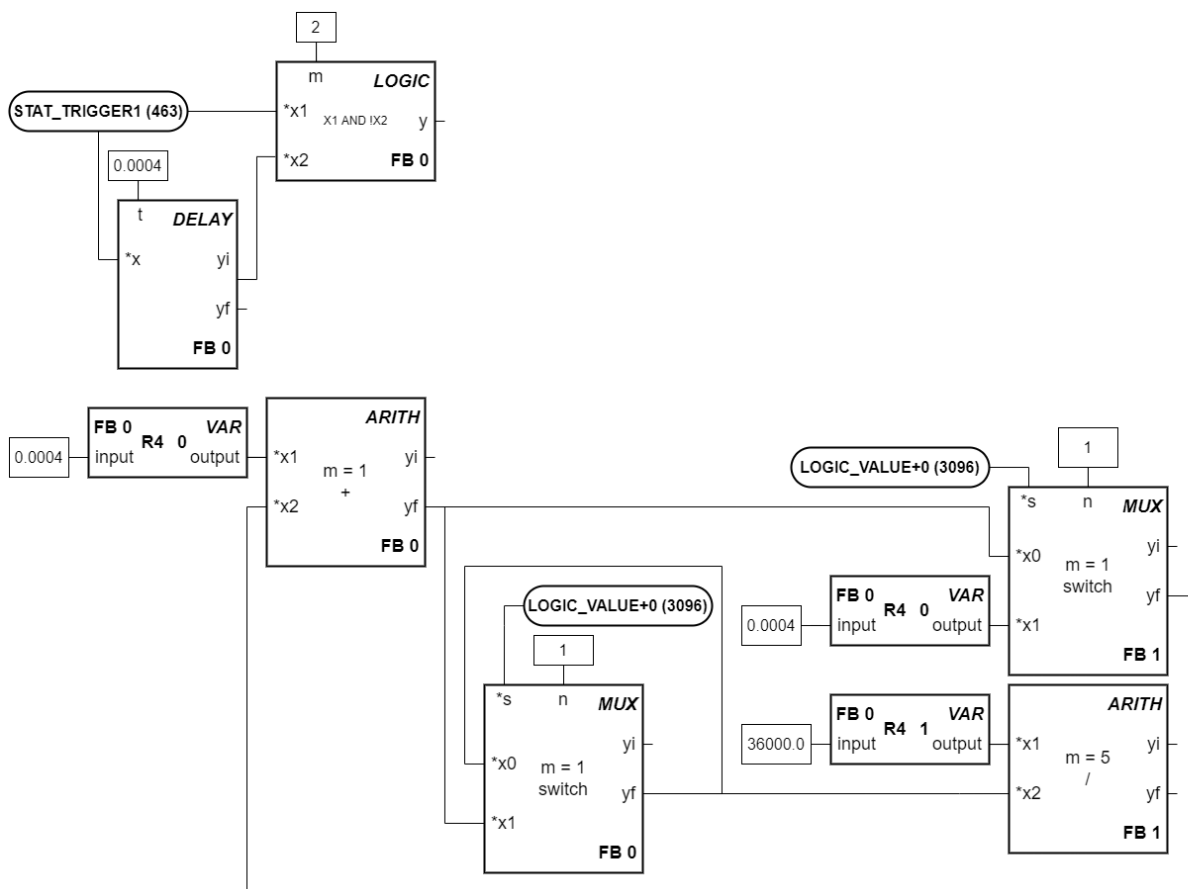


Figure 13: Solution: Group exercise - Virtual encoder - Phase 2

Parameter table

DELAY - 0 - Delay trigger signal by ACOPOS function block execution cycle time

FUNCTION_BLOCK_CREATE	777	DELAY_IN_PARID+0	Delay signal by 400 µs
DELAY_IN_PARID+0	7168	STAT_TRIGGER1	Sensor is connected to trigger 1
DELAY_TIME+0	7176	0.0004	Delay by 400 µs - ACOPOS cycle time

LOGIC - 0 - Generating rising edge

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Rising edge generator
LOGIC_IN1_PARID	3080	STAT_TRIGGER1	Sensor is connected to trigger 1
LOGIC_IN2_PARID	3088	DELAY_VALUE_I4+0	Delayed signal input
LOGIC_MODE	3072	2	X1 AND !X2

Function block creation

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Variable for value passing
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Timer counter
FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Time counter value holding
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+1	Speed calculation
FUNCTION_BLOCK_CREATE	777	MUX_MODE+1	Time counter resetting

VAR - 0 - Variable value passing

VAR_R4_0+0	4128	0.0004	400 µs scan time for counting
VAR_R4_1+0	4136	36000.0	1 revolution counter for virtual encoder

ARITH - 0 - Time counter

ARITH_IN1_PARID+0	3592	VAR_R4_0+0	400 µs scan time for counting
ARITH_IN2_PARID+0	3600	MUX_VALUE_R4+1	Value from time counter resetter
ARITH_MODE+0	3584	1	Addition mode for time counting

MUX - 0 - Time counter value holder

MUX_SELECTOR_PARID+0	11272	LOGIC_VALUE+0	Rising edge of sensor
MUX_SELECTOR_MAX+0	11280	1	Logic has two states 0 and 1
MUX_IN0_PARID+0	11320	MUX_VALUE_R4+0	Holding old value of time counter
MUX_IN1_PARID+0	11328	ARITH_VALUE_R4+0	Time counter value on edge
MUX_MODE+0	11264	1	Mode - active as Switch

ARITH - 1 - Speed for Virtual Encoder

ARITH_IN1_PARID+1	3593	VAR_R4_1+0	Counts of Virtual encoder
ARITH_IN2_PARID+1	3601	MUX_VALUE_R4+0	Time between trigger to trigger
ARITH_MODE+1	3585	5	Dividing for speed calculation

MUX - 1 - Time counter resetter

MUX_SELECTOR_PARID+1	11273	LOGIC_VALUE+0	Rising edge of sensor
MUX_SELECTOR_MAX+1	11281	1	Logic has two states 0 and 1
MUX_IN0_PARID+1	11321	ARITH_VALUE_R4+0	Time counter value if no edge
MUX_IN1_PARID+1	11329	VAR_R4_0+0	Resetting time counter to 400 µs
MUX_MODE+1	11265	1	Mode - active as Switch

8.2 Exercise: Registration mark correction

About the solution

LATCH_MODE is a combination of 1 (window shift from latch position), +16 (old latch value remains without trigger) and +32 (no latch status without trigger), which makes it possible to have LATCH_STATUS and LATCH_VALUE only if a valid trigger is found. LATCH_EV_TYPE is set to 18, and that is a combination of 2 (positive edge and evaluation of the signal width in the range smin to smax) and +16 (activated on the high level, the rest of the signal width is evaluated). LATCH_DELTA_IV will provide the result of registration mark deviation from the expected position, and this has to be corrected. Correction value would be a maximum half of the interval value LATCH_POS_IV, so it must be limited. Therefore, LATCH_DELTA_IV is passed through a couple of MINMAX function blocks in order to limit correction. The output of the MINMAX function block is multiplied by LATCH_STATUS so the correction value is available only for one scan cycle, and after that, ARITH_VALUE_I4 will be ZERO.

ARITH_VALUE_I4+0 is added to the self output of ARITH FB1, so whenever correction is available, output ARITH_VALUE_I4+1 is increased by the correction value. ARITH_VALUE_I4+1 is set to an input for MPGEN at MPGEN_SET_VALUE_PARID, MPGEN will generate a motion profile based on a change of ARITH_VALUE_I4+1 from the old value to the new value.

MPGEN_VALUE_I4 must be configured as an additive slave axis in order to observe that correction is taking place .

Schematic

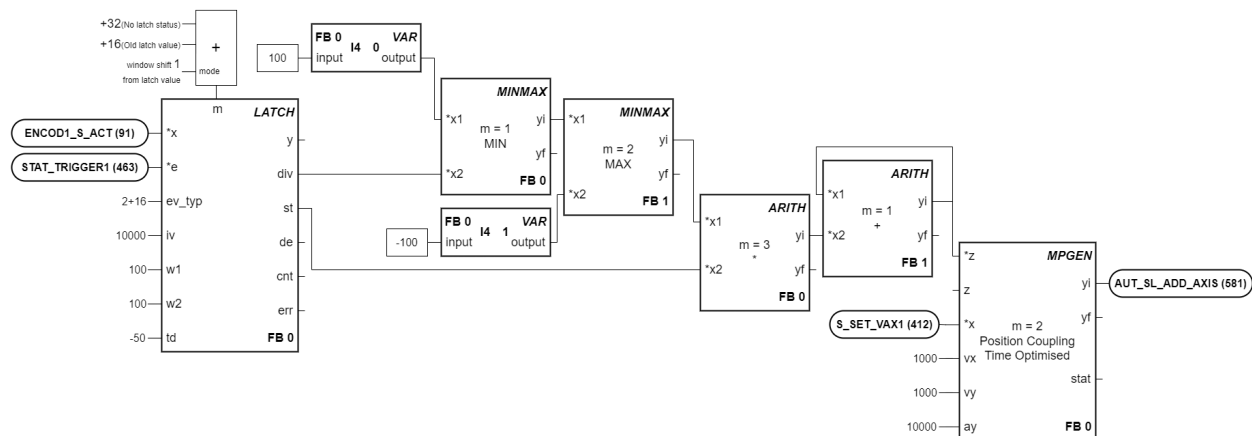


Figure 14: Solution: Group exercise - Registration mark correction

Parameter table

LATCH - 0 - Capturing registration mark via trigger 1

FUNCTION_BLOCK_CREATE	777	LATCH_MODE+0	Create LATCH - To capture RM
LATCH_IN_PARID+0	9736	ENCOD1_S_ACT	Encoder position to be captured on RM
LATCH_EV_PARID+0	9744	STAT_TRIGGER1	Registration mark sensor connection
LATCH_EV_TYPE+0	9752	18	Width evaluation + Active on high
LATCH_EV_WIDTH_MIN+0	9760	10	Minimum width of registration mark
LATCH_EV_WIDTH_MAX+0	9768	100	Maximum width of registration mark
LATCH_WINDOW1+0	9856	100	Window negative from expected
LATCH_WINDOW2+0	9864	100	Window positive from expected
LATCH_WINDOW_POS+0	9784	36000	Expected registration mark position
LATCH_POS_IV+0	9792	36000	Interval of registration mark
LATCH_POS_IV_ELONG+0	9800	0	Elongation of interval
LATCH_T_DELAY+0	9808	-50	Sensor delay in μ s
LATCH_MODE+0	9728	49	

VAR - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Variable for value passing
VAR_I4_0+0	4096	500	PM correction positive limit
VAR_I4_1+0	4104	-500	PM correction negative limit

MINMAX - 0 - Limiting correction in positive range

FUNCTION_BLOCK_CREATE	777	MINMAX_MODE+0	Create FB - Positive range limit
MINMAX_IN1_PARID+0	12312	VAR_I4_0+0	Positive range limit
MINMAX_IN2_PARID+0	12320	LATCH_DELTA_IV	Deviation of registration mark
MINMAX_MODE+0	12288	1	Minimum value from inputs

MINMAX - 1 - Limiting correction in negative range

FUNCTION_BLOCK_CREATE	777	MINMAX_MODE+1	Create FB - Negative range limit
-----------------------	-----	---------------	----------------------------------

MINMAX_IN1_PARID+1	12313	MINMAX_VALUE_I4+0	Output from positive limited range
MINMAX_IN2_PARID+1	12321	VAR_I4_1+0	Negative range limit
MINMAX_MODE+1	12289	2	Maximum value from inputs

ARITH - 0 - Correction value at the time of valid latch found

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Create FB - Getting correction value
ARITH_IN1_PARID+0	3592	MINMAX_VALUE_I4+1	Limited correction value
ARITH_IN2_PARID+0	3600	LATCH_STATUS+0	Valid registration mark status only for 1 cycle
ARITH_MODE+0	3584	3	Multiplication

ARITH - 1 - Adding correction value to self for profile generator

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+1	Create FB - Adding correction value
ARITH_IN1_PARID+1	3593	ARITH_VALUE_I4+0	Correction value if valid PM found
ARITH_IN2_PARID+1	3601	ARITH_VALUE_I4+1	Adding to self for profile generator
ARITH_MODE+1	3585	1	Addition

MPGEN - 0 - Registration mark correction profile generation

FUNCTION_BLOCK_CREATE	777	MPGEN_MODE+0	Create FB - Registration mark correction profile
MPGEN_SET_VALUE_PARID+0	5208	ARITH_VALUE_I4+1	ParID Value for profile generation
MPGEN_MA_PARID+0	5168	S_SET_VAX1	Master for profile generation
MPGEN_MA_V_MAX+0	5176	36000	Maximum master speed
MPGEN_V_MAX+0	5184	36000	Maximum profile speed
MPGEN_A_MAX+0	5192	360000	Maximum profile acceleration
MPGEN_MODE+0	5120	2	Position coupled profile generation

Slave additive axis in CAM AUTOMAT

The user should configure MPGEN_VALUE_I4+0 for slave additive axis in cam automat for correction to take place for registration mark.

Disclaimer



It is not guaranteed that the solution shown above works in practical applications. Practical application scenarios differ from the example. The solution may be workable if the participant tweaks it.

8.3 Exercise: Flying saw

8.3.1 Method 1

About the solution

COUNT_MODE is set to 21 (Ev1 != Ev2, PosEdge) to count holes. COUNT_CMP_VALUE is used to configure the number of holes and COUNT_CMP_STATUS will be high once the count for the number of holes reaches the configured value. COUNT_CMP_STATUS is also set as an input to COUNT_SET_TRG_PARID to reset the counter value so COUNT_CMP_STATUS will be high only for one cycle of scan and the COUNT function block is ready to count holes from COUNT_SET_VALUE (0).

Two EVWR function blocks use the event of COUNT_CMP_STATUS to store ARITH_VALUE_I4+0 (axis position value + cutting offset) and ARITH_VALUE_I4+1 (ARITH_VALUE_I4+0 (axis position value + cutting offset) + pulse duration) to VAR_I4_0+0 and VAR_I4_1+0 respectively.

The LOGIC function block uses inputs CMP_VALUE of FB0 and FB1. CMP function blocks compare axis position to stored value VAR_I4_0+0 (axis position value + cutting offset) and VAR_I4_1+0 (axis position value + cutting offset + pulse duration) to generate a pulse to trigger a movement for flying saw .

Schematic

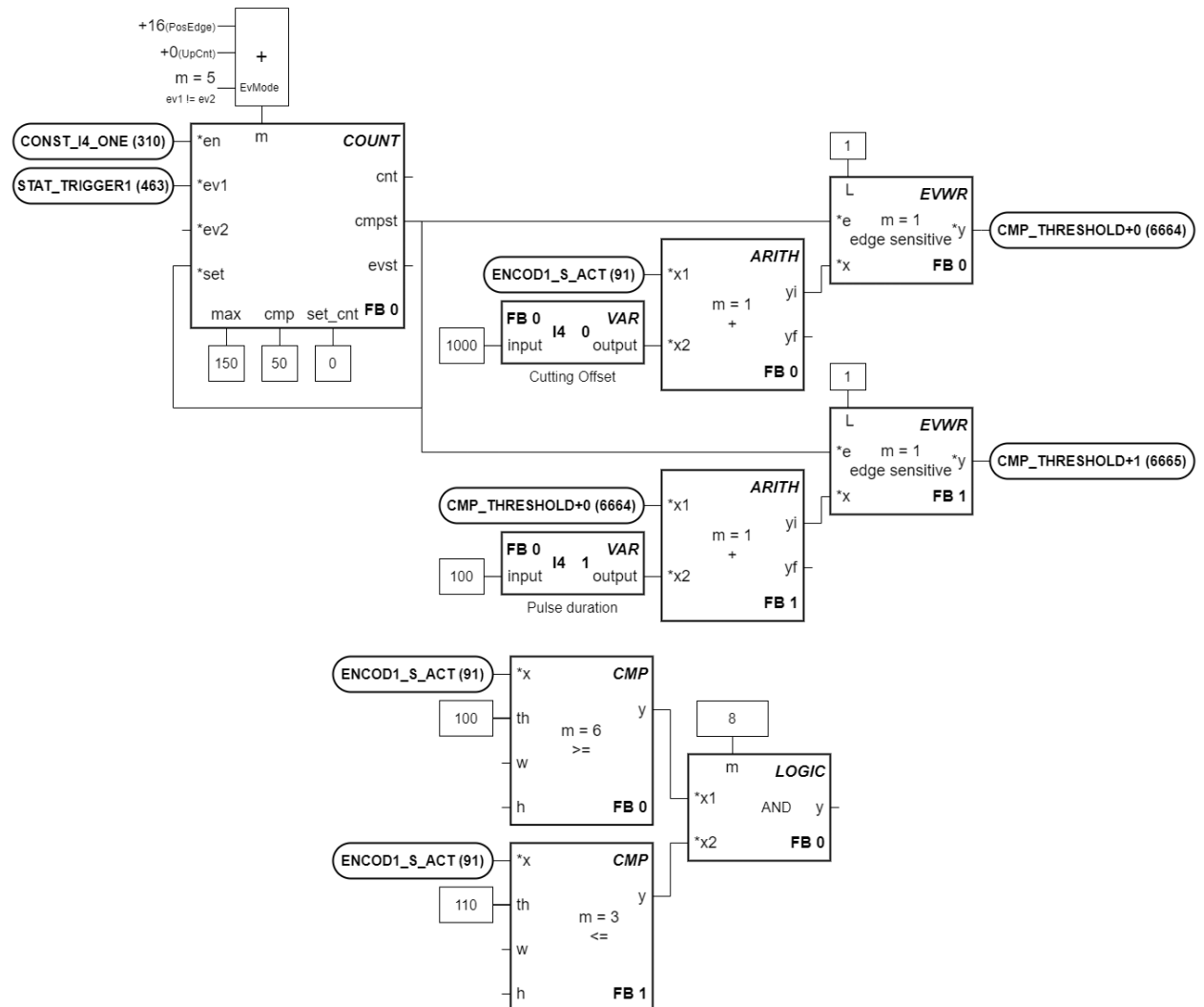


Figure 15: Solution: Group exercise - Flying saw - Method 1

Parameter table

COUNT - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	COUNT_MODE+0	For counting holes
COUNT_ENABLE_PARID+0	13904	CONST_I4_ONE	Counting always enable
COUNT_EV1_PARID+0	13832	STAT_TRIGGER1	Sensing holes to count
COUNT_SET_TRG_PARID+0	13848	COUNT_CMP_STATUS +0	Trigger counter value to set value
COUNT_SET_VALUE+0	13856	0	Counter set value to ZERO
COUNT_MAX_VALUE+0	13864	150	Max count of holes
COUNT_CMP_VALUE+0	13872	50	Cutting after counting holes
COUNT_MODE+0	13824	21	ev1 != ev2 (5) and PosEdge(+16)

VAR - 0 - For cutting offset and pulse duration

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Create FB - Passing values
VAR_I4_0+0	4096	1000	Cutting offset
VAR_I4_1+0	4104	10	Cutting offset + Pulse duration

ARITH - 0 - Calculate cutting offset from current position of motor encoder

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0	Create FB - Offset from current position
ARITH_IN1_PARID+0	3592	ENCOD1_S_ACT	Actual position of motor encoder
ARITH_IN2_PARID+0	3600	VAR_I4_0+0	Cutting offset
ARITH_MODE+0	3584	1	Addition mode

Function block create for writing cutting offset as threshold to comparator

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Writing cutting offset
FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Comparing for cutting offset

EVWR - 0 - Writing cutting offset to threshold of comparator

EVWR_EVENT_PARID+0	4608	COUNT_CMP_STATUS +0	Event of number of holes passed
EVWR_IN_PARID+0	4616	ARITH_VALUE_I4+0	Value of cutting offset + Encoder Position
EVWR_EVENT_LEVEL+0	4624	1	Level of count compare status
EVWR_WR_PARID+0	4632	CMP_THRESHOLD+0	Link to compare threshold
EVWR_MODE+0	4640	1	Edge sensitive mode

ARITH - 1 - Calculate cutting offset+pulse to trigger movement from comparator threshold

FUNCTION_BLOCK_CREATE	777	ARITH_MODE+1	Creating FB - Value till pulse duration
ARITH_IN1_PARID+1	3593	CMP_THRESHOLD+0	Cutting offset + Encoder Position
ARITH_IN2_PARID+1	3601	VAR_I4_1+0	Pulse duration for trigger movement
ARITH_MODE+1	3585	1	Addition mode

Function block create for writing cutting offset + Pulse to trigger movement as threshold to comparator

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+1	Writing cutting offset + Pulse duration
FUNCTION_BLOCK_CREATE	777	CMP_MODE+1	Comparing for cutting offset + Pulse duration

EVWR - 1 - Writing cutting offset + Pulse to trigger movement to threshold of comparator

EVWR_EVENT_PARID+1	4609	COUNT_CMP_STATUS +0	Event of number of holes passed
EVWR_IN_PARID+1	4617	ARITH_VALUE_I4+1	Value till pulse duration
EVWR_EVENT_LEVEL+1	4625	1	Level of count compare status
EVWR_WR_PARID+1	4633	CMP_THRESHOLD+1	Link to compare threshold
EVWR_MODE+1	4641	1	Edge sensitive mode

CMP - 0 - Checking motor encoder position has crossed cutting position

CMP_IN_PARID+0	6656	ENCOD1_S_ACT	Encoder actual position
CMP_THRESHOLD+0	6664	1000	Cutting position
CMP_MODE+0	6688	6	in >= th

CMP - 1 - Checking motor encoder position is still in range of cutting position

CMP_IN_PARID+1	6657	ENCOD1_S_ACT	Encoder actual position
CMP_THRESHOLD+1	6665	1100	Cutting offset + Pulse duration

CMP_MODE+1	6689	3	in <= th
LOGIC - 0 - Generating pulse to trigger movement			
FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Creating - To generate pulse
LOGIC_IN1_PARID+0	3080	CMP_VALUE+0	Value high from cutting offset
LOGIC_IN2_PARID+0	3088	CMP_VALUE+1	Value high till pulse end
LOGIC_MODE+0	3072	8	AND mode to generate pulse

8.3.2 Method 2

About the solution

COUNT_MODE is set to 21 (Ev1 != Ev2, PosEdge) to count holes. COUNT_CMP_VALUE is used to configure the number of holes and COUNT_CMP_STATUS will be high once the count for the number of holes reaches the configured value. COUNT_CMP_STATUS is also set as an input to COUNT_SET_TRG_PARID to reset the counter value so COUNT_CMP_STATUS will be high only for one cycle of scan and the COUNT function block is ready to count holes from COUNT_SET_VALUE (0).

CAMCON_MODE is set to 5; this mode considers the event to start the interval for cam generation. CAMCON_CAM_S1 is the cutting offset from a hole and CAMCON_CAM_S2 is cutting the offset + pulse duration to generate a pulse to trigger flying saw movement .

Schematic

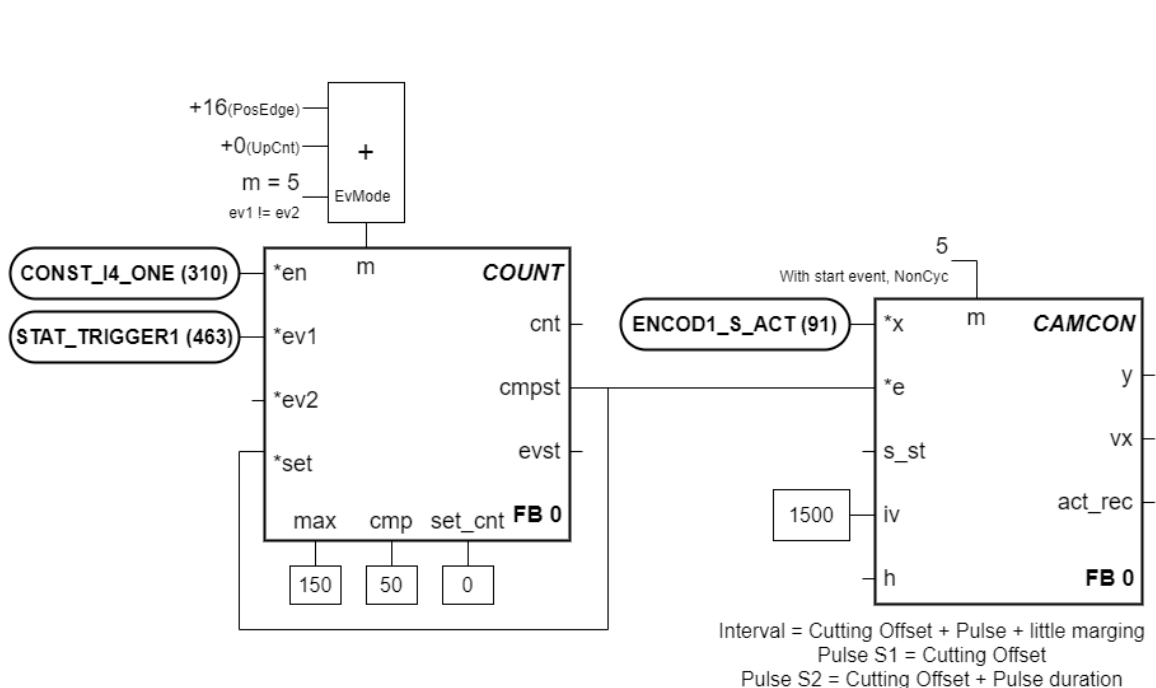


Figure 16: Solution: Group exercise - Flying saw - Method 2

Parameter table

COUNT - 0 - Variable value passing			
FUNCTION_BLOCK_CREATE	777	COUNT_MODE+0	For counting holes
COUNT_ENABLE_PARID+0	13904	CONST_I4_ONE	Counting always enable
COUNT_EV1_PARID+0	13832	STAT_TRIGGER1	Sensing holes to count
COUNT_SET_TRG_PARID+0	13848	COUNT_CMP_STATUS +0	Trigger counter value to set value
COUNT_SET_VALUE+0	13856	0	Counter set value to ZERO

COUNT_MAX_VALUE+0	13864	150	Max count of holes
COUNT_CMP_VALUE+0	13872	50	Cutting after counting holes
COUNT_MODE+0	13824	21	ev1 != ev2 (5) and PosEdge(+16)

CAMCON - 0 - Generate pulse to trigger movement

FUNCTION_BLOCK_CREATE	777	CAMCON_MODE+0	Create FB to generate pulse
CAMCON_IN_PARID+0	10760	ENCOD1_S_ACT	Cutting offset
CAMCON_EV_PARID+0	10864	COUNT_CMP_STATUS +0	Event to start CAMCON
CAMCON_S_START+0	10768	0	No shift start interval
CAMCON_S_IV+0	10776	1110	Cutting offset + Pulse + Margin
CAMCON_MAX_CAM+0	10816	1	Only 1 cam required
CAMCON_CAM_INDEX+0	10824	1	Index of first cam
CAMCON_CAM_S1+0	10832	1000	Cutting offset start
CAMCON_CAM_S2+0	10840	1100	End of pulse duration
CAMCON_MODE+0	10752	5	With start event and non-cyclic

8.4 Exercise: Torque limiter

About the solution

Torque limiter functionality is used where there is requirement for the torque of the motor to be limited, and which has two modes.

TLIM_MODE as 1 (static) and 2 (cyclic). In brief, the static value is fixed, and therefore it has to be changed logically on the ACOPOS drive or by a parameter download via PLC logic. On the ACOPOS drive, the EVWR function block can be used to write/overwrite/change the value of parameters. In cyclic mode, the user needs to assign a ParID and based on the value of the ParID torque, the limit will change .

Description	Torque limit in percentage (UI1 data type)	Torque limit in Nm (R4 data type)
Maximum acceleration torque in positive direction	LIM_T1_POS_OVER	LIM_T1_POS
Maximum deceleration torque in positive direction	LIM_T2_POS_OVER	LIM_T2_POS
Maximum acceleration torque in negative direction	LIM_T1_NEG_OVER	LIM_T1_NEG
Maximum deceleration torque in negative direction	LIM_T2_NEG_OVER	LIM_T2_NEG

Table 1: TLIM_MODE = 1 (static)

Description	Torque limit in Nm as ParID
Maximum acceleration torque in positive direction	LIM_T1_POS_PARID
Maximum deceleration torque in positive direction	LIM_T2_POS_PARID
Maximum acceleration torque in negative direction	LIM_T1_NEG_PARID
Maximum deceleration torque in negative direction	LIM_T2_NEG_PARID

Table 2: TLIM_MODE = 2 (cyclic)

Based on position

In this case, the position controller set position of axis PCTRL_S_SET is taken into consideration and it is compared with threshold value. The output of compare block CMP_VALUE has two values, 0 or 1.

Based on the actual state index of the cam automat

In this case, the actual state index of the cam automat is used as an input to the MUX function block. Based on the cam automat state index, the user can pass CONST_I4_ZERO or CONST_I4_ONE on so the output of MUX_VALUE_I4 will have 0 or 1 as a value. It could be also possible for the user to pass the VAR value directly for the torque limit.

Static torque limit

MUX has a possible selection range of 0 and 1 depending on which method the user has adopted, based on position based or based on the actual state index of the cam automat. When based on the selector value, MUX passes the value according to the inputs provided either in percentage or in Nm. The MUX output MUX_VALUE_I4 or MUX_VALUE_R4 for torque limit in percentage or in Nm respectively needs to be written continuously to the ParIDs listed above via EVWR function blocks.

Cyclic torque limit

In this method, MUX output MUX_VALUE_R4 needs to be assigned to torque limiter ParIDs as shown above, and this torque limit will be in Nm.

8.4.1 Torque limit in Nm based on axis position

Schematic

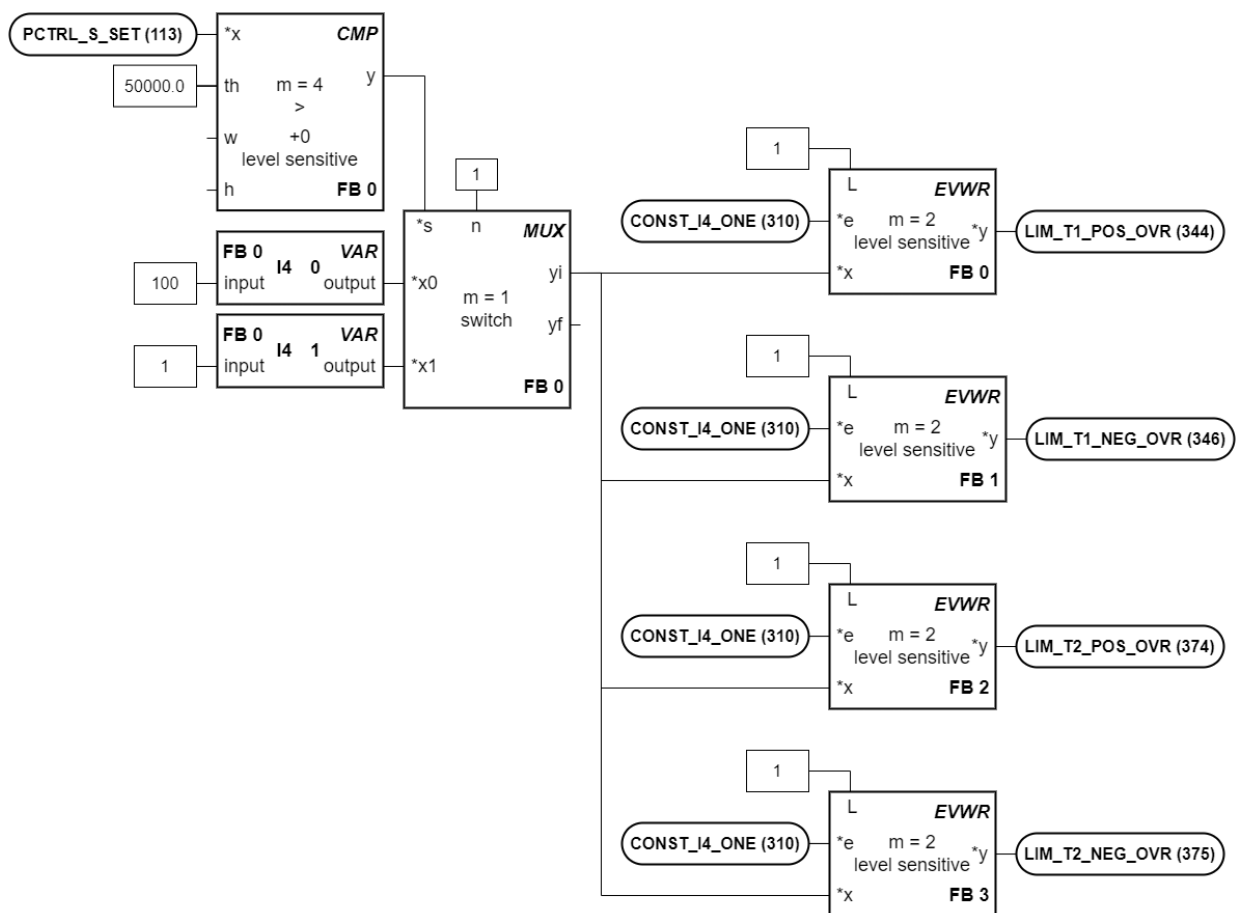


Figure 17: Solution: Group exercise - Torque limiter - Position based - In Nm

Parameter table

Changing torque limit control mode - Static mode - Torque limits in percentage

TLIM_MODE	1480	1	Static mode
LIM_T1_POS_OVR	344	100	Initializing limit to 100%
LIM_T1_NEG_OVR	346	100	Initializing limit to 100%

LIM_T2_POS_OVR	374	100	Initializing limit to 100%
LIM_T2_NEG_OVR	375	100	Initializing limit to 100%

CMP - 0 - Comparing axis set position

FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Axis position comparison
CMP_IN_PARID+0	6656	PCTRL_S_SET	Position controller set position
CMP_THRESHOLD+0	6664	50000.0	Axis position to change torque limit
CMP_MODE+0	6688	4	(in > th)

VAR - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Variable for value passing
VAR_I4_0+0	4096	100	Full torque
VAR_I4_1+0	4104	2	Torque limit to 2%

MUX - 0 - Torque limit value selector

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Create torque limit selector MUX
MUX_SELECTOR_PARID+0	11272	CMP_VALUE+0	Comparing axis position
MUX_SELECTOR_MAX+0	11280	1	Compare has two states 0 and 1
MUX_IN0_PARID+0	11320	VAR_I4_0+0	No torque limit apply
MUX_IN1_PARID+0	11328	VAR_I4_1+0	Torque limit apply
MUX_MODE+0	11264	1	Mode - active as Switch

EVWR - 0 - Torque limit writing to T1_POS_OVR - Acceleration torque override in positive direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Create event write for T1 POS
EVWR_EVENT_PARID+0	4608	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+0	4616	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+0	4624	1	Value/level of constant is 1
EVWR_WR_PARID+0	4632	LIM_T1_POS_OVR	Acc torque override in positive direction
EVWR_MODE+0	4640	2	Continuous writing mode

EVWR - 1 - Torque limit writing to T1_NEG_OVR - Acceleration torque override in negative direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+1	Create event write for T1 NEG
EVWR_EVENT_PARID+1	4609	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+1	4617	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+1	4625	1	Value/level of constant is 1
EVWR_WR_PARID+1	4633	LIM_T1_NEG_OVR	Acc torque override in negative direction
EVWR_MODE+1	4641	2	Continuous writing mode

EVWR - 2 - Torque limit writing to T2_POS_OVR - Deceleration torque override in positive direction

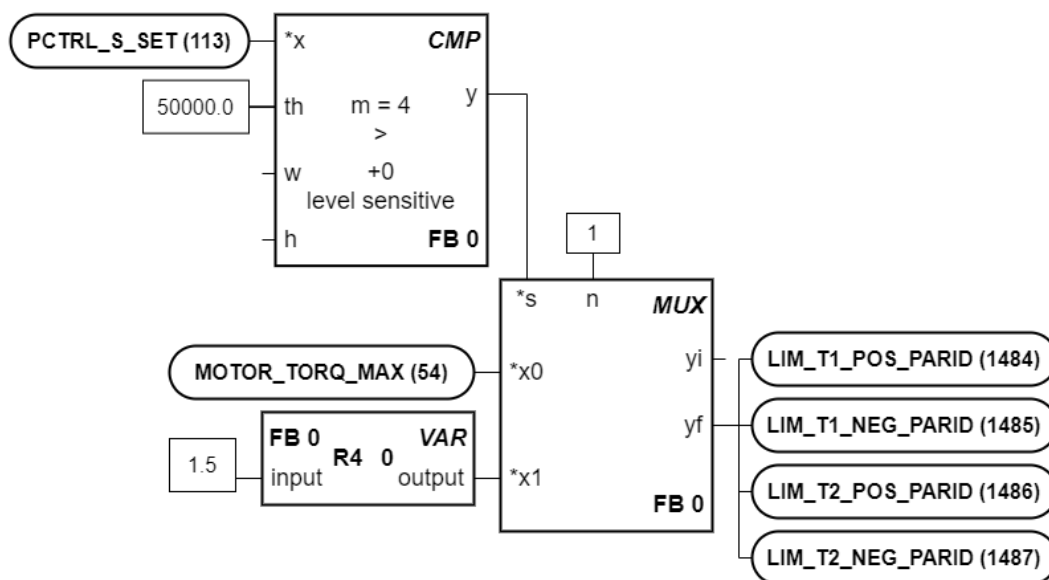
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+2	Create event write for T2 POS
EVWR_EVENT_PARID+2	4610	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+2	4618	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+2	4626	1	Value/level of constant is 1
EVWR_WR_PARID+2	4634	LIM_T2_POS_OVR	Dec torque override in positive direction
EVWR_MODE+2	4642	2	Continuous writing mode

EVWR - 3 - Torque limit writing to T2_NEG_OVR - Deceleration torque override in negative direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+3	Create event write for T2 NEG
EVWR_EVENT_PARID+3	4611	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+3	4619	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+3	4627	1	Value/level of constant is 1
EVWR_WR_PARID+3	4635	LIM_T2_NEG_OVR	Dec torque override in negative direction
EVWR_MODE+3	4643	2	Continuous writing mode

8.4.2 Torque limit in Nm as ParIDs based on axis position

Schematic



Parameter table

CMP - 0 - Comparing axis set position

FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Axis position comparison
CMP_IN_PARID+0	6656	PCTRL_S_SET	Position controller set position
CMP_THRESHOLD+0	6664	50000.0	Axis position to change torque limit
CMP_MODE+0	6688	4	(in > th)

VAR - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Variable for value passing
VAR_R4_0+0	4128	1.5	Torque limit in Nm

MUX - 0 - Torque limit value selector

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Create torque limit selector MUX
MUX_SELECTOR_PARID+0	11272	CMP_VALUE+0	Comparing axis position
MUX_SELECTOR_MAX+0	11280	1	Compare has two states 0 and 1
MUX_IN0_PARID+0	11320	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN1_PARID+0	11328	VAR_R4_0+0	Torque limit apply

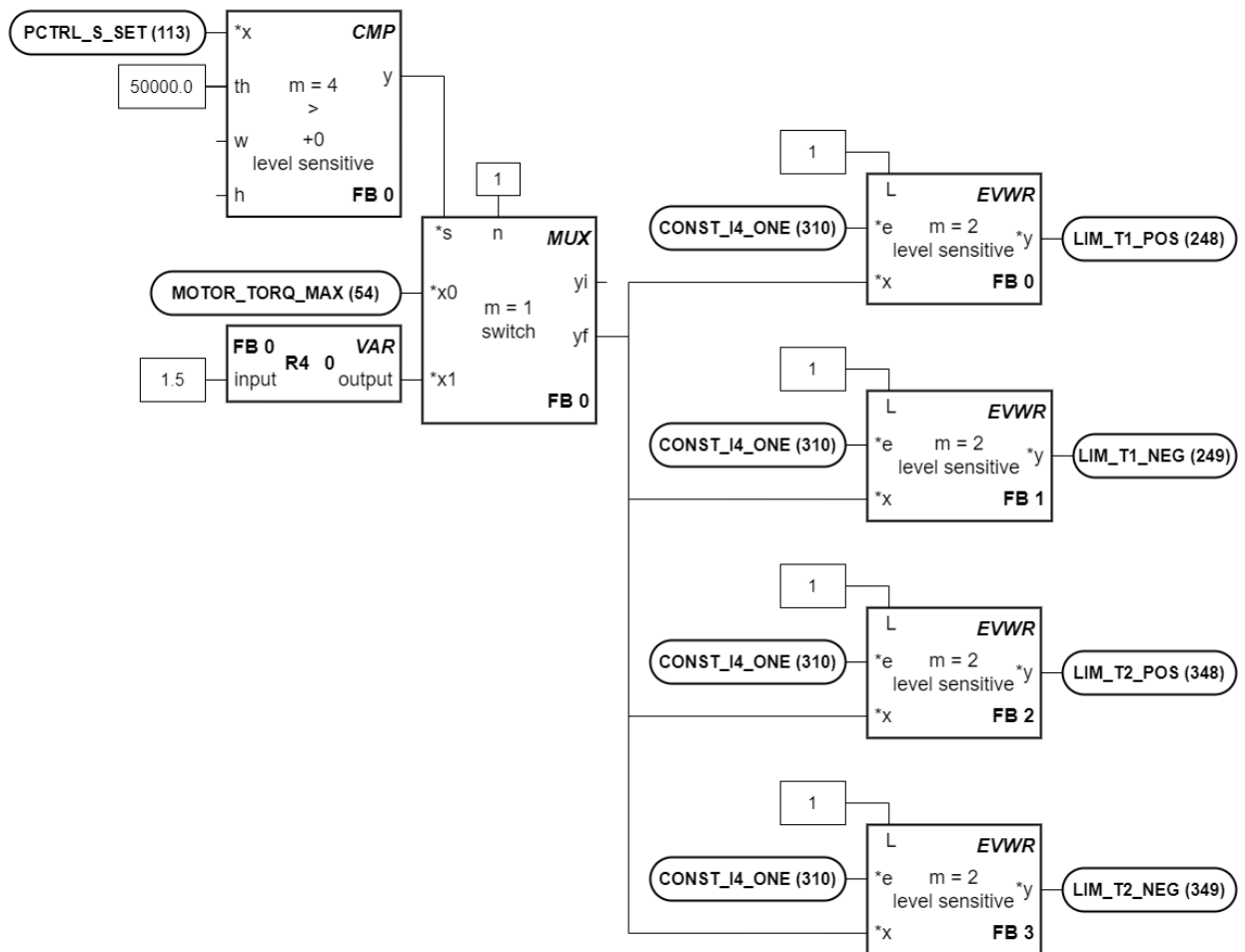
MUX_MODE+0	11264	1	Mode - active as Switch
------------	-------	---	-------------------------

Changing torque limit control mode - Cyclic mode - Torque limits in Nm

TLIM_MODE	1480	2	Cyclic mode
LIM_T1_POS_PARID	1484	MUX_VALUE_R4+0	Torque limit value from selector MUX
LIM_T1_NEG_PARID	1485	MUX_VALUE_R4+0	Torque limit value from selector MUX
LIM_T2_POS_PARID	1486	MUX_VALUE_R4+0	Torque limit value from selector MUX
LIM_T2_NEG_PARID	1487	MUX_VALUE_R4+0	Torque limit value from selector MUX

8.4.3 Torque limit in Nm based on axis position

Schematic



Parameter table

Changing torque limit control mode - Static mode - Torque limits in Nm			
TLIM_MODE	1480	1	Static mode

CMP - 0 - Comparing axis set position

FUNCTION_BLOCK_CREATE	777	CMP_MODE+0	Axis position comparison
CMP_IN_PARID+0	6656	PCTRL_S_SET	Position controller set position
CMP_THRESHOLD+0	6664	50000.0	Axis position to change torque limit
CMP_MODE+0	6688	4	(in > th)

VAR - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Variable for value passing
VAR_R4_0+0	4128	1.5	Torque limit in Nm

MUX - 0 - Torque limit value selector

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Create torque limit selector MUX
MUX_SELECTOR_PARID+0	11272	CMP_VALUE+0	Comparing axis position
MUX_SELECTOR_MAX+0	11280	1	Compare has two states 0 and 1
MUX_IN0_PARID+0	11320	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN1_PARID+0	11328	VAR_R4_0+0	Torque limit apply
MUX_MODE+0	11264	1	Mode - active as Switch

EVWR - 0 - Torque limit writing to T1_POS - Acceleration torque limit in positive direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Create event write for T1 POS
EVWR_EVENT_PARID+0	4608	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+0	4616	MUX_VALUE_R4+0	Selection of torque limit MUX
EVWR_EVENT_LEVEL+0	4624	1	Value/level of constant is 1
EVWR_WR_PARID+0	4632	LIM_T1_POS	Acc torque limit in positive direction
EVWR_MODE+0	4640	2	Continuous writing mode

EVWR - 1 - Torque limit writing to T1_NEG - Acceleration torque limit in negative direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+1	Create event write for T1 NEG
EVWR_EVENT_PARID+1	4609	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+1	4617	MUX_VALUE_R4+0	Selection of torque limit from MUX
EVWR_EVENT_LEVEL+1	4625	1	Value/level of constant is 1
EVWR_WR_PARID+1	4633	LIM_T1_NEG	Acc torque limit in negative direction
EVWR_MODE+1	4641	2	Continuous writing mode

EVWR - 2 - Torque limit writing to T2_POS - Deceleration torque limit in positive direction

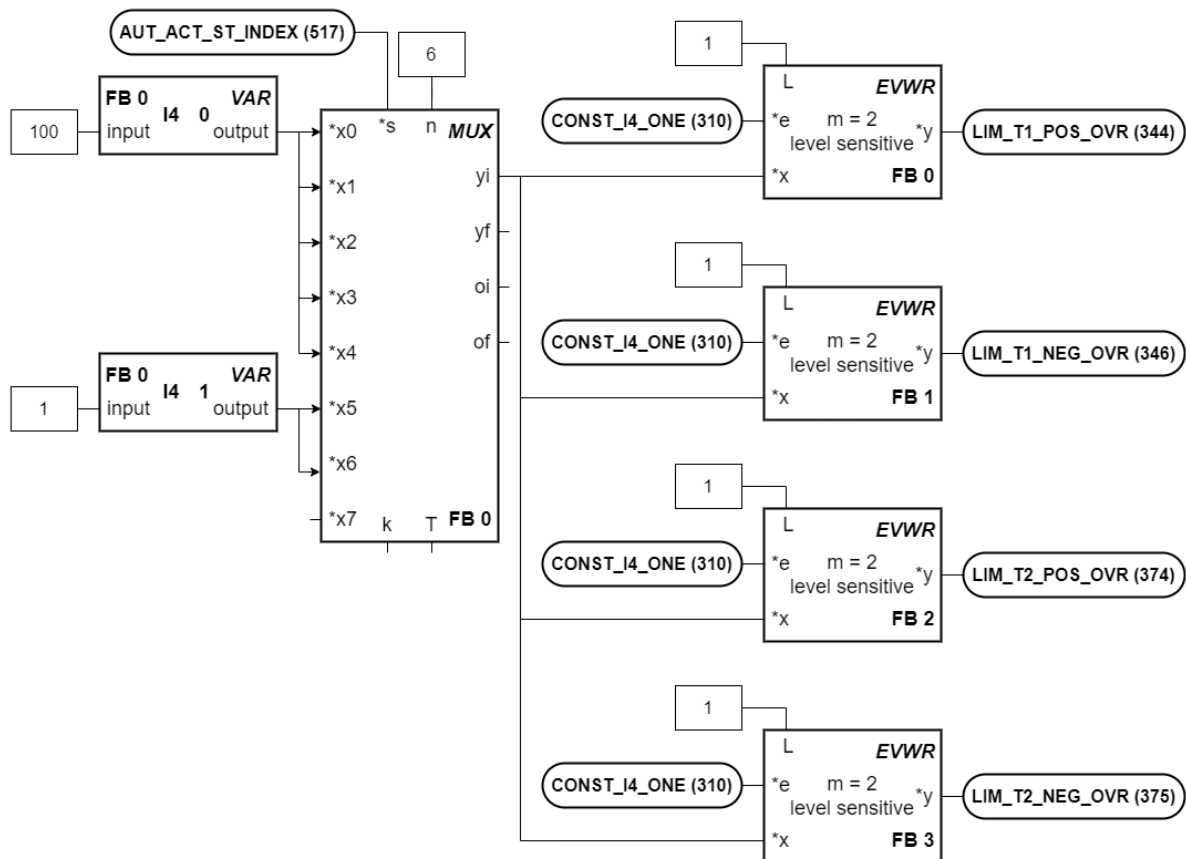
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+2	Create event write for T2 POS
EVWR_EVENT_PARID+2	4610	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+2	4618	MUX_VALUE_R4+0	Selection of torque limit from MUX
EVWR_EVENT_LEVEL+2	4626	1	Value/level of constant is 1
EVWR_WR_PARID+2	4634	LIM_T2_POS	Dec torque limit in positive direction
EVWR_MODE+2	4642	2	Continuous writing mode

EVWR - 3 - Torque limit writing to T2_NEG - Deceleration torque limit in negative direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+3	Create event write for T2 NEG
EVWR_EVENT_PARID+3	4611	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+3	4619	MUX_VALUE_R4+0	Selection of torque limit from MUX
EVWR_EVENT_LEVEL+3	4627	1	Value/level of constant is 1
EVWR_WR_PARID+3	4635	LIM_T2_NEG	Dec torque limit in negative direction
EVWR_MODE+3	4643	2	Continuous writing mode

8.4.4 Torque limit in percentage based on cam automat state index

Schematic



Parameter table

Changing torque limit control mode - Static mode - Torque limits in percentage

TLIM_MODE	1480	1	Static mode
LIM_T1_POS_OVR	344	100	Initializing limit to 100%
LIM_T1_NEG_OVR	346	100	Initializing limit to 100%
LIM_T2_POS_OVR	374	100	Initializing limit to 100%
LIM_T2_NEG_OVR	375	100	Initializing limit to 100%

VAR - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	VAR_I4_0+0	Variable for value passing
VAR_I4_0+0	4096	100	Full torque
VAR_I4_1+0	4104	2	Torque limit to 2%

MUX - 0 - Torque limit value selector based on cam automat actual state index

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Create torque limit selector MUX
MUX_SELECTOR_PARID+0	11272	AUT_ACT_ST_INDEX	Cam automat actual state index
MUX_SELECTOR_MAX+0	11280	6	6 states and each state different limit
MUX_IN0_PARID+0	11320	VAR_I4_0+0	No torque limit apply

MUX_IN1_PARID+0	11328	VAR_I4_0+0	No torque limit apply
MUX_IN2_PARID+0	11336	VAR_I4_0+0	No torque limit apply
MUX_IN3_PARID+0	11344	VAR_I4_0+0	No torque limit apply
MUX_IN4_PARID+0	11352	VAR_I4_0+0	No torque limit apply
MUX_IN5_PARID+0	11360	VAR_I4_1+0	Torque limit apply
MUX_IN6_PARID+0	11368	VAR_I4_1+0	Torque limit apply
MUX_MODE+0	11264	1	Mode - active as Switch

EVWR - 0 - Torque limit writing to T1_POS_OVR - Acceleration torque override in positive direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Create event write for T1 POS
EVWR_EVENT_PARID+0	4608	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+0	4616	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+0	4624	1	Value/level of constant is 1
EVWR_WR_PARID+0	4632	LIM_T1_POS_OVR	Acc torque override in positive direction
EVWR_MODE+0	4640	2	Continuous writing mode

EVWR - 1 - Torque limit writing to T1_NEG_OVR - Acceleration torque override in negative direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+1	Create event write for T1 NEG
EVWR_EVENT_PARID+1	4609	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+1	4617	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+1	4625	1	Value/level of constant is 1
EVWR_WR_PARID+1	4633	LIM_T1_NEG_OVR	Acc torque override in negative direction
EVWR_MODE+1	4641	2	Continuous writing mode

EVWR - 2 - Torque limit writing to T2_POS_OVR - Deceleration torque override in positive direction

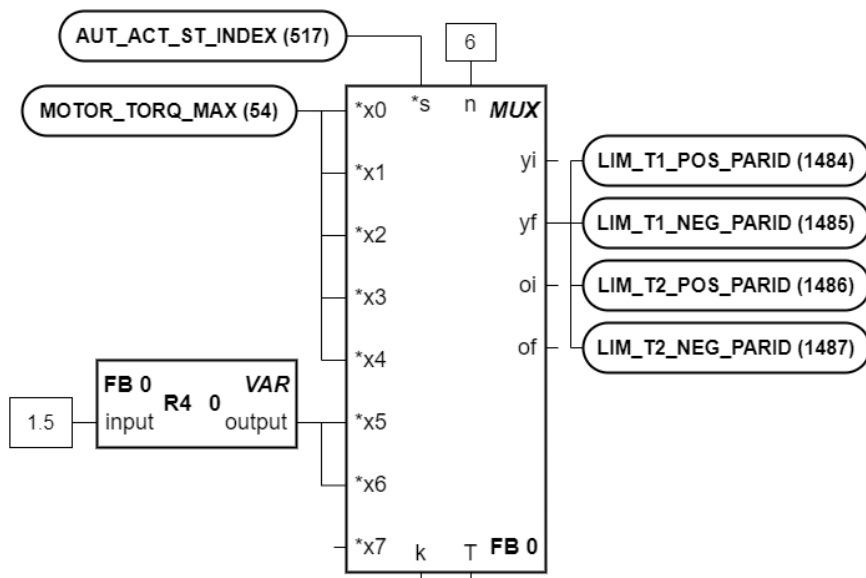
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+2	Create event write for T2 POS
EVWR_EVENT_PARID+2	4610	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+2	4618	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+2	4626	1	Value/level of constant is 1
EVWR_WR_PARID+2	4634	LIM_T2_POS_OVR	Dec torque override in positive direction
EVWR_MODE+2	4642	2	Continuous writing mode

EVWR - 3 - Torque limit writing to T2_NEG_OVR - Deceleration torque override in negative direction

FUNCTION_BLOCK_CREATE	777	EVWR_MODE+3	Create event write for T2 NEG
EVWR_EVENT_PARID+3	4611	CONST_I4_ONE	Constant for value ONE (1)
EVWR_IN_PARID+3	4619	MUX_VALUE_I4+0	Selection of torque override from MUX
EVWR_EVENT_LEVEL+3	4627	1	Value/level of constant is 1
EVWR_WR_PARID+3	4635	LIM_T2_NEG_OVR	Dec torque override in negative direction
EVWR_MODE+3	4643	2	Continuous writing mode

8.4.5 Torque limit in Nm as ParIDs based on cam automat state index

Schematic



Parameter table

VAR - 0 - Variable value passing

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Variable for value passing
VAR_R4_0+0	4128	1.5	Torque limit in Nm

MUX - 0 - Torque limit value selector based on cam automat actual state index

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Create torque limit selector MUX
MUX_SELECTOR_PARID+0	11272	AUT_ACT_ST_INDEX	Cam automat actual state index
MUX_SELECTOR_MAX+0	11280	6	6 states and each state different limit
MUX_IN0_PARID+0	11320	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN1_PARID+0	11328	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN2_PARID+0	11336	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN3_PARID+0	11344	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN4_PARID+0	11352	MOTOR_TORQ_MAX	No torque limit apply
MUX_IN5_PARID+0	11360	VAR_R4_0+0	Torque limit apply
MUX_IN6_PARID+0	11368	VAR_R4_0+0	Torque limit apply
MUX_MODE+0	11264	1	Mode - active as Switch

Changing torque limit control mode - Cyclic mode - Torque limits in Nm

TLIM_MODE	1480	2	Cyclic mode
LIM_T1_POS_PARID	1484	MUX_VALUE_R4+0	Torque limit value from selector MUX
LIM_T1_NEG_PARID	1485	MUX_VALUE_R4+0	Torque limit value from selector MUX
LIM_T2_POS_PARID	1486	MUX_VALUE_R4+0	Torque limit value from selector MUX

LIM_T2_NEG_PARID	1487	MUX_VALUE_R4+0	Torque limit value from selector MUX
------------------	------	----------------	--------------------------------------

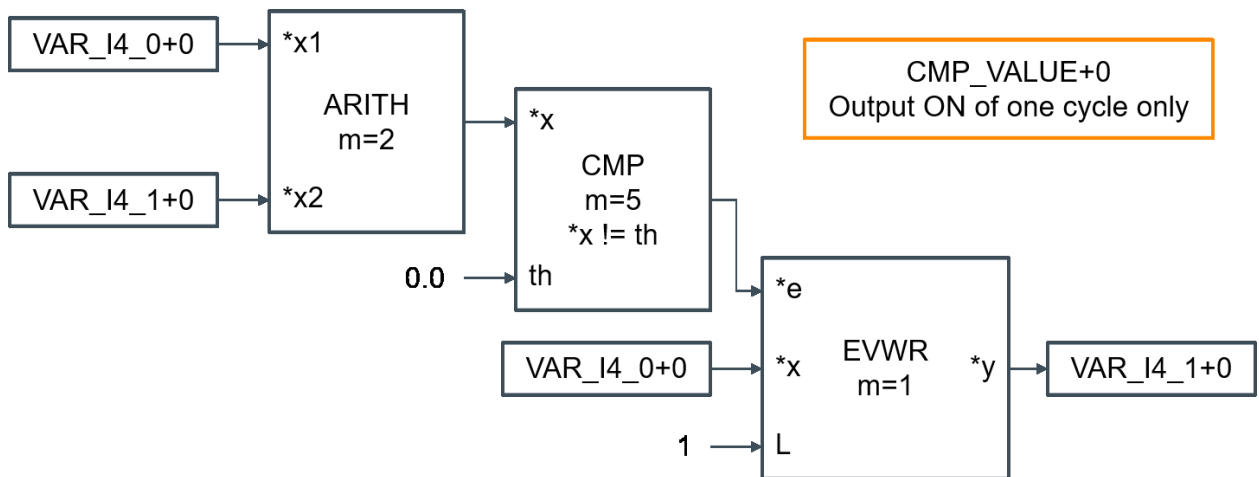
8.5 Solution - Task/Exercise

8.5.1 Old <> New value generate pulse for one cycle

About the solution

ARITH FB0 subtracts VAR_I4_0+0 (4096) and VAR_I4_1+0 (4104), so a difference will be generated. ARITH_VALUE_I4+0 (3608) is provided as input to CMP FB0 with mode 5 (IN != TH), which compares this to the threshold and generates output CMP_VALUE+0 (6696) when the input is not equal to the threshold. CMP_VALUE+0 (6696) is used as an event ParID for EVWR FB0 and an event ParID1 for COUNT FB0. COUNT FB0 will increment the counter value by 1 with every scan. However, EVWR FB0 will write value of VAR_I4_0+0 (4096) to VAR_I4_1+0 (4104) and, in next scan cycle, the ARITH FB output will be zero. Eventually, it will turn CMP_VALUE+0 (6696) to zero as well. Updated counter value (COUNT_VALUE+0 (13880)) will trigger interpolation between the old VAR_I4_0+0 (4096) value and the new.

Schematic



Parameter table

VAR - 0 - Creating and configuring

FUNCTION_BLOCK_CREATE	777	← VAR_I4_0+0	Creates VAR FB0
VAR_I4_0+0	4096	← 0	Initializing I4 var 0 FB0
VAR_I4_1+0	4104	← 0	Initializing I4 var 1 FB0

ARITH - 0 - Creating, configuring and defining the connection structure

FUNCTION_BLOCK_CREATE	777	← ARITH_MODE+0	Creates ARITH FB0
ARITH_IN1_PARID+0	3592	← VAR_I4_0+0	Link input 1 with user set value
ARITH_IN2_PARID+0	3600	← VAR_I4_1+0	Link input 2 with copy user set value
ARITH_MODE+0	3584	← 2	Enable FB in mode subtraction

CMP - 0 - Creating, configuring and defining the connection structure

FUNCTION_BLOCK_CREATE	777	← CMP_MODE+0	Creates CMP FB0
CMP_IN_PARID+0	6656	← ARITH_VALUE_I4+0	Link input with I4 ARITH output
CMP_THRESHOLD+0	6664	← 0.0	Define level
CMP_MODE+0	6688	← 5	in != th

EVWR - 0 - Creating, configuring and defining the connection structure

FUNCTION_BLOCK_CREATE	777	← EVWR_MODE+0	Creates EVWR FB0
-----------------------	-----	---------------	------------------

EVWR_EVENT_PARID+0	4608	← CMP_VALUE+0	Link event with compare output fb0
EVWR_IN_PARID+0	4616	← VAR_I4_0+0	Link input 1 with User set value
EVWR_EVENT_LEVEL+0	4626	← 1	Event level high of compare output
EVWR_WR_PARID+0	4632	← VAR_I4_1+0	Link input 1 with copy User set value
EVWR_MODE+0	4640	← 1	Enables the FB with mode "edge sensitive"

8.5.2 Rms, Min and Max of Actual quadrature stator current

Rising edge trigger and cycle time counter

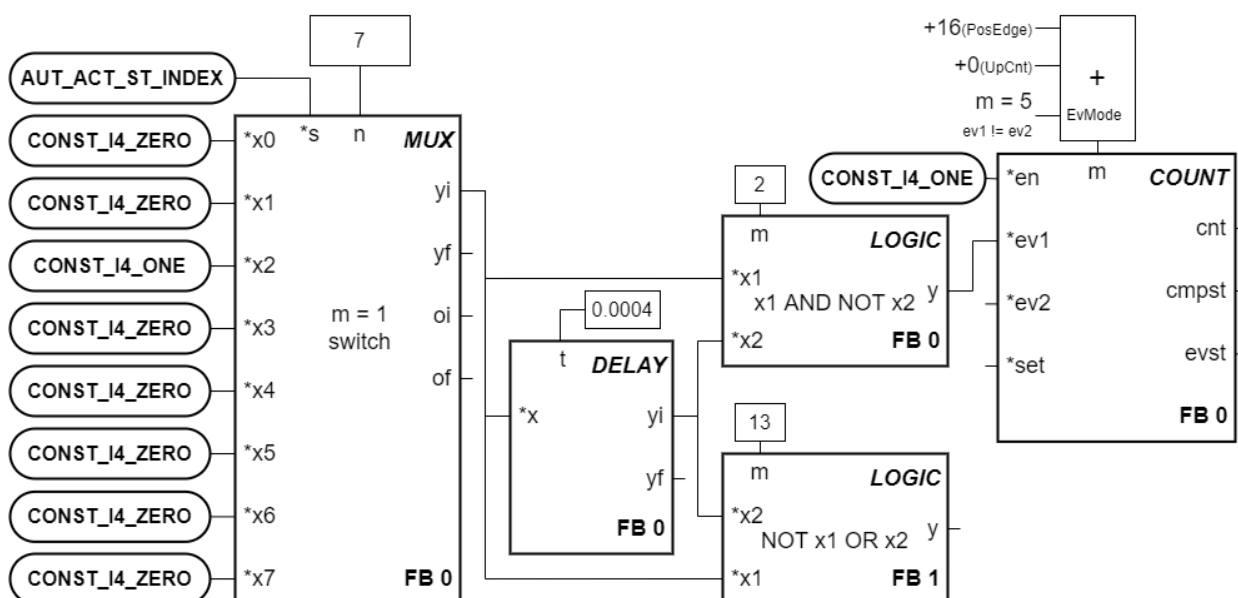
About the solution

MUX FB0 works as a state machine. Here, it is considered that movement of the axis starts from state index 2. When the selector value is 2, MUX will provide output CONST_I4_ONE as a value of 1. The output of MUX FB0 is a link to LOGIC FB0, LOGIC FB1 and DELAY FB0 as well. DELAY FB0 delays the input by 400 μs (in short one scan cycle) and the function block output is linked to LOGIC FB0 and LOGIC FB1.

LOGIC FB0 mode is selected such that the output of LOGIC FB0 will be a rising edge at the start of movement. LOGIC FB1 mode is selected such that the output of LOGIC FB1 will be high until the next movement starts. However, the output will not be high at the rising edge when movement starts.

COUNT FB0 works as an edge counter to provide an update that the last cycle result is available. The user can decide whether the last cycle result should be discarded or kept based on the machine condition.

Schematic



Parameter table

MUX - 0 - State index comparison

FUNCTION_BLOCK_CREATE	777	MUX_MODE+0	Creating FB MUX
MUX_SELECTOR_PARID+0	11272	AUT_ACT_ST_INDEX	Cam automat actual state index
MUX_SELECTOR_MAX+0	11280	7	Max cam automat states
MUX_IN0_PARID+0	11320	CONST_I4_ZERO	State 0 - OP 0 - base State
MUX_IN1_PARID+0	11328	CONST_I4_ZERO	State 1 - OP 0 - Wait state for move
MUX_IN2_PARID+0	11336	CONST_I4_ONE	State 2 - OP 1 - Movement started
MUX_IN3_PARID+0	11344	CONST_I4_ZERO	State 3 - OP 0
MUX_IN4_PARID+0	11352	CONST_I4_ZERO	State 4 - OP 0

MUX_IN5_PARID+0	11360	CONST_I4_ZERO	State 5 - OP 0
MUX_IN6_PARID+0	11368	CONST_I4_ZERO	State 6 - OP 0
MUX_IN7_PARID+0	11376	CONST_I4_ZERO	State 7 - OP 0
MUX_MODE+0	11264	1	Mode - active as Switch

DELAY - 0 - Delaying signal to generate rising edge

FUNCTION_BLOCK_CREATE	777	DELAY_IN_PARID+0	Creating FB DELAY
DELAY_IN_PARID+0	7168	MUX_VALUE_I4+0	MUX output to delay
DELAY_TIME+0	7176	0.0004	Delay by 400µs

LOGIC - 0 - Rising edge for starting of movement

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+0	Creating FB LOGIC
LOGIC_IN1_PARID+0	3080	MUX_VALUE_I4+0	Start of movement
LOGIC_IN2_PARID+0	3088	DELAY_VALUE_I4+0	Delayed start of movement
LOGIC_MODE+0	3072	2	OP = IN1 AND NOT IN2

LOGIC - 1 - Invert of LOGIC 0 - Signal of cycle in progress

FUNCTION_BLOCK_CREATE	777	LOGIC_MODE+1	Creating FB LOGIC
LOGIC_IN1_PARID+1	3081	MUX_VALUE_I4+0	Start of movement
LOGIC_IN2_PARID+1	3089	DELAY_VALUE_I4+0	Delayed start of movement
LOGIC_MODE+1	3073	13	OP = NOT IN1 OR IN2

COUNT - 0 - Cycle counter

FUNCTION_BLOCK_CREATE	777	COUNT_MODE+0	Creating FB COUNT
COUNT_ENABLE_PARID+0	13904	CONST_I4_ONE	Counting always enable
COUNT_EV1_PARID+0	13832	LOGIC_VALUE+0	Trigger of new cycle started
COUNT_MODE+0	13824	21	EV1 != EV2 + PosEdge + UpCnt

Minimum and maximum of actual quadrature stator current

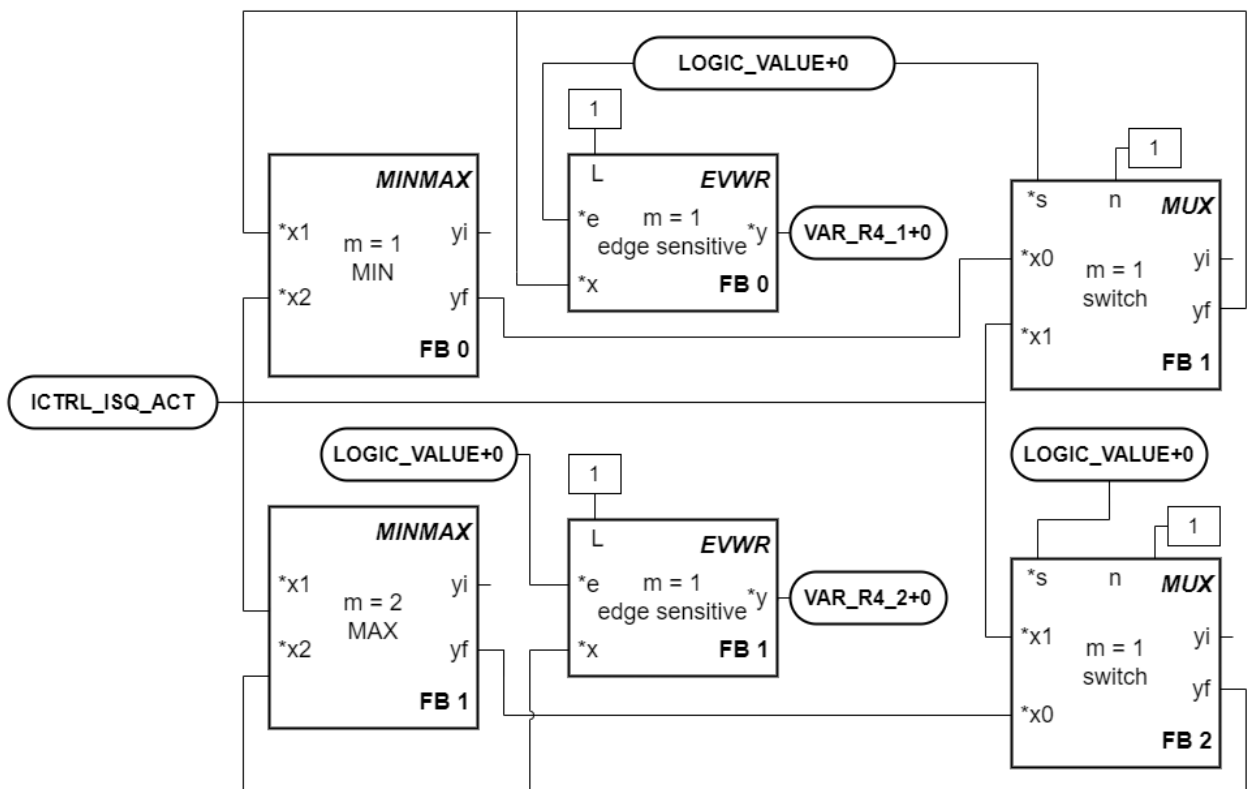
About the solution

VAR_R4_0+0 and VAR_R4_1+0 work to store the minimum and maximum value of the actual quadrature stator current from the last cycle respectively. MINMAX FB0 and FB1 mode are set to take the minimum and maximum from the specified inputs respectively.

MINMAX FB0 has actual quadrature stator current as input and MUX_VALUE_R4+1. MUX_VALUE_R4+1 is the min value of the actual quadrature stator current from the last scan cycle or the last scan value of the actual quadrature stator current if a new cycle is started. When the event for a new cycle start occurs, EVWR FB0 transfers the value of MUX_VALUE_R4+1 to VAR_R4_0+0, which is the min value of the actual quadrature stator current from the last movement cycle. MUX FB1 works as a selector between the min value and current value of the actual quadrature stator current. When the new cycle starts, the selector value turns to 1 and the output has the current value of the actual quadrature stator current.

MINMAX FB1 has actual quadrature stator current as input and MUX_VALUE_R4+2. MUX_VALUE_R4+2 is the max value of the actual quadrature stator current from the last scan cycle or the last scan value of the actual quadrature stator current if a new cycle is started. When the event for a new cycle start occurs, EVWR FB1 transfers the value of MUX_VALUE_R4+2 to VAR_R4_1+0, which is the max value of the actual quadrature stator current from the last movement cycle. MUX FB2 works as a selector between the max value and current value of the actual quadrature stator current. When the new cycle starts, the selector value turns to 1 and the output has the current value of the actual quadrature stator current.

Schematic



Parameter table

VAR - 0 - Declaration

FUNCTION_BLOCK_CREATE	777	VAR_R4_0+0	Creating FB VAR
VAR_R4_1+0	4128	0.0	Initializing to ZERO
VAR_R4_2+0	4136	0.0	Initializing to ZERO

Minimum - Actual quadrature stator current finding

Function block creation

FUNCTION_BLOCK_CREATE	777	MINMAX_MODE+0	Creating FB MINMAX 0
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+0	Creating FB EVWR 0
FUNCTION_BLOCK_CREATE	777	MUX_MODE+1	Creating FB MUX 1

MINMAX - 0 - Quadrature act current - Min

MINMAX_MODE+0	12288	0	Initializing mode to 0
MINMAX_IN1_PARID+0	12312	ICTRL_ISQ_ACT	Actual quadrature stator current
MINMAX_IN2_PARID+0	12320	MUX_VALUE_R4+1	Past min value from MUX
MINMAX_MODE+0	12288	1	Mode as MINIMUM

EVWR - 0 - Storing last value

EVWR_MODE+0	4640	0	Initializing mode to 0
EVWR_EVENT_PARID+0	4608	LOGIC_VALUE+0	Edge of new cycle started
EVWR_IN_PARID+0	4616	MUX_VALUE_R4+1	Past min value of quad current
EVWR_EVENT_LEVEL+0	4624	1	Level of logic
EVWR_WR_PARID+0	4632	VAR_R4_1+0	Min value of quad current of last cycle
EVWR_MODE+0	4640	1	Edge based writing

MUX - 1 - Last cycle - Min quadrature act current

MUX_SELECTOR_PARID+1	11273	LOGIC_VALUE+0	Edge of new cycle started
MUX_SELECTOR_MAX+1	11281	1	Selector values 0 and 1
MUX_IN0_PARID+1	11321	MINMAX_VALUE_R4+0	Min value of quad current till now
MUX_IN1_PARID+1	11329	ICTRL_ISQ_ACT	Actual quadrature stator current
MUX_MODE+1	11265	1	Mode - active as Switch

Maximum - Actual quadrature stator current finding

Function block creation

FUNCTION_BLOCK_CREATE	777	MINMAX_MODE+1	Creating FB MINMAX 1
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+1	Creating FB EVWR 1
FUNCTION_BLOCK_CREATE	777	MUX_MODE+2	Creating FB MUX 2

MINMAX - 1 - Quadrature act current - Max

MINMAX_MODE+1	12289	0	Initializing mode to 0
MINMAX_IN1_PARID+1	12313	ICTRL_ISQ_ACT	Actual quadrature stator current
MINMAX_IN2_PARID+1	12321	MUX_VALUE_R4+2	Past max value from MUX
MINMAX_MODE+1	12289	2	Mode as MAXIMUM

EVWR - 1 - Storing last value

EVWR_MODE+1	4641	0	Initializing mode to 0
EVWR_EVENT_PARID+1	4609	LOGIC_VALUE+0	Edge of new cycle started
EVWR_IN_PARID+1	4617	MUX_VALUE_R4+2	Past max value of quad current
EVWR_EVENT_LEVEL+1	4625	1	Level of logic
EVWR_WR_PARID+1	4633	VAR_R4_2+0	Max value of quad current of last cycle
EVWR_MODE+1	4641	1	Edge based writing

MUX - 2 - Last cycle - Max quadrature act current

MUX_SELECTOR_PARID+2	11274	LOGIC_VALUE+0	Edge of new cycle started
MUX_SELECTOR_MAX+2	11282	1	Selector values 0 and 1
MUX_IN0_PARID+2	11322	MINMAX_VALUE_R4+1	Max value of quad current till now
MUX_IN1_PARID+2	11330	ICTRL_ISQ_ACT	Actual quadrature stator current
MUX_MODE+2	11266	1	Mode - active as Switch

Code on the PLC after cyclic reading of the ParIDs from the ACOPOS drive for MIN current (VAR_R4_1+0) and MAX current (VAR_R4_2+0) for a movement cycle.

$$\text{Min Current} = \frac{\text{VAR_R4_1+0}}{\sqrt{2}}$$

$$\text{Max Current} = \frac{\text{VAR_R4_2+0}}{\sqrt{2}}$$

RMS value of actual quadrature stator current without square root - Part 1

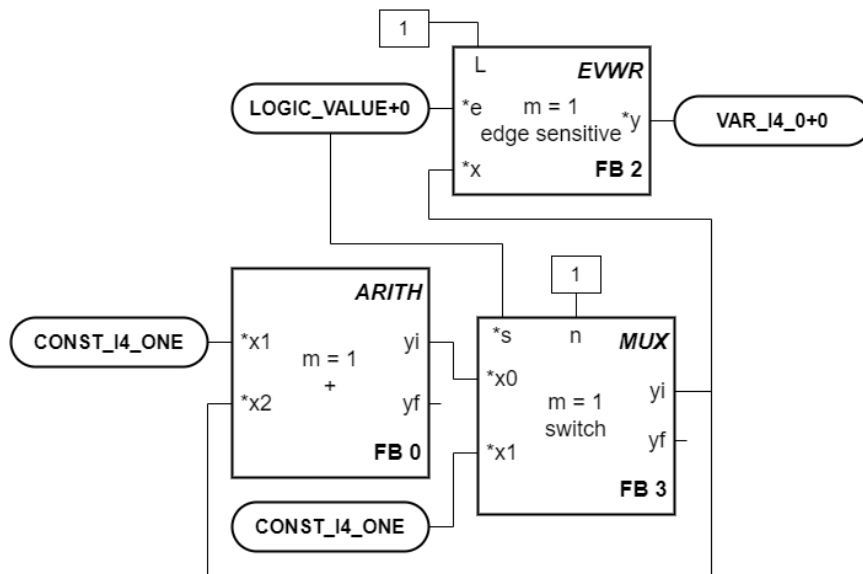
Generating samples counter

About the solution

When the event for a new cycle start occurs (LOGIC_VALUE+0), EVWR FB2 will write the last number of samples to VAR_I4_0+0.

ARITH FBO will simply keep adding 1 to "self routed" via MUX FB3. MUX FB3 works as a reset for the sample counter when the event for a new cycle start occurs (LOGIC_VALUE+0)

Schematic



Parameter table

Generating samples counter

VAR - 0 - Initialization				
VAR_R4_0+0	4128	0.0		Initializing value to 0.0
VAR_I4_0+0	4096	0		Initializing value to 0
Function block creation				
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+2		Creating FB EVWR 2
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+0		Creating FB ARITH 0
FUNCTION_BLOCK_CREATE	777	MUX_MODE+3		Creating FB MUX 3
EVWR - 2 - Storing last counter value				
EVWR_MODE+2	4642	0		Initializing mode to 0
EVWR_EVENT_PARID+2	4610	LOGIC_VALUE+0		Edge of new cycle started
EVWR_IN_PARID+2	4618	MUX_VALUE_I4+3		Number of samples till now
EVWR_EVENT_LEVEL+2	4626	1		Level of logic
EVWR_WR_PARID+2	4634	VAR_I4_0+0		Number of samples in a cycle
EVWR_MODE+2	4642	1		Edge based writing
ARITH - 0 - Sample counter				
ARITH_MODE+0	3584	0		Initializing mode to 0
ARITH_IN1_PARID+0	3592	CONST_I4_ONE		Adding 1 with each scan
ARITH_IN2_PARID+0	3600	MUX_VALUE_I4+3		Number of samples till now
ARITH_MODE+0	3584	1		Mode as Addition
MUX - 3 - Last cycle - Counter value				
MUX_SELECTOR_PARID+3	11275	LOGIC_VALUE+0		Edge of new cycle started
MUX_SELECTOR_MAX+3	11283	1		Selector values 0 and 1
MUX_IN0_PARID+3	11323	ARITH_VALUE_I4+0		Number of samples counter

MUX_IN1_PARID+3	11331	CONST_I4_ONE	Starting value 1 while new cycle
MUX_MODE+3	11267	1	Mode - active as Switch

RMS value of actual quadrature stator current without SquareRoot - Part 2

Sum of the square of the actual quadrature current of a cycle

About solution

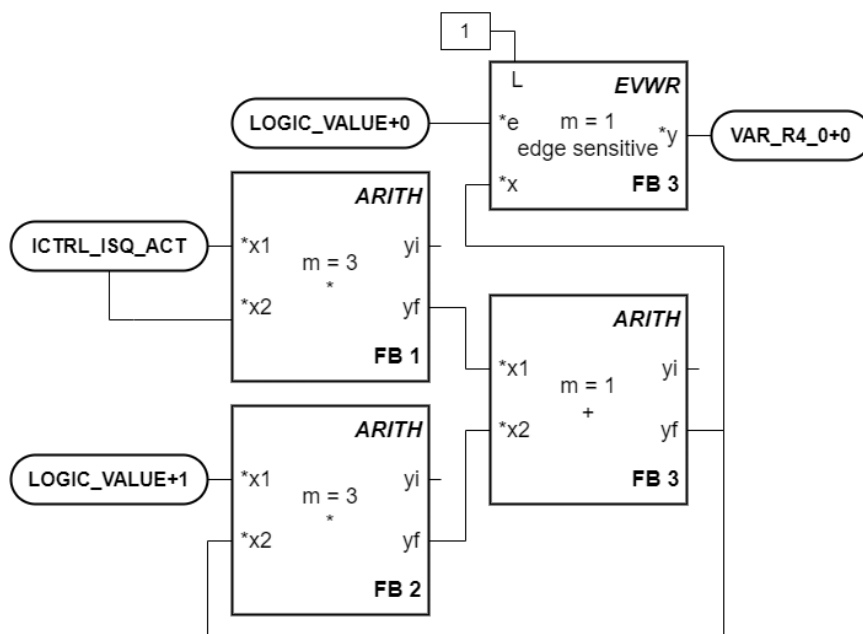
For calculating the RMS value over the selected time period, the value must be squared in order to have a uniform sign (+ve); ARITH FB1 performs continuous squaring of the quadrature current.

ARIH FB2 works as a valid cycle running to define a movement cycle and passes values for cumulative addition; the value from ARITH FB2 will be reset to ZERO when a new cycle starts.

EVWR FB3 will move the value of ARITH FB3 to VAR_R4_0+0 for the full sum of the last cycle.

ARITH FB3 performs cumulative addition for summation of the square of the quadrature current.

Schematic



Parameter table

Sum of the square of the actual quadrature current of a cycle

Function block creation			
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+1	Creating FB ARITH 1
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+2	Creating FB ARITH 2
FUNCTION_BLOCK_CREATE	777	EVWR_MODE+3	Creating FB EVWR 3
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+3	Creating FB ARITH 3
FUNCTION_BLOCK_CREATE	777	ARITH_MODE+4	Creating FB ARITH 4
ARITH - 1 - Generating square of quadrature current			
ARITH_MODE+1	3585	0	Initializing mode to 0
ARITH_IN1_PARID+1	3593	ICTRL_ISQ_ACT	Actual quadrature stator current
ARITH_IN2_PARID+1	3601	ICTRL_ISQ_ACT	Actual quadrature stator current
ARITH_MODE+1	3585	3	Mode as Multiplication
ARITH - 2 - Resetting old value with multiplier			
ARITH_MODE+2	3586	0	Initializing mode to 0

ARITH_IN1_PARID+2	3594	LOGIC_VALUE+1	Sign of cycle running
ARITH_IN2_PARID+2	3602	ARITH_VALUE_R4+3	Sum of square of quadrature current till now
ARITH_MODE+2	3586	3	Mode as Multiplication
EVWR - 3 - Storing last value of sum of square of quadrature Current			
EVWR_MODE+3	4643	0	Initializing mode to 0
EVWR_EVENT_PARID+3	4611	LOGIC_VALUE+0	Edge of new cycle started
EVWR_IN_PARID+3	4619	ARITH_VALUE_R4+3	Sum of square of quadrature current till now
EVWR_EVENT_LEVEL+3	4627	1	Level of logic
EVWR_WR_PARID+3	4635	VAR_R4_0+0	Sum of square of quadrature current in a cycle
EVWR_MODE+3	4643	1	Edge based writing
ARITH - 3 - Cumulative value for cycle			
ARITH_MODE+3	3587	0	Initializing mode to 0
ARITH_IN1_PARID+3	3595	ARITH_VALUE_R4+1	Square of quadrature current
ARITH_IN2_PARID+3	3603	ARITH_VALUE_R4+2	Valid Sum of square of quadrature current
ARITH_MODE+3	3587	1	Mode as Addition

RMS value of actual quadrature stator current without square root - Part 3

RMS of quadrature act current of a cycle without square root

About the solution

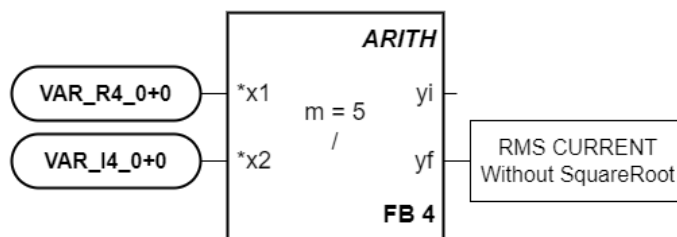
The formula for the RMS of any value is as shown below.

$$X_{RMS} = \sqrt{\frac{x_1^2 + x_2^2 + x_3^2 + x_4^2 + \dots + x_n^2}{n}}$$

We have the sum of the square of the quadrature current in VAR_R4_0+0 and the total number of samples in VAR_I4_0+0.

Since square root is not possible in the ACOPOS function block, the RMS value will be available without the square root in ARITH_VALUE_I4+4 and below you can see what code needs to be executed on the PLC in order to get an exact RMS value of the actual quadrature stator current.

Schematic



Parameter table

RMS of quadrature act current of a cycle without square root

ARITH - 4 - RMS current w/o sqrt () /sqrt(2)			
ARITH_MODE+4	3588	0	Initializing mode to 0
ARITH_IN1_PARID+4	3596	VAR_R4_0+0	Sum of square of quad current in a cycle
ARITH_IN2_PARID+4	3604	VAR_I4_0+0	Number of sample in a cycle

ARITH_MODE+4	3588	5	Division mode
--------------	------	---	---------------

Code on the PLC after cyclic reading of the ParIDs from the ACOPOS drive for RMS current (ARITH_VALUE_R4+4) for a movement cycle.

$$RMS\ Current = \frac{\sqrt{ARITH_VALUE_R4+4}}{\sqrt{2}}$$

9 Summary

ACOPOS function blocks are unique and powerful. They are executed directly on an ACOPOS drive with a cycle time of 400 μ s. These function blocks are available across the entire ACOPOS product range. With this wide range of function blocks, demanding high-precision machine requirements can be achieved with limited resources.

Since ACOPOS function blocks are directly executed on the ACOPOS drive, a massive reduction of CPU and network load is possible with a comparatively lower-capacity CPU allowing demanding applications to be created and fulfilled.

Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. The available seminars are designed to build and expand your professional competence in the field of automation technology. After attending a seminar, you will be able to implement **efficient automation solutions** with B&R systems. This will make it possible for you to secure a **decisive competitive edge** by allowing you and your company to react faster to constantly changing market demands.



B&R offers **standard seminars** at all B&R locations. Services include: seminar documentation, sustainable teaching methods and an automation diploma.



Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online.



The **B&R Tutorial Portal** provides tutorials on a range of subjects and in a variety of languages. Because the tutorials are interactive, they allow content to be learned effectively.



The abbreviation **ETA** stands for "Evaluation and Training for Automation". B&R components are partially pre-wired with sensors and actuators in order to use them as training hardware for seminars and in the laboratory.



Training modules provide the basis for both in-person seminars and self-study. These compact modules follow a clear, uniform structure, where the topics build on one another.

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



Automation Academy



B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

